

ieeditorial

Reservados todos los derechos.

2025 | IEDITORIAL

Queda prohibida, salvo excepción prevista en la ley, cualquier forma de reproducción, distribución, comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de propiedad intelectual. La infracción de los derechos mencionada puede ser constitutiva de delito contra la propiedad intelectual (arts. 270 y s. del Código Penal). El Centro Español de Derechos Reprográficos (www.cedro.org) vela por el respeto de los citados derechos.

Editado por: iEditorial,

Web: www.ieditorial.net

IFCT055PO GESTIÓN DE PROYECTOS DE DESARROLLO DE SOFTWARE

ieeditorial

ÍNDICE

IFCT055PO GESTIÓN DE PROYECTOS DE DESARROLLO DE SOFTWARE 9

Evaluación Inicial	11
Evaluación Inicial	12
Vídeo Presentación	15
Vídeo Presentación	16
Unidad Didáctica: 1 INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE.	17
Introducción	19
Objetivos	21
Mapa Conceptual	22
Conceptos generales de ingeniería del software.	23
Conceptos generales.	32
Definiciones.	33
Ciclo de vida del software.	37
Metodologías de ingeniería del software.	44
Ingeniería del software estructurada.	53
Ingeniería del software orientada a objetos.	70
Ingeniería web.	79
Ingeniería del software asistida por computador.	85
Análisis de sistemas de información.	93
Principios del análisis.	93
Ingeniería de requisitos.	93
Informes del análisis.	96
Análisis de funciones o procesos.	97
Análisis de datos.	98
Análisis orientado a objetos.	99
Diagramas UML para el análisis de sistemas.	102
Diseño de software.	108
Enfoques en el diseño.	109
Niveles de diseño.	110
Principios del diseño.	111
Diseño de la interfaz de usuario.	111

Diseño arquitectónico.	113
Diseño detallado.	114
Técnicas de diseño.	114
Técnicas de diseño de datos.	115
Técnicas de diseño estructurado.	116
Técnicas de diseño orientado a objetos.	118
Diagramas UML para el diseño de software.	121
Prueba del software.	124
Pruebas unitarias.	124
Pruebas del sistema.	125
Pruebas de implantación.	128
Pruebas de aceptación.	129
Pruebas de regresión.	130
Recuerda	132
Autoevaluación	133
Unidad Didáctica: 2 GESTIÓN DE PROYECTOS DE INGENIERÍA DEL SOFTWARE.	137
Introducción	139
Objetivos	140
Mapa Conceptual	141
Introducción a la gestión de proyectos informáticos.	142
Conceptos: Objetivos. Temporalidad. Presupuesto. Ciclo de vida de un producto informático. Especificaciones del Proyecto. El Director del Proyecto. Tipos de Proyectos. Organización de un proyecto.	142
Etapas de un proyecto: Iniciación, Planificación, Ejecución, Control, Cierre.	146
Definición del problema y estrategias de solución.	150
Objetivos a alcanzar.	150
Especificaciones del producto.	151
Los requerimientos de los interesados.	152
Búsqueda de una estrategia de solución y su desarrollo.	153
Planificación del tiempo.	157
Representación gráfica de la ordenación de tareas.	157
De los gráficos de barras al análisis de red.	162
PERT (Performance Evaluation and Review Technique).	164
Camino crítico.	168
Tiempos.	170
Hitos, documentos y revisiones.	171

Ordenar las etapas.	171
Relación de tareas.	173
Diagrama de Gantt.	175
Los hitos y sus fechas límite.	175
La documentación técnica como herramienta de seguimiento de la planificación.	177
Conflictos entre actividades.	179
Factores de productividad.	180
Métricas de productividad del software.	180
Herramientas que mejoran la productividad.	180
Disponibilidad de los recursos.	182
La experiencia y el entrenamiento del equipo de desarrollo.	182
COCOMO (Constructive Cost Model).	184
Recuerda	185
Autoevaluación	186
Unidad Didáctica: 3 EVALUACIÓN.	189
Introducción	190
Objetivos	191
Mapa Conceptual	192
Test de conocimientos.	193
Trabajo voluntario.	194
Trabajo obligatorio.	196
Recuerda	199
Autoevaluación	200
Glosario	203
Glosario	204
Bibliografía	209
Bibliografía	210
Examen	213
Examen	214

ieeditorial

MODULO

IFCT055PO GESTIÓN DE PROYECTOS DE DESARROLLO DE SOFTWARE

- Evaluación Inicial
- Vídeo Presentación
- INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE.
- GESTIÓN DE PROYECTOS DE INGENIERÍA DEL SOFTWARE.
- EVALUACIÓN.
- Glosario
- Bibliografía
- Examen

ieeditorial

IFCT055PO GESTIÓN DE PROYECTOS DE
DESARROLLO DE SOFTWARE
Evaluación Inicial

Evaluación Inicial

Indica si la siguiente afirmación es verdadera o falsa: El software es el conjunto de los componentes físicos.

- a) Verdadera.
- b) Falsa.

Indica si la siguiente afirmación es verdadera o falsa: La ingeniería de software se ocupa del mantenimiento del software

- a) Verdadera.
- b) Falso.

Indica si la siguiente afirmación es verdadera o falsa: Un proceso de software es un conjunto no estructurado de pruebas.

- a) Verdadera.
- b) Falsa.

Que es un CASE.

- a) Conjunto de pruebas para software.
- b) Un método de programación.
- c) Un conjunto de herramientas de programación.

Indica si la siguiente afirmación es verdadera o falsa: Los atributos de software de calidad son: mantenible, confiable y fácil de usar.

- a) Verdadera.
- b) Falsa.

Soluciones :

1:

b) Falsa.

2:

a) Verdadera.

3:

b) Falsa.

4:

c) Un conjunto de herramientas de programación.

5:

a) Verdadera.

ieeditorial

IFCT055PO GESTIÓN DE PROYECTOS DE
DESARROLLO DE SOFTWARE
Vídeo Presentación

Vídeo Presentación

A continuación, un breve resumen de lo que veremos a lo largo de la formación:

[[[Elemento Multimedia]]]

ieeditorial

INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE.

- Introducción
- Objetivos
- Mapa Conceptual
- Conceptos generales de ingeniería del software.
 - Conceptos generales.
 - Definiciones.
 - Ciclo de vida del software.
 - Metodologías de ingeniería del software.
 - Ingeniería del software estructurada.
 - Ingeniería del software orientada a objetos.
 - Ingeniería web.
 - Ingeniería del software asistida por computador.
- Análisis de sistemas de información.
 - Principios del análisis.
 - Ingeniería de requisitos.
 - Informes del análisis.

- Análisis de funciones o procesos.
- Análisis de datos.
- Análisis orientado a objetos.
- Diagramas UML para el análisis de sistemas.
- Diseño de software.
 - Enfoques en el diseño.
 - Niveles de diseño.
 - Principios del diseño.
 - Diseño de la interfaz de usuario.
 - Diseño arquitectónico.
 - Diseño detallado.
 - Técnicas de diseño.
 - Técnicas de diseño de datos.
 - Técnicas de diseño estructurado.
 - Técnicas de diseño orientado a objetos.
 - Diagramas UML para el diseño de software.
- Prueba del software.
 - Pruebas unitarias.
 - Pruebas del sistema.
 - Pruebas de implantación.
 - Pruebas de aceptación.
 - Pruebas de regresión.
- Recuerda
- Autoevaluación

Introducción

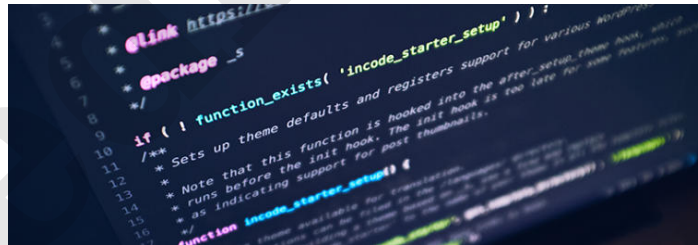
El **software** se define como un **conjunto de programas**, procedimientos y documentación relacionados asociados con un sistema, especialmente un sistema informático.



Características del Software

- El software se desarrolla, no se fabrica.
- El software no se estropea, se deteriora.
- Aunque la industria tiende a ensamblar componentes, la mayoría del software se construye a medida.
- En el software, el recurso principal son las personas.

Historia de la Ingeniería de Software



Primera Fase. Los albores (1945-1955)

- Programar no es una tarea diferenciada del diseño de una máquina.
- Uso de lenguaje máquina y ensamblador.

Segunda Fase. El florecimiento (1955-1965)

- Aparecen multitud de lenguajes, entre ellos Fortran y Cobol.
- Se pensaba que era posible hacer casi todo.

Tercera Fase. La crisis (1965-1970)

- Desarrollo inacabable de grandes programas.
- Ineficiencia, errores, coste impredecible.

Cuarta Fase. Innovación conceptual (1970-1980)

- Fundamentos de programación.
- Se introdujo el concepto de la modularidad y encapsulación.
- Verificación de programas.
- También apareció un sistema para gestionar el hardware: SO.

Quinta Fase. El diseño es el problema (1980-1990)

- Aparecen los lenguajes orientados a objetos.
- Entornos de programación.
- Especificación formal.

Sexta Fase. Prominencia de Internet (1990-1999)

- Auge de la Web y el correo electrónico.
- Aparición de los sistemas distribuidos.
- Surge el software libre.

Séptima Fase. Metodologías ligeras (2000-Actualidad)

- Metodologías más simples y rápidas.
- Aplicaciones móviles.

Crisis del Software

Se desarrolló durante la tercera fase y se debió a que el entorno de desarrollo de sistemas software contaban con:

- Retrasos considerables en la planificación.
- Poca productividad.
- Demandas cada vez más desfasadas frente a las ofertas.
- Elevadas cargas de mantenimiento.
- Baja calidad y fiabilidad del producto.
- Dependencia de los realizadores.

Objetivos

Los objetivos de esta unidad son

- Entender el significado de la Ingeniería de Software.
- Exponer las metas de la Ingeniería de Software.
- Analizar los atributos de un buen Software.
- Conocer el código de ética del Ingeniero de Software.

ieeditorial

Mapa Conceptual

[[[Elemento Multimedia]]]

ieeditorial

(2) El estudio de enfoques como en la declaración anterior.

Otra definición es la de Fritz Bauer, un informático alemán:

La ingeniería de software es el establecimiento y uso de principios sólidos de ingeniería para obtener económicamente un software que sea **confiable y funcione de manera eficiente** en máquinas reales.

Evolución del software

El proceso de desarrollo de un producto de software utilizando principios y métodos de ingeniería de software se conoce como **evolución de software**. Esto incluye el desarrollo inicial del software y su mantenimiento y actualizaciones, hasta que se desarrolle el producto de software deseado, que satisfaga los requisitos esperados.



La evolución comienza desde el proceso de **recopilación de requisitos**. Luego, los desarrolladores crean un prototipo del software previsto y lo muestran a los usuarios para obtener sus comentarios en la etapa inicial del desarrollo del producto de software. Los usuarios sugieren cambios, en los que varias actualizaciones consecutivas y mantenimiento continúan cambiando también. Este proceso cambia al software original, hasta que se logra el software deseado.

Incluso después de que el usuario haya deseado tener el software disponible, la tecnología avanzada y los requisitos cambiantes obligan al producto de software a cambiar en consecuencia. Volver a crear software desde cero e ir uno a uno con los requisitos no es factible. La única solución factible y económica es **actualizar el software** existente para que cumpla con los requisitos más recientes.

Leyes de evolución de software

Lehman ha dado leyes para la evolución del software. Dividió el software en **tres categorías diferentes**:

Tipo S (tipo estático)

Este es un software que funciona estrictamente de acuerdo con especificaciones y soluciones definidas. La solución y el método para lograrlo, ambos se entienden inmediatamente antes de la codificación. El software de tipo S está menos sujeto a cambios, por lo tanto, este es el más simple de todos. Por ejemplo, programa de calculadora para cálculo matemático.

Tipo P (tipo práctico)

Este es un software con una colección de procedimientos. Esto se define exactamente por lo que pueden hacer los procedimientos. En este software, se pueden describir las especificaciones, pero la solución no es obvia al instante. Por ejemplo, software de juegos.

Tipo E (tipo incrustado)

Este software funciona estrechamente como el requisito del entorno del mundo real. Este software tiene un alto grado de evolución ya que hay varios cambios en las leyes, impuestos, etc. en las situaciones del mundo real. Por ejemplo, software de comercio en línea.

Evolución del software E-Type

Lehman ha dado ocho leyes para la evolución del **software E-Type**:

Cambio continuo

Un sistema de software de tipo E debe continuar adaptándose a los cambios del mundo real; de lo contrario, será cada vez menos útil.

Complejidad creciente

A medida que evoluciona un sistema de software tipo E, su complejidad tiende a aumentar a menos que se trabaje para mantenerlo o reducirlo.

Conservación de la familiaridad

La familiaridad con el software o el conocimiento sobre cómo se desarrolló, por qué se desarrolló de esa manera particular, etc., debe mantenerse a cualquier costo para implementar los cambios en el sistema.

Crecimiento continuo

Para un sistema de tipo E destinado a resolver algún problema comercial, su tamaño de implementación de los cambios crece de acuerdo con los cambios en el estilo de vida de la empresa.

Reducción de la calidad

Un sistema de software de tipo E disminuye en calidad a menos que se mantenga rigurosamente y se adapte a un entorno operativo cambiante.

Sistemas de retroalimentación

Los sistemas de software de tipo E constituyen sistemas de retroalimentación de múltiples niveles y múltiples bucles y deben ser tratados como tales para ser modificados o mejorados con éxito.

Autoregulación

Los procesos de evolución del sistema tipo E se autorregulan con la distribución de productos y medidas de proceso cercanas a lo normal.

Estabilidad organizativa

La tasa de actividad global efectiva promedio en un sistema de tipo E en evolución es invariable durante la vida útil del producto.

Paradigmas de software

Los paradigmas de software se refieren a los **métodos y pasos** que se toman al diseñar el software. Hay muchos métodos propuestos y están en funcionamiento hoy, pero necesitamos ver en qué parte de la ingeniería del software se encuentran estos paradigmas. Estos se pueden combinar en varias categorías, aunque cada una de ellas está contenida entre sí.



El paradigma de programación es un subconjunto del paradigma de diseño de software que además es un subconjunto del paradigma de desarrollo de software.

Paradigma de desarrollo de software

Este paradigma se conoce como paradigmas de ingeniería de software donde se aplican todos los conceptos de ingeniería relacionados con el **desarrollo de software**. Incluye varias investigaciones y recopilación de requisitos que ayudan a construir el producto de software. Consiste en las siguientes actividades:

- Recopilación de requisitos.
- Diseño de software.
- Programación.

Paradigma de diseño de software

Este paradigma es parte del desarrollo de software e incluye:

- Diseño.
- Mantenimiento.
- Programación.

Paradigma de programación

Este paradigma está estrechamente relacionado con el aspecto de programación del desarrollo de software. Esto incluye lo siguiente:

- Codificación.
- Pruebas.
- Integración.

Necesidad de ingeniería de software

La necesidad de la ingeniería de software surge debido a la **mayor tasa de cambio** en los requisitos del usuario y el entorno en el que está trabajando el software.

Software grande

Es más fácil construir un muro que una casa o un edificio. De la misma forma, a medida que crece el tamaño del software la ingeniería debe hacerse cargo para darle un proceso científico.

Escalabilidad

Si el proceso del software no se basara en conceptos científicos y de ingeniería, sería más fácil recrear un nuevo software que escalar uno ya existente.

Coste

La industria del hardware, gracias a la gran fabricación ha bajado el precio del hardware. Pero el costo del software sigue siendo alto si no se adapta el proceso de forma adecuada.

Naturaleza dinámica

La naturaleza siempre creciente y adaptable del software depende en gran medida del entorno en el que trabaja el usuario. Si la naturaleza del software siempre está cambiando, se deben realizar nuevas mejoras en la existente. Aquí es donde la ingeniería de software juega un buen papel.

Gestión de calidad

Un mejor proceso de desarrollo de software proporciona un producto de software mejor y de calidad.

Características del buen software



Un producto de software se puede juzgar por lo que ofrece y por lo bien que se puede utilizar. Este software **debe rendir de forma adecuada** en los siguientes aspectos:

- Operacional.
- Transicional.
- Mantenimiento.

En este contexto, se espera que el software **bien diseñado** tenga las siguientes características.

Operacional

Esto nos dice qué tan **bien funciona** el software en las operaciones. Se puede medir en:

- Presupuesto.
- Usabilidad.
- Eficiencia.
- Exactitud.
- Funcionalidad.
- Confianza.
- Seguridad.

Transicional

Este aspecto es importante cuando el software **se mueve de una plataforma a otra**:

- Portabilidad.
- Interoperabilidad.
- Reusabilidad.
- Adaptabilidad.

Mantenimiento

Este aspecto resume la capacidad de un software para **mantenerse en un entorno siempre cambiante**:

- Modularidad.
- Mantenibilidad.
- Flexibilidad.
- Escalabilidad.

En resumen, la ingeniería de software es una rama de la informática, que utiliza conceptos de ingeniería **bien definidos** necesarios para producir productos de software eficientes, duraderos, escalables, dentro del presupuesto y a tiempo.

Relaciona los elementos de la columna Derecha con la columna Izquierda

Estabilidad organizativa

1

El software

2

Sistemas de retroalimentación

3

La tasa de actividad global efectiva promedio en un sistema de tipo E en evolución es invariable durante la vida útil del producto.

Es más que un simple código de programa. Un programa es un código ejecutable, que tiene algún propósito computacional.

Los sistemas de software de tipo E constituyen sistemas de retroalimentación de múltiples niveles y múltiples bucles y deben ser tratados como tales para ser modificados o mejorados con éxito.

Los paradigmas de software se refieren a los métodos y pasos que se toman al diseñar el software. Hay muchos métodos propuestos y están en funcionamiento hoy, pero necesitamos ver en qué parte de la ingeniería del software se encuentran estos paradigmas. Estos se pueden combinar en varias categorías, aunque cada una de ellas está contenida entre sí.

Conceptos generales.

¿Qué es un proceso de software?

- Un conjunto estructurado de actividades cuya meta es el desarrollo o evolución de un software.

¿Qué es un modelo de proceso de software?

- Representación formal y simplificada de un proceso de software, presentada desde una perspectiva específica.

¿Cuáles son los costos de la Ingeniería de Software?

- El costo total de un software esta dividido aproximadamente de la siguiente forma:

- 60 % costos de **desarrollo**
- 40 % costos de **pruebas**

- En el software a la medida los costos de evolución a menudo exceden los costos de desarrollo.

¿Qué es CASE?

- Programas que son usados para dar soporte automatizado a las actividades del proceso de software

¿Cuáles son los atributos del software de calidad?

- El software debe proveer la funcionalidad y desempeño requeridos por el usuario y debe ser mantenible, confiable y aceptable:

- **Mantenible**, el software debe poder evolucionar para continuar cumpliendo con las especificaciones
- **Confiable**, el software no debe causar daños físicos o económicos en el caso de que falle Eficiente, el software no debe desperdiciar los recursos del sistema
- **Aceptable**, el software debe ser aceptado por los usuarios para los que fue diseñado. Debe ser entendible, utilizable y compatible con otros sistemas

Definiciones.

Software Development Life Cycle (SDLC), es una secuencia bien estructurada y estructurada de etapas en ingeniería de software para desarrollar el producto de software previsto.

SDLC proporciona una serie de pasos a seguir para diseñar y desarrollar un producto de software de manera **eficiente**. El marco SDLC incluye los siguientes pasos:

Comunicación

Este es el primer paso donde el usuario inicia la solicitud de un producto de software deseado. Se pone en contacto con el proveedor de servicios e intenta negociar los términos. Presenta su solicitud a la organización proveedora de servicios por escrito.

Recopilación de requisitos

El equipo de desarrollo de software trabaja para llevar a cabo el proyecto. El equipo mantiene conversaciones con varias partes interesadas del dominio del problema y trata de sacar tanta información como sea posible sobre sus requisitos. Los **requisitos** se contemplan y organizan en requisitos de usuario, requisitos del sistema y requisitos funcionales. Los requisitos se recopilan utilizando una serie de prácticas como se indica:

- Estudiando el sistema y el software existentes u obsoletos.
- Realización de entrevistas a usuarios y desarrolladores.
- Refiriéndose a la base de datos.
- Recolectando respuestas de los cuestionarios.

Estudio de factibilidad

Después de reunir los requisitos, el equipo elabora un plan aproximado de proceso de software. En este paso, el equipo analiza si se puede hacer que un software cumpla con todos los requisitos del usuario y si existe alguna posibilidad de que el software **deje de ser útil**. Se descubre si el proyecto es financieramente, práctica y tecnológicamente factible para la organización. Hay muchos algoritmos disponibles, que ayudan a los desarrolladores a concluir la viabilidad de un proyecto de software.

Análisis del sistema

En este paso, los desarrolladores deciden una hoja de ruta de su plan e intentan presentar el mejor modelo de software adecuado para el proyecto. El análisis del sistema incluye la comprensión de las limitaciones del producto de software, los problemas relacionados con el **sistema de aprendizaje** o los

cambios a realizar en los sistemas existentes de antemano, identificar y abordar el impacto del proyecto en la organización y el personal, etc. El equipo del proyecto analiza el alcance del proyecto y planifica el cronograma y recursos en consecuencia.

Diseño de software

El siguiente paso es reducir el conocimiento completo de los requisitos y el análisis en el escritorio y diseñar el producto de software. Las entradas de los usuarios y la información recopilada en la fase de recopilación de requisitos son las **entradas** de este paso. El resultado de este paso viene en forma de dos diseños; Diseño lógico y diseño físico. Los ingenieros producen metadatos y diccionarios de datos, diagramas lógicos, diagramas de flujo de datos y, en algunos casos, pseudocódigos.

Codificación

Este paso también se conoce como fase de programación. La implementación del diseño de software comienza en términos de escribir código de programa en el lenguaje de programación adecuado y desarrollar programas ejecutables sin errores de manera eficiente.



Pruebas

Una estimación dice que el 50% de todo el proceso de desarrollo de software debe ser probado. Los errores pueden arruinar el software desde el nivel

crítico hasta su propia eliminación. Los desarrolladores realizan las **pruebas de software** mientras los desarrolladores codifican y los expertos realizan pruebas exhaustivas en varios niveles de código, como pruebas de módulos, pruebas de programas, pruebas de productos, pruebas internas y pruebas del producto al final del usuario. El descubrimiento temprano de errores y su solución es la clave para un software confiable.

Integración

Es posible que el software deba integrarse con las **bibliotecas, bases de datos y otros programas**. Esta etapa de SDLC está involucrada en la integración de software con entidades del mundo exterior.

Implementación

Esto significa **instalar el software** en las máquinas de los usuarios. A veces, el software necesita configuraciones posteriores a la instalación al final del usuario. El software se prueba para portabilidad y adaptabilidad y los problemas relacionados con la integración se resuelven durante la implementación.

Operación y mantenimiento

Esta fase confirma la operación del software en términos de mayor eficiencia y menos errores. Si es necesario, los usuarios reciben capacitación o se les ayuda con la **documentación** sobre cómo operar el software y cómo mantenerlo operativo. El software se mantiene a tiempo actualizando el código de acuerdo con los cambios que tienen lugar en el entorno o la tecnología del usuario final. Esta fase puede enfrentar desafíos de errores ocultos y problemas no identificados del mundo real.

Disposición

A medida que transcurre el tiempo, el software puede disminuir en el frente del rendimiento. Puede quedar completamente obsoleto o puede necesitar una intensa actualización. Por lo tanto, surge una necesidad apremiante de eliminar una parte importante del sistema. Esta fase incluye el **archivo de datos** y los **componentes** de software requeridos, el cierre del sistema, la planificación de la actividad de disposición y la terminación del sistema en el momento adecuado del final del sistema.

Software Development Life Cycle (SDLC), es una secuencia bien estructurada y estructurada de etapas en ingeniería de software para desarrollar el producto de software previsto.

Verdadero.

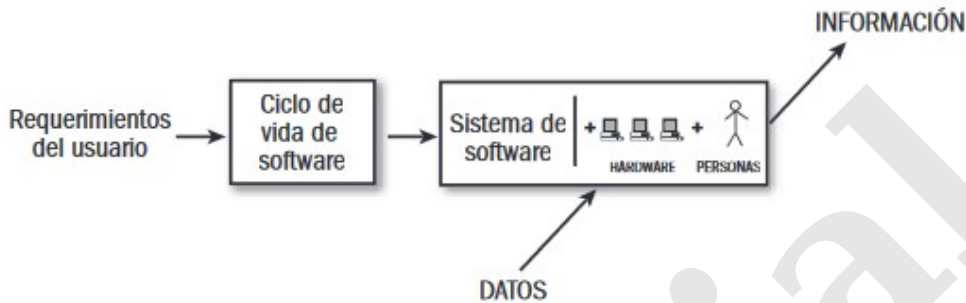
Falso.

Ciclo de vida del software.

El ciclo de vida del desarrollo del software (también conocido como Systems Development Life Cycle) contempla las fases necesarias para validar el desarrollo del software y así garantizar que este cumpla los requisitos para la aplicación y verificación de los procedimientos de desarrollo, asegurándose de que los métodos usados son apropiados. Su origen radica en que es muy costoso rectificar los posibles errores que se detectan tarde en la fase de implementación. Utilizando metodologías apropiadas, se podría detectar a tiempo para que los programadores puedan centrarse en la calidad del software, cumpliendo los plazos y los costes asociados.

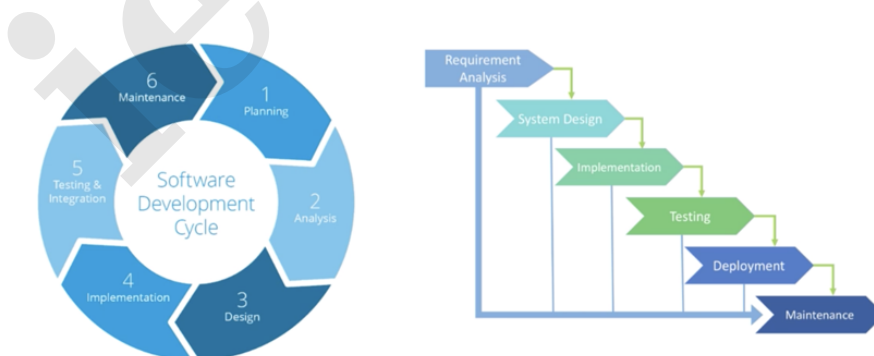
Aunque existen diferentes ciclos de desarrollo de software, la normativa **ISO/IEC/IEEE 12207:2017** establece:

“Un marco común para los procesos del ciclo de vida de los programas informáticos, con una terminología bien definida, a la que pueda remitirse la industria del software. Contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, el funcionamiento, el mantenimiento o la eliminación de sistemas, productos y servicios informáticos. Estos procesos del ciclo de vida se llevan a cabo mediante la participación de los interesados, con el objetivo final de lograr la satisfacción del cliente”.



En detalle, el **Ciclo de Vida** de desarrollo del software consta de las siguientes actividades:

- Planificación y análisis de requerimientos.
- Análisis.
- Diseño.
- Implementación.
- Pruebas e integración.
- Despliegue y mantenimiento.



1. Planificación y análisis de requisitos

Cada modelo de ciclo de vida de desarrollo de software comienza con el

análisis, en el cual las partes interesadas del proceso discuten los requisitos para el producto final. Las partes más importantes del desarrollo de software, la recopilación de requisitos o el análisis de requisitos suelen ser realizadas por los miembros principales del equipo con aportes del cliente, el departamento de ventas, encuestas de mercado y expertos en dominios de la industria. Esta información luego se utiliza para planificar el enfoque básico del proyecto y para realizar un estudio de viabilidad del producto en las áreas económica, operativa y técnica.

El objetivo de esta etapa es la definición detallada de los requisitos del sistema. Además, es necesario asegurarse de que todos los participantes del proceso hayan entendido claramente las tareas y cómo se implementarán todos los requisitos. A menudo, la discusión involucra a los especialistas en control de calidad que pueden interferir el proceso con adiciones incluso durante la etapa de desarrollo si es necesario. La planificación de los requisitos de garantía de calidad y la identificación de los riesgos asociados con el proyecto también se realiza en la etapa de planificación.

El resultado del estudio de factibilidad técnica es definir los diversos **enfoques técnicos** que se pueden seguir para implementar el proyecto con éxito con riesgos mínimos. Después de que los requisitos del cliente se hayan reunido, se crea un documento de alcance en el que se determina y documenta el alcance del proyecto. Sin el plan perfecto, el cálculo de las fortalezas y debilidades del proyecto, el desarrollo de software no tiene sentido.

2. Análisis

Este paso consiste en analizar el rendimiento del software en varias etapas y tomar notas sobre requisitos adicionales. El análisis es muy importante para continuar con el siguiente paso. Se trata de definir y documentar claramente los requisitos del producto y obtener la aprobación del cliente o de los analistas del mercado. Esto se realiza a través de un documento SRS (Especificación de requisitos de software) que consta de todos los requisitos del producto que se diseñarán y desarrollarán durante el ciclo de vida del proyecto. Se refiere a especificar cómo funcionará el software de manera general sin entrar en detalles.

Se basan en incorporar consideraciones de la **implementación tecnológica**, como el hardware, la red, etc. Todas las partes interesadas, incluido el cliente, analizan las diferentes preguntas técnicas que pueden aparecer en esta etapa.

Además, aquí se definen las tecnologías utilizadas en el proyecto, la carga del equipo, las limitaciones, los plazos y el presupuesto. Las decisiones de proyecto más apropiadas se toman de acuerdo con los requisitos definidos. Se determinan los casos de uso para cubrir las funciones que tendrá que realizar el sistema, y se transformarán las entidades determinadas en el análisis de requisitos en clases de diseño, logrando un modelo cercano a la programación orientada a objetos.

Obtener los requisitos y requerimientos de un producto de software es la principal etapa para crearlo. Mientras que los clientes creen que ellos saben lo que el software debe de hacer, se necesita de **habilidad** y experiencia en la ingeniería de software para reconocer requerimientos incompletos, ambiguos o contradictorios. El final del análisis de requerimientos con el cliente se plasma en el documento SRS, Especificación de Requerimientos del Sistema, cuya organización puede venir definida por múltiples estándares, tales como CMMI. De esta manera, se define un diagrama de Entidad/Relación, en el que se plantan las principales entidades que tendrán participación en el desarrollo del software. La captura, análisis y determinación de requerimientos (incluso pruebas de ellos), es una sección crucial. De esta etapa depende en gran medida el logro de los objetivos. Se han creado modelos y diversos procesos de trabajo para estos fines. Pese a que aún no está formalizada, ya se habla de la Ingeniería de requerimientos, como por ejemplo en dos capítulos del libro de Sommerville "Ingeniería del software" llamados "Requerimientos del software" y "Procesos de la Ingeniería de Requerimientos"

La IEEE Std. 830-1998 normaliza el impulso de las especificaciones de necesidades de software (Software Requirements Specification).

3. Diseño

Una vez que se completa el análisis, el paso del diseño se hace cargo, que básicamente es construir la arquitectura del proyecto. Este paso ayuda a eliminar posibles fallos al establecer un estándar e intentar cumplirlo. **SRS** es la referencia para que los arquitectos de productos salgan con la mejor arquitectura para el producto a desarrollar.

Con base en los requisitos especificados en SRS, generalmente se propone y documenta más de un enfoque de diseño para la arquitectura del producto en una DDS (Especificación de documento de diseño). Todas las partes interesadas importantes revisan este DDS y, basándose en diversos parámetros como la evaluación de riesgos, la solidez del producto, la

modularidad del diseño, el presupuesto y las limitaciones de tiempo, se selecciona el mejor enfoque de diseño para el producto.

Un **enfoque** de diseño define claramente todos los módulos arquitectónicos del producto junto con su comunicación y representación del flujo de datos con los módulos externos y de terceros (si los hay). El diseño interno de todos los módulos de la arquitectura propuesta debe definirse claramente con el más mínimo de los detalles en DDS.

4. Desarrollo e implementación

Los ingenieros de software comienzan a escribir el código de acuerdo con los requisitos del cliente. Se puede llevar a cabo un estudio piloto para ver si funciona correctamente, si es así, el software se implementa después de que se haya aprobado su lanzamiento. El código de programación se genera según DDS durante esta etapa. Si el diseño se realiza de manera detallada y organizada, la generación de código se puede lograr sin mucha molestia. Traducir un diseño a código puede llegar a ser la parte más obvia del trabajo de ingeniería del software, pero no tiene que ser necesariamente la que demanda mayor trabajo ni la más compleja.

La complejidad y la duración de esta **etapa** está muy relacionada al o a los lenguajes de programación usados, así como al diseño previamente hecho. Los desarrolladores deben seguir las pautas de codificación definidas por su organización y se utilizan herramientas de programación como compiladores, intérpretes, depuradores, etc. para generar el código. Para la codificación se utilizan diferentes lenguajes de programación de alto nivel como C, C ++, Pascal, Java y PHP. El lenguaje de programación se elige con respecto al tipo de software que se está desarrollando.

Los programadores comienzan aquí con la escritura del código fuente teniendo en cuenta los requisitos previamente definidos. Los administradores del sistema ajustan el entorno del software, los programadores front-end desarrollan la interfaz de usuario del programa y las lógicas para su interacción con el servidor. La programación por sí sola asume cuatro etapas .Desarrollo de algoritmos .Escritura de código fuente .Compilación .Pruebas y depuración.

5. Pruebas e integración

Proceso de buscar defectos o errores en el software creado. Esta etapa suele

ser un subconjunto de todas las etapas, ya que en los modelos SDLC modernos, las actividades de prueba están involucradas principalmente en todas las etapas de **SDLC**. Sin embargo, esta etapa se refiere a la única etapa de prueba del producto donde se informan, rastrean, reparan y vuelven a aprobar los defectos del producto, hasta que el producto alcanza los estándares de calidad especificados en el SRS.

Consiste en comprobar que el software realice correctamente las tareas indicadas en la especificación del problema. Una técnica de prueba es comprobar por separado cada módulo del software, y después probarlo de forma integral, para así alcanzar al objetivo. Se puede llegar a considerar una buena práctica el que las pruebas sean realizadas por alguien diferente al desarrollador que la programó, idealmente un área de pruebas; pese a lo anterior el programador tiene que hacer sus propias pruebas.

En general existen dos grandes maneras de organizar un área de pruebas, la primera es que esté formada por personal inexperto y que desconozca el tema de pruebas, de esta manera se puede evaluar que la **documentación** entregada sea de calidad, que los procesos mostrados son tan claros que cualquiera puede comprenderlos y el software hace las cosas tal y como están propuestas. El segundo enfoque es poseer un área de pruebas conformada por programadores expertos, personas que conocen sin mayores indicaciones en qué condiciones puede errar una aplicación y que pueden poner atención en detalles que personal inexperto no tomaría en cuenta. Todos los defectos de código perdidos durante el desarrollo se detectan aquí, se documentan y se pasan a los desarrolladores para que los solucionen. El proceso de prueba se repite hasta que se eliminen todos los problemas críticos y el flujo de trabajo del software sea estable.

6. Despliegue y mantenimiento

Una vez que el producto se prueba y está listo para implementarse, se lanza formalmente en el mercado apropiado. A veces, el despliegue del producto ocurre en etapas según la estrategia comercial de la organización. El producto puede lanzarse primero en un segmento limitado y probarse en el entorno empresarial real (UAT: prueba de aceptación del usuario). Después de que el producto se lanza al mercado, su mantenimiento se realiza para la base de clientes existente. El mantenimiento del software se realiza para referencia futura. La mejora del software y los nuevos requisitos (solicitudes de cambio) pueden llevar más tiempo del necesario para crear el desarrollo inicial del software. Una vez que el software pasa por todas las etapas sin ningún

problema, debe someterse a un proceso de mantenimiento en el que se mantendrá y actualizará de vez en cuando para adaptarse a los cambios.

Después del lanzamiento de la **nueva versión** del programa, se une el equipo de soporte técnico. Este departamento proporciona comentarios de los usuarios; consultar y apoyar a los usuarios durante el tiempo de explotación. Además, la actualización de los componentes seleccionados se incluye en estafase, para asegurarse de que el software esté actualizado y sea invulnerable a una violación de seguridad.

Se trata de mantener y mejorar el software para arreglar errores descubierto se intentar de tratar con nuevos requisitos. El mantenimiento puede ser de cuatro maneras: Correctivo Corregir errores detectados en el sistema Perfectivo Aumentar la calidad interna de los sistemas Evolutivo Incorporaciones, cambios y eliminaciones necesarias en un producto software para cubrir la expansión o cambio en las necesidades del usuario. Adaptativo Modificaciones que afectan a los entornos en los que el sistema opera, por ejemplo, cambios de configuración del hardware, software de base.



[[[Elemento Multimedia]]]

Una vez que se completa el análisis, el paso del diseño se hace cargo, que básicamente es construir la arquitectura del proyecto. Este paso ayuda a eliminar posibles fallos al establecer un estándar e intentar cumplirlo. SRS es la referencia para que los arquitectos de productos salgan con la mejor arquitectura para el producto a desarrollar. Con base en los requisitos especificados en SRS, generalmente se propone y documenta más de un enfoque de diseño para la arquitectura del producto en una DDS (Especificación de documento de diseño)

Verdadero

Falso

Metodologías de ingeniería del software.

Desde una perspectiva de Ingeniería de Software, una metodología:

- Describe cómo se organiza un proyecto.
- Establece el orden en el que la mayoría de las actividades tienen que realizarse y los enlaces entre ellas.
- Indica cómo tienen que realizarse algunas tareas proporcionando las herramientas concretas e intelectuales.

Con una metodología se intentan cubrir las siguientes necesidades:

- Mejores aplicaciones.
- Mejor proceso de desarrollo.
- Establecer un proceso estándar en una organización.

Hay varios modelos de **ciclo de vida** de desarrollo de software definidos y diseñados que se siguen durante el proceso de desarrollo de software. Estos modelos también se denominan Modelos de proceso de desarrollo de

software. Cada modelo de proceso sigue una serie de pasos únicos según su tipo para garantizar el **éxito** en el proceso de desarrollo de software. Hay varios modelos de desarrollo de software seguidos por las organizaciones.



Modelo en cascada

Este modelo implica terminar cada fase por completo antes de comenzar la siguiente. Cuando cada fase se completa con éxito, se revisa para ver si el proyecto está en marcha y si es factible continuar.

Modelo en forma de V

Este modelo se centra en la ejecución de procesos de manera secuencial, similar al modelo en cascada pero con más importancia en las pruebas. Los procedimientos de prueba se escriben incluso antes del comienzo de la escritura del código. Se genera un plan del sistema antes de comenzar la fase de desarrollo.

Modelo incremental

Este modelo de ciclo de vida implica múltiples ciclos de desarrollo. Los ciclos se dividen en iteraciones más pequeñas. Estas iteraciones pueden administrarse fácilmente y pasar por un conjunto de fases que incluyen requisitos, diseño, implementación y pruebas. Se produce una versión

funcional del software durante la primera iteración, por lo que el software funcional se crea temprano en el proceso de desarrollo.

Estrategias de diseño del sistema

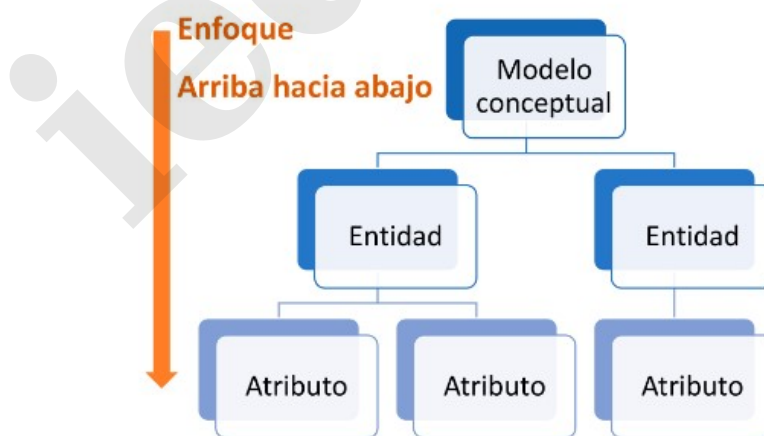
Un buen diseño del sistema es organizar los módulos del programa de tal manera que sean fáciles de desarrollar y cambiar. Las técnicas de diseño estructurado ayudan a los desarrolladores a lidiar con el tamaño y la complejidad de los programas. Los analistas crean instrucciones para los desarrolladores sobre cómo se debe escribir el código y cómo las piezas de código deben encajar para formar un programa.

Existen muchas **estrategias o técnicas** para realizar el diseño del sistema. Son:

- Enfoque de arriba hacia abajo (Top-Down).
- Enfoque ascendente (Bottom-Up).
- Híbrido.
- Diseño estructurado.

Enfoque de arriba hacia abajo

La estrategia de arriba hacia abajo utiliza el enfoque modular para desarrollar el diseño de un sistema. Se llama así porque comienza desde el módulo de nivel superior o superior y **se mueve hacia los módulos de nivel inferior**.



En esta técnica, se identifica el módulo de nivel **más alto o el módulo**

principal para desarrollar el software. El módulo principal se divide en varios **submódulos o segmentos** más pequeños y simples según la tarea realizada por cada módulo. Luego, cada submódulo se subdivide en varios submódulos del siguiente nivel inferior. Este proceso de dividir cada módulo en varios submódulos continúa hasta que no se identifican los módulos de nivel más bajo, que no pueden subdividirse más.

Ventajas: La principal ventaja del enfoque de arriba hacia abajo es que su **fuerte enfoque** en los requisitos ayuda a que un diseño responda de acuerdo con sus requisitos.

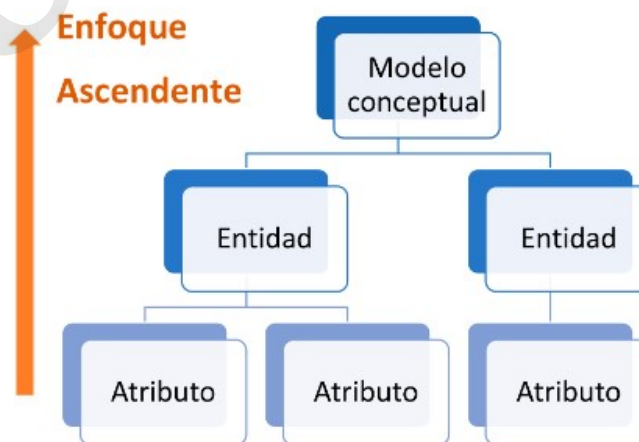
Desventajas:

- Los límites del proyecto y del sistema tienden a estar orientados a las especificaciones de la aplicación. Por lo tanto, es más probable que se pierdan las ventajas de la reutilización de componentes.
- Es probable que el sistema pierda los beneficios de una arquitectura simple y bien estructurada.

Enfoque de abajo hacia arriba

La estrategia de abajo hacia arriba sigue el enfoque modular para desarrollar el diseño del sistema. Se llama así porque comienza desde la parte inferior o desde los módulos de nivel más básicos y avanza hacia los módulos de nivel más alto.

En esta técnica, se identifican los módulos en el nivel más básico o más bajo.



Estos módulos se agrupan en función de la función realizada por cada módulo para formar los siguientes módulos de nivel superior. Luego, estos módulos se combinan para formar los siguientes módulos de nivel superior.

Este proceso de agrupar varios módulos más simples para formar módulos de nivel superior continúa hasta que se logra el módulo principal del proceso de desarrollo del sistema.

Ventajas

- La economía puede resultar cuando se pueden reutilizar soluciones generales.
- Se puede usar para ocultar los detalles de bajo nivel de implementación y combinarse con la técnica de arriba hacia abajo.

Desventajas

- No está tan estrechamente relacionado con la estructura del problema.
- Las soluciones ascendentes de alta calidad son muy difíciles de construir.
- Conduce a la proliferación de funciones 'potencialmente útiles' en lugar de las más apropiadas.

Híbridos

Combinan las mejores prácticas de las metodologías tradicionales y ágiles, incorporando las ventajas de ambas. El uso de estas metodologías se ha ido incrementando de manera paulatina, ya que se adaptan a la cultura y necesidades de cada organismo, generando un incremento de productividad. Algunos ejemplos de este tipo de metodología son: EssUP (Essencial Unified Process), la combinación de Cascada y Scrum y SXP (Scrum y XP).

Entre los beneficios del uso de la metodología híbrida, podemos encontrar:

- Simple y fácil de entender y usar.
- Mejora continua exponencial.
- Flexibilidad a cambios dentro de un alcance de presupuesto definido.



Debemos tener en cuenta que en una misma organización pueden convivir más de un tipo de metodología, en base a una serie de criterios, entre los que podemos destacar:

Criterios Internos: el nivel de madurez de la organización, la naturaleza del proyecto, los interesados, los procesos y la tecnología a utilizar.

Criterios Externos: las expectativas y requisitos de los agentes externos que participan en los diferentes procesos de la organización.

Diseño estructurado

El diseño estructurado es una metodología basada en el flujo de datos que ayuda a identificar la entrada y salida del sistema en desarrollo. El objetivo principal del diseño estructurado es minimizar la complejidad y aumentar la modularidad de un programa. El diseño estructurado también ayuda a describir los aspectos funcionales del sistema.

En el diseño estructurado, las especificaciones del sistema actúan como una base para representar gráficamente el flujo de datos y la secuencia de procesos involucrados en un desarrollo de software con la ayuda de los DFD. Después de desarrollar los DFD para el sistema de software, el siguiente paso es desarrollar el diagrama de estructura.

Modularización

El diseño estructurado divide el programa en módulos pequeños e independientes. Estos se organizan de arriba hacia abajo con los detalles que

se muestran en la parte inferior.

Por lo tanto, el diseño estructurado utiliza un enfoque llamado modularización o descomposición para minimizar la complejidad y gestionar el problema subdividiéndolo en segmentos más pequeños.

Ventajas

- Las interfaces críticas se prueban primero.
- Proporciona abstracción.
- Permite que varios programadores trabajen simultáneamente.
- Permite la reutilización de código.
- Proporciona control y mejora la moral.
- Facilita la identificación de la estructura.

Factores que afectan la complejidad del sistema

Para desarrollar una buena calidad de software del sistema, es necesario desarrollar un buen diseño. Por lo tanto, el enfoque principal al desarrollar el diseño del sistema es la calidad del diseño del software. Un diseño de software de buena calidad es el que minimiza la complejidad y el gasto en el desarrollo de software.

Los dos conceptos importantes relacionados con el desarrollo del sistema que ayudan a determinar la complejidad de un sistema son el acoplamiento y la cohesión.

Acoplamiento

El acoplamiento es la medida de la independencia de los componentes. Define el grado de dependencia de cada módulo de desarrollo del sistema en el otro. En la práctica, esto significa que cuanto más fuerte sea el acoplamiento entre los módulos en un sistema, más difícil será implementar y mantener el sistema.

Cada módulo debe tener una interfaz simple y limpia con otros módulos, y que el número mínimo de elementos de datos se debe compartir entre los módulos.

Alto acoplamiento

Este tipo de sistemas tienen interconexiones con unidades de programa que dependen unas de otras. Los cambios en un subsistema provocan un alto

impacto en el otro subsistema.

Acoplamiento bajo

Este tipo de sistemas están formados por componentes que son independientes o casi independientes. Un cambio en un subsistema no afecta a ningún otro subsistema.

Medidas de acoplamiento

Acoplamiento de contenido

Cuando un componente realmente modifica a otro.

Acoplamiento común

Cuando la cantidad de acoplamiento se reduce algo organizando el diseño del sistema para que los datos sean accesibles desde un almacén de datos común.

Acoplamiento de control

Cuando un componente pasa parámetros para controlar la actividad de otro componente.

Acoplamiento de sello

Cuando las estructuras de datos se utilizan para pasar información de un componente a otro.

Acoplamiento de datos

Cuando solo se pasan datos, los componentes se conectan mediante este acoplamiento.

Cohesión

La cohesión es la medida de la cercanía de la relación entre sus componentes.

Define la cantidad de dependencia de los componentes de un módulo entre sí. En la práctica, esto significa que el diseñador de sistemas debe asegurarse de que: No se dividen los procesos esenciales en módulos fragmentados. No se reúnen procesos no relacionados representados como procesos en el DFD en módulos sin sentido.

Los mejores módulos son aquellos que son funcionalmente cohesivos. Los peores módulos son aquellos que son casualmente cohesivos.

El peor grado de cohesión

La cohesión coincidente se encuentra en un componente cuyas partes no están relacionadas con otro.

- **Cohesión lógica:** varias funciones o elementos de datos relacionados lógicamente se colocan en el mismo componente.
- **Cohesión temporal:** un componente que se usa para inicializar un sistema o establecer variables realiza varias funciones en secuencia, pero las funciones están relacionadas por el tiempo involucrado.
- **Cohesión de procedimiento:** las funciones se agrupan en un mismo componente solo para garantizar este orden.
- **Cohesión secuencial:** la salida de una parte de un componente es la entrada a la siguiente parte de este componente.

Relaciona los elementos de la columna Derecha con la columna Izquierda

Modelo en forma de V 1

1

Modelo de cascada

2

Modelo incremental

3

Este modelo de ciclo de vida implica múltiples ciclos de desarrollo.

Este modelo implica terminar cada fase por completo antes de comenzar la siguiente.

Este modelo se centra en la ejecución de procesos de maneras secuencial, similar al modelo en cascada pero con más importancia en las pruebas.

¡Muy bien! ¡Sigue adelante!

Ingeniería del software estructurada.

En ingeniería de **software**, el análisis estructurado (SA) y el diseño estructurado (SD) son métodos para analizar los requisitos comerciales y desarrollar especificaciones para convertir las prácticas en programas informáticos, configuraciones de hardware y procedimientos manuales relacionados.

El análisis estructurado y las técnicas de diseño son herramientas fundamentales del análisis de sistemas. Se desarrollaron a partir del análisis de sistemas clásicos de los años sesenta y setenta.

El análisis estructurado es un método de desarrollo que permite al analista comprender el sistema y sus actividades de forma lógica. Es un enfoque

sistemático, que utiliza herramientas gráficas que analizan y refinan los objetivos de un sistema existente y desarrollan una nueva especificación del sistema que el usuario puede entender fácilmente.

Tiene los siguientes atributos:

- Es un gráfico que especifica la presentación de la aplicación.
- Divide los procesos para que proporcione una imagen clara del flujo del sistema.
- Es más lógico que físico, es decir, los elementos del sistema no dependen del proveedor o del hardware.
- Es un enfoque que funciona desde descripciones generales de alto nivel hasta detalles de nivel inferior.

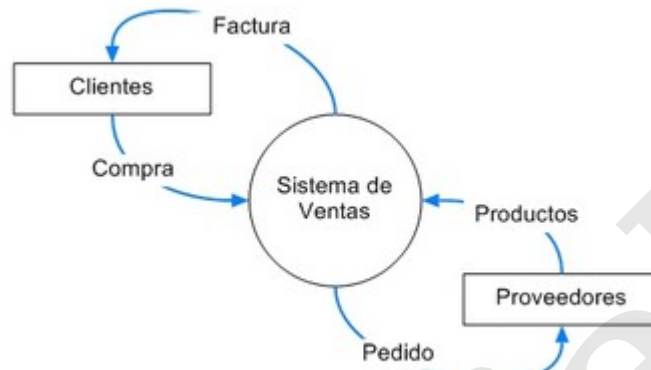
Herramientas de análisis estructurado

Durante el análisis estructurado, se utilizan diversas herramientas y técnicas para el desarrollo del sistema. Son las siguientes:

- Diagramas de flujo de datos.
- Diccionario de datos.
- Árboles de decisión.
- Tablas de decisiones.
- Inglés estructurado.
- Pseudocódigo.

Diagramas de flujo de datos (DFD) o gráfico de burbujas

Es una técnica desarrollada por Larry Constantine para expresar los requisitos del sistema en forma gráfica. Muestra el flujo de datos entre varias funciones del sistema y especifica cómo se implementa el sistema actual.



Es una **etapa inicial** de la fase de diseño que divide funcionalmente las especificaciones de requisitos hasta el nivel más bajo de detalle. Su naturaleza gráfica lo convierte en una buena herramienta de comunicación entre usuario y analista o analista y diseñador de sistemas.

Ofrece una visión general de qué datos procesa un sistema, qué transformaciones se realizan, qué datos se almacenan, qué resultados se producen y dónde fluyen.

Elementos básicos de un DFD

El DFD es fácil de entender y bastante efectivo cuando el diseño requerido no es claro y el usuario desea un lenguaje de notación para la comunicación. Sin embargo, requiere una gran cantidad de iteraciones para obtener la solución más precisa y completa.

Los símbolos utilizados en el diseño de un DFD pueden ser los siguientes:

- **Cuadrado:** Origen o destino de los datos.
- **Flecha:** Flujo de datos.
- **Círculo:** Proceso de transformación de flujo de datos.
- **Lineas:** Almacén de datos y archivos.



Tipos de DFD

Los DFD son de dos tipos: **DFD físico** y **DFD lógico**. Las diferencias entre un DFD físico de uno DFD lógico son las siguientes:

DFD Físico	DFD Lógico
Depende de la implementación. Muestra qué funciones se realizan	Es una implementación independiente. Se enfoca solo en el flujo de datos entre procesos
Proporciona detalles de bajo nivel de hardware, software, archivos y personas	Explica eventos de sistemas y datos requeridos por cada evento
Describe cómo funciona el sistema actual y cómo se implementará un sistema	Muestra cómo opera el negocio, no cómo se puede implementar el sistema

A continuación se muestra un ejemplo de ambos DFD.

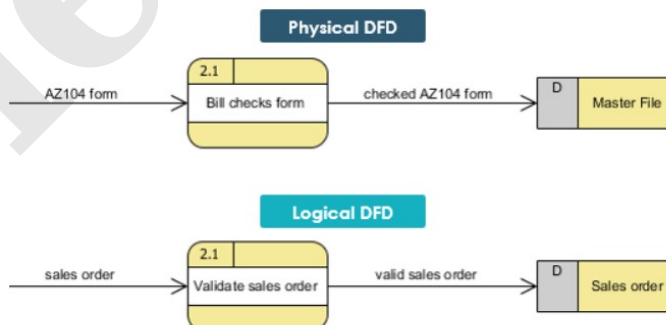


Diagrama contextual

Un diagrama de contexto ayuda a comprender todo el sistema mediante un

DFD que proporciona la **descripción general** de un sistema. Comienza mencionando los procesos principales con pequeños detalles y luego continúa dando más detalles de los procesos con el enfoque de arriba hacia abajo.

Diccionario de datos

Un diccionario de datos es un **depósito estructurado** de elementos de datos en el sistema. Almacena las descripciones de todos los elementos de datos del DFD, es decir, detalles y definiciones de flujos de datos, almacenes de datos, datos almacenados en almacenes de datos y los procesos.

Un diccionario de datos mejora la comunicación entre el analista y el usuario. Desempeña un papel importante en la construcción de una base de datos. La mayoría de los DBMS (Sistema Gestor de Base de Datos) tienen un diccionario de datos como característica estándar.

Data Dictionary

Data Dictionary outlining a Database on Driver Details in NSW

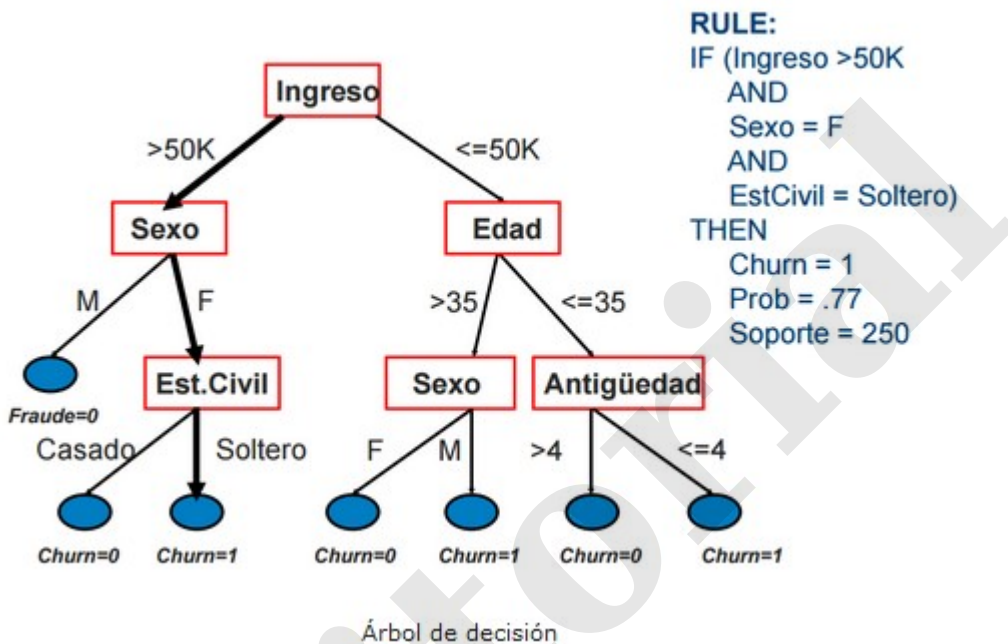
Field Name	Data Type	Data Format	Field Size	Description	Example
License ID	Integer	NNNNNN	6	Unique number ID for all drivers	12345
Surname	Text		20	Surname for Driver	Jones
First Name	Text		20	First Name for Driver	Arnold
Address	Text		50	First Name for Driver	11 Rocky st Como 2233
Phone No.	Text		10	License holders contact number	0400111222
D.O.B	Date / Time	DD/MM/YYYY	10	Drivers Date of Birth	08/05/1956

Diccionario de datos

Árboles de decisión

Los árboles de decisión son un método para definir relaciones complejas al describir decisiones y evitar los problemas de comunicación. Un árbol de decisión es un diagrama que muestra acciones y condiciones alternativas dentro del marco de árbol horizontal. Por lo tanto, describe qué condiciones considerar primero, segundo, etc. Los árboles de decisión representan la relación de cada condición y sus acciones permitidas. Obliga a los analistas a considerar la secuencia de decisiones e identifica la decisión real que debe

tomarse.



La principal limitación de un árbol de decisión es que **carece de información** en su formato para describir qué otras combinaciones de condiciones puede tomar para las pruebas. Es una representación única de las relaciones entre condiciones y acciones.

Tablas de decisión

Las tablas de decisión son un método para describir la compleja relación lógica de una manera precisa que es fácilmente comprensible. Es útil en situaciones donde las acciones resultantes dependen de la ocurrencia de una o varias combinaciones de condiciones independientes. Es una matriz que contiene filas o columnas para definir un problema y las acciones.

Componentes de una tabla de decisiones:

Condición

Se encuentra en el cuadrante superior izquierdo que enumera todas las

condiciones que se deben verificar.

Acción

Está en el cuadrante inferior izquierdo que describe todas las acciones que se llevarán a cabo para cumplir con dicha condición.

Entrada de condición

Se encuentra en el cuadrante superior derecho que proporciona respuestas alas preguntas formuladas en el cuadrante de código auxiliar.

Entrada de acción

Está en el cuadrante inferior derecho que indica la acción apropiada que resulta de las respuestas a las condiciones en el cuadrante de entrada de condición.

Las **entradas** en la tabla de decisiones están dadas por las **Reglas de Decisión** que definen las relaciones entre combinaciones de condiciones y cursos de acción. En la sección de reglas, se sigue la siguiente notación: Si muestra la existencia de una condición. No representa la condición, que no se cumple. Un espacio en blanco contra la acción indica que debe ser ignorado. X (o una marca de verificación) indica os estados de acción que se van a llevar a cabo.

A continuación se muestra un ejemplo de tabla de decisión:

	1	2	3	4	5	6	7	8
Pago con tarjeta oro	Sí	Sí	Sí	No	No	Sí	No	No
Pago con tarjeta club	Sí	Sí	No	Sí	No	No	Sí	No
Modalidad joven de tarjeta	Sí	No	Sí	Sí	Sí	No	No	No
Inviabile	X	X			X			
Descuento 15% (oro)			X			X		
Descuento 5% (club)				X			X	
Descuento 5% (joven)			X	X				
Calcular importe			X	X		X	X	X

Inglés estructurado

Se deriva de un lenguaje de **programación** estructurado que proporciona una descripción más precisa y comprensible del proceso. Se basa en la lógica de procedimiento que utiliza la construcción y oraciones imperativas diseñadas para realizar operaciones para la acción.

Se utiliza mejor cuando se deben considerar secuencias y bucles en un programa y el problema necesita secuencias de acciones con decisiones. No tiene una regla de sintaxis estricta. Expresa toda la lógica en términos de estructuras de decisión secuenciales e iteraciones.

A continuación se muestra una secuencia de acciones:

```

if customer pays advance
then
    Give 5% Discount
else
    if purchase amount >=10,000
then
    if the customer is a regular customer
    then Give 5% Discount
    else No Discount
end if
    
```

else No Discount end if end if

Pseudocódigo

Un pseudocódigo no se ajusta a ningún **lenguaje de programación** y expresa lógica en un lenguaje simple. También se podría hacer en español. Puede especificar la lógica de programación física sin codificación real durante y después del diseño físico. Se utiliza junto con la programación estructurada. Reemplaza los diagramas de flujo de un programa.

Ejemplo de un Pseudocódigo para ordenar una lista de elementos (L):

```
procedimiento Ordenar (L)
  k ← 0
  repetir
    intercambio ← falso
    k ← k + 1
    para i ← 1 hasta n - k hacer
      si  $L_i > L_{i+1}$  entonces
        intercambiar ( $L_i, L_{i+1}$ )
        intercambio ← verdadero
      fin si
    fin para
  hasta que intercambio = falso
fin procedimiento
```

Pseudocódigo

Pautas para seleccionar las herramientas apropiadas

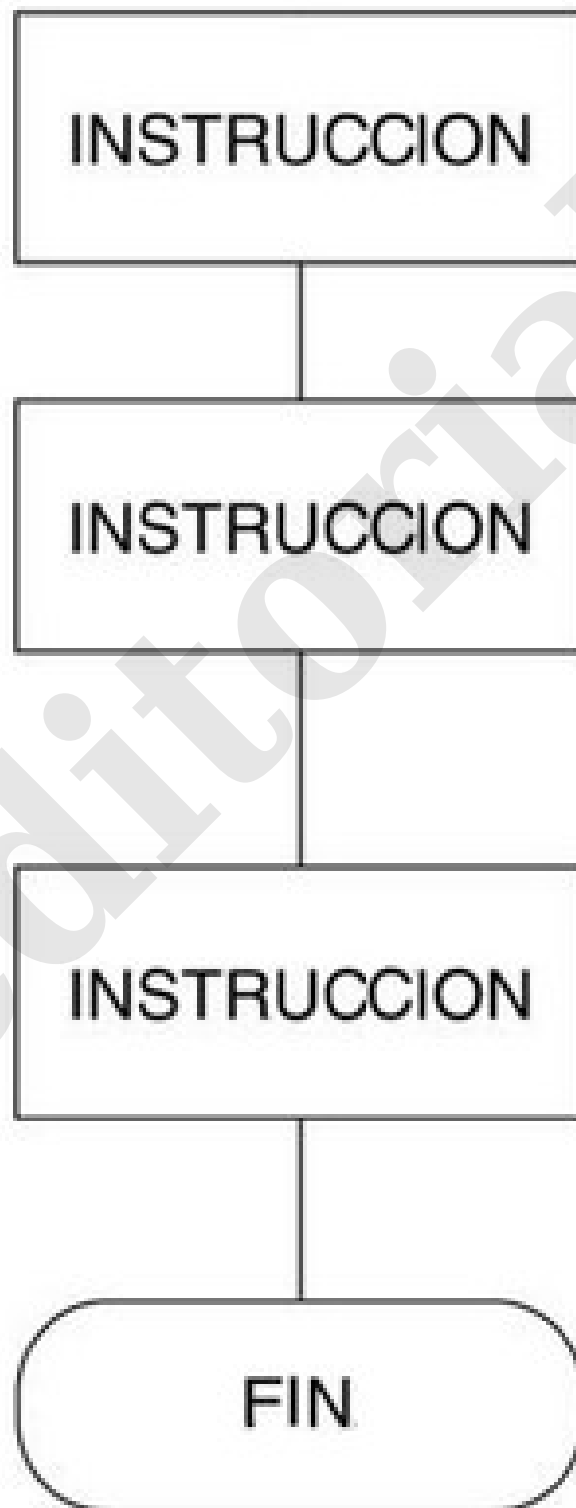
A continuación se incluyen unas pautas para seleccionar la herramienta más adecuada que se adapte a los requerimientos:

- Utilice DFD en análisis de alto o bajo nivel para proporcionar buenas documentaciones del sistema.

- Utilice el diccionario de datos para simplificar la estructura para cumplir con los requisitos de datos del sistema.
- Use inglés estructurado si hay muchos bucles y las acciones son complejas.
- Utilice las tablas de decisión cuando haya una gran cantidad de condiciones para verificar y la lógica sea compleja.
- Use árboles de decisión cuando la secuencia de condiciones sea importante y si hay pocas condiciones para ser probadas.

Estructuras de control

La ingeniería estructurada se basa en las estructuras de control que van a determinar el orden en el que se ejecutan las instrucciones de un programa. La estructura más básica y fundamental es la secuencial, es decir, las órdenes se ejecutarán secuencialmente en el orden en el que aparezcan.



Por otra parte, las **estructura condicionales** nos permiten ejecutar un bloque de **instrucciones** en función de que se cumplan o no una o varias condiciones. La estructura de control clásica es:

```
if (condición)
    acciones1
else
    acciones2
```

Estructura de decisión

Donde la cláusula else es opcional. Tenemos que entender esta estructura del siguiente tipo: Si condición es verdadera, será ejecutada acciones1, en otro caso, se ejecutará acciones2. Veamos por ejemplo el siguiente código JavaScript:


```
1 <html>
2 <head>
3
4 <SCRIPT type="text/javascript">
5 <!--
6   var respuesta
7   respuesta = prompt("Introduzca su nombre:", "anónimo")
8   if (respuesta == "Juan")
9     document.write("Bienvenido, señor " + respuesta)
10  else
11    document.write("Usted no puede estar aquí.")
12  //-->
13 </SCRIPT>
14
15
16 </head>
17 <body>
18
19
20 </body>
21 </html>
```

Código

Hay que destacar que podemos hacer estructuras más complejas de este tipo, sin más que introducir más if/else dentro de otros if/else.

Las condiciones también suelen llamarse expresiones lógicas porque sólo pueden dar como resultado los valores verdadero o falso.

Usted puede usar los operadores de comparación siguientes para emplear expresiones de este tipo: == (igual a), != (diferente a), < (menor que), > (mayor que), <= (menor o igual que), >= (mayor o igual que)

Además de estos operadores, también pueden utilizar los operadores lógicos AND, OR y NOT.

En **JavaScript** el **operador** AND se muestra con los símbolos &&, OR con || y NOT con !



De esta manera, una expresión del tipo `exp1 AND exp2` es evaluada a verdadero sólo en el caso de que tanto `exp1` como `exp2` se correspondan con verdadero. En cualquier otro caso, la expresión se evaluará a falso.

Pese a ello, `exp1 OR exp2` se evalúa a verdadero en el caso de que `exp1` o `exp2` se correspondan a verdadero (observe que es suficiente con que lo sea una de las dos o las dos al mismo tiempo). En el caso de que ninguna de estas se evalúe a verdadero, la expresión total se evaluará a **falso**.

Finalmente, `NOT exp1` se evalúa a verdadero si `exp1` se evalúa a falso y se evalúa a falso si **`exp1`** se evalúa a verdadero. Es decir, realiza la negación de `exp1`.

Otro tipo de **estructuras** que pueden cambiar el flujo de ejecución son las estructuras de repetición o además conocidas como bucles. Estas estructuras se utilizan para repetir un grupo de **instrucciones**.

De esta manera, puede querer repetir un número fijo de veces el conjunto de instrucciones o, sin conocer tal número, querer repetirlas mientras se cumpla tal condición.

Esto indica que hay dos tipos de estructuras de repetición: esas en que el número de repeticiones es conocido y esas en las que no lo es.

Usando la estructura `for`, usted indica el número de veces que quiere que se repita un conjunto de instrucciones. Por lo tanto, ese número tiene que ser conocido.

La sintaxis correcta de esta estructura de repetición es:

```
for (expresión; condición; operación)
{
    acciones
}
```

Sintaxis correcta

Debemos entender esta estructura como:

La “expresión” suele consistir en dar un valor inicial a una variable.

Este valor se va comparando en cada una de las repeticiones según la “condición” establecida. “Operación” permite acercar el valor inicial al valor final, es decir, en el cual ya no se realizarán más repeticiones.

En cada vuelta del bucle se ejecutará 1 vez lo que diga “acciones”.

Veamos este código

```
1 <HTML>
2 <HEAD>
3 <TITLE>Bucle for</TITLE>
4 </HEAD>
5 <BODY>
6 <SCRIPT type="text/javascript">
7 <!--
8   for (var i = 1; i < 10; i++)
9   {
10    document.write(i)
11    document.write("<BR>")
12  }
13  //-->
14 </SCRIPT>
15 </BODY>
16 </HTML>
```

Código

Y el resultado en pantalla será:



Resultado

Es decir, i vale 1 al principio, como 1 es $<$ que 10, se ejecuta lo que hay dentro de las $\{ \}$. Imprime lo que vale i (vale 1), imprime un salto de línea y cuando ha terminado de imprimir todo el cuerpo del bucle, hace $i++$, es decir, ahora $i = 2$, y vuelve al principio...

Así hasta que $i=10$, que al comprobar si $10 < 10$, esta condición es igual a FALSE y se sale del bucle.

Por ejemplo, piense en esta **situación**: usted solicita al **usuario** que ingrese un número e imprime los números que abarcan desde el 1 hasta dicho número. Observe cómo, de antemano, usted no sabe cuántas veces se va a repetir el **bucle**.

Utilizando la estructura while, usted podrá repetir un conjunto de instrucciones mientras se cumpla cierta condición. La sintaxis correcta de esta estructura es:

```
while (condición)  
{  
  acciones  
}
```

Acciones

Que lo que hará será ejecutar las acciones hasta que la condición sea Falsa.

Veamos el siguiente código:

```
1 <HTML>  
2 <HEAD>  
3 <TITLE>Bucle while</TITLE>  
4 </HEAD>  
5 <BODY>  
6 <SCRIPT type="text/javascript">  
7 <!--  
8   var respuesta = prompt("Por favor, introduzca un número:")  
9   var i = 1  
10  while (i < respuesta){  
11    document.write(i)  
12    document.write("<BR>")  
13    i++  
14  }  
15  
16  //-->  
17 </SCRIPT>  
18 </BODY>  
19 </HTML>
```

Código

Observamos en la línea 8 que le pedimos al usuario mediante un prompt un número de veces a ejecutar el bucle while.

Ésta se almacena en la variable respuesta, y luego podremos ejecutar el bucle while tantas veces como diga respuesta.

Notar que al estar programando la función, no sabíamos el valor exacto de respuesta, por lo que no podíamos utilizar un for.

En ocasiones, usted deseará salir de un bucle aunque la condición de prueba no se haya cumplido. En estos casos puede utilizar la sentencia break si desea salir completamente del bucle o continue si lo que quiere es no ejecutar el resto de sentencias del cuerpo del bucle, sino pasar a la siguiente **repetición**. Al igual que ocurría con las estructuras de decisión, puede anidar bucles en el interior de otros bucles. En estos casos, para cada repetición del bucle exterior, se ejecuta completamente el bucle interior.

Tenga en cuenta que la incorporación de bucles en el interior de otros bucles puede hacer que la ejecución del código consuma mucho tiempo.

Son métodos para analizar los requisitos comerciales y desarrollar especificaciones para convertir las prácticas en programas informáticos, configuraciones de hardware y procedimientos manuales relacionados.

El diseño estructurado (SD)

El análisis estructurado(SA)

Ingeniería del software orientada a objetos.

Hoy en día el paradigma de **orientación a objetos** es el más utilizado para programar y prácticamente lo podemos encontrar en la mayoría de los lenguajes de programación más utilizados: Java, PHP, C++, etc.

Pero, ¿qué es realmente la orientación a objetos? La orientación a objetos es una forma de programar que **rompe** con la forma de programar clásica.

En un entorno clásico de programación el problema o sistema que se está analizando se divide en funciones: este sistema hace esto, esto y esto.

Para llevar a cabo el análisis con la programación orientada a objetos hay que tener en cuenta que los programas se **codifican** pensando en objetos que interactúan.

Los **objetos** tendrán unas determinadas propiedades y serán capaces de hacer ciertas cosas mediante los métodos que se les hayan definido. Si se hace un buen diseño de los objetos y de las clases, se podrán usar en **nuevos programas y proyectos** fácilmente, favoreciendo la reusabilidad.



En el enfoque orientado a objetos, el objetivo es capturar la estructura y el comportamiento de los sistemas de información en **pequeños módulos** que combinan datos y procesos. El objetivo principal del diseño orientado a objetos (OOD) es mejorar la calidad y productividad del análisis y diseño del sistema haciéndolo más utilizable.

En la **fase de análisis**, los modelos OO se utilizan para llenar el vacío entre el problema y la solución. Funciona bien en situaciones en las que los sistemas están en continuo diseño, adaptación y mantenimiento. Identifica los objetos en el dominio del problema, clasificándolos en términos de datos y comportamiento.

El **modelo OO** conlleva las siguientes ventajas:

- Facilita cambios en el sistema a bajo costo.
- Promueve la reutilización de componentes.
- Simplifica el problema de integrar componentes para configurar un sistema grande.
- Simplifica el diseño de sistemas distribuidos.
- El diseño ya no se lleva a cabo independientemente de la implementación posterior porque durante la fase de diseño debemos

considerar qué componentes están disponibles para la solución del problema.

- El diseño y la implementación se asocian más estrechamente.
- La duración de la fase de implementación se reduce.
- Surge un nuevo título de trabajo, el bibliotecario de la clase, que es responsable de garantizar la usabilidad eficiente de la biblioteca de la clase.

Los **componentes** de un **Sistema orientado a objetos** son las siguientes:

Objetos

Un objeto es algo que existe dentro del dominio del problema y puede identificarse por datos (atributo) o comportamiento. Todas las entidades tangibles (estudiante, paciente) y algunas entidades intangibles (cuenta bancaria) se modelan como objeto.

Atributos

Describen información sobre el objeto.

Comportamiento

Especifica lo que puede hacer el objeto. Define la operación realizada en los objetos.

Clase

Una clase encapsula los datos y su comportamiento. Objetos con significado y propósito similares agrupados como clase.

Métodos

Los métodos determinan el comportamiento de una clase. No son más que una

acción que un objeto puede realizar.

Mensaje

Un mensaje es una llamada a una función o procedimiento de un objeto a otro. Son información enviada a objetos para activar métodos. Esencialmente, un mensaje es una llamada a una función o procedimiento de un objeto a otro.

A continuación vamos a ver las **diferentes características** de la programación orientada a objetos:

Clases y objetos

Actualmente una de las áreas más candentes en la industria y en el ámbito académico es la orientación a **objetos**. La orientación a objetos promete grandes mejoras de amplio alcance en la manera de diseñar, desarrollar y mantener del software ofreciendo una solución a largo plazo a los problemas que hay desde el inicio en el desarrollo de software: la **escasez de portabilidad** del código y **reusabilidad**, código que no es fácil de cambiar, ciclos de desarrollo largos y técnicas de codificación no intuitivas.

Un lenguaje orientado a objetos trata de **solventar estos problemas**. Posee tres características básicas: tiene que estar basado en objetos, basado en clases y poder tener herencia de clases. Muchos lenguajes realizan uno o dos de estos puntos; muchos menos realizan los tres. La barrera más difícil de sortear normalmente es la herencia.

La definición de **programación orientada a objetos** (OOP) no es nueva, lenguajes clásicos como SmallTalk están basados en ella. Dado que la OOP. se basa en la idea natural de que hay un mundo lleno de objetos y que la resolución del problema se hace en términos de objetos, un lenguaje podemos decir que está basado en objetos si puede hacer uso de objetos como una propiedad fundamental del mismo.

El elemento fundamental de la OOP es, como su propio nombre lo indica, el **objeto**. Podemos determinar un objeto como un grupo complejo de datos y programas que tienen estructura y se comportan como parte de una organización.

Esta definición determina varias propiedades importantes de los objetos. Primero, un objeto no es un dato simple, sino que posee en su interior cierto número de componentes bien organizados. En segundo lugar, cada objeto no podemos decir que sea un ente aislado, sino que **forma parte de una organización jerárquica** o de otro tipo.

Estructura de un objeto



Un objeto puede ser considerado como una **especie de cápsula separada** en tres partes:

- Relaciones.
- Propiedades.
- Métodos.

Cada uno de estos elementos realiza un papel **totalmente independiente**:

Relaciones

Las relaciones posibilitan que el objeto sea insertado en la organización y están compuestas esencialmente por punteros a otros objetos.

Propiedades

Las propiedades diferencian un objeto determinado de los restantes que

pertenecen a parte de la misma organización y posee valores que dependen de la propiedad de que se hable. Las características de un objeto pueden ser heredadas a sus hijos en la organización.

Métodos

Los métodos son las operaciones que pueden hacerse sobre el objeto, que normalmente estarán implementados en forma de programas (código) que el objeto es capaz de realizar y que también pone a disposición de sus descendientes mediante la herencia.

Encapsulamiento y ocultación

Como hemos visto, cada objeto se trata de una estructura compleja en cuyo interior existen datos y programas, todos ellos **relacionados entre sí**, como si se encontraran encerrados de manera conjunta en una cápsula. Esta característica (encapsulamiento), es una de las características principales en la OOP.

La **encapsulación** es un proceso de ocultación de información. Es simplemente la combinación de proceso y datos en una sola entidad. Los datos de un objeto están ocultos del resto del sistema y solo están disponibles a través de los servicios de la clase. Permite mejorar o modificar los métodos utilizados por los objetos sin afectar otras partes de un sistema.

Los objetos no son accesibles, e impiden que otros objetos, los usuarios, o por ejemplo los programadores conozcan cómo está distribuida la información o qué información se encuentra disponible. Esta propiedad de los objetos se llama ocultación de la información.

Esto no significa, sin embargo, que sea imposible saber lo necesario respecto a un objeto y a lo que posee. Si así fuera no se podría realizar gran cosa con él. Lo que ocurre es que las peticiones de información a un objeto. Tienen que realizarse mediante mensajes dirigidos a él, con la orden de realizar la operación pertinente. La respuesta a estas órdenes será la información necesitada, siempre que el objeto valore que quien envía el mensaje está autorizado para obtenerla.

El hecho de que cada objeto sea una cápsula ayuda en gran medida que un

objeto determinado pueda ser transportado a otro punto de la organización, o además a otra organización totalmente distinta que necesite de él. Si el objeto ha sido bien construido, sus métodos **continuarán funcionando** en el nuevo entorno sin problemas. Esta cualidad hace que la OOP sea muy correcta para la reutilización de programas.

Abstracción

Es un proceso de tomar o seleccionar el método y los atributos necesarios para especificar el objeto. Se centra en las **características esenciales** de un objeto en relación con la perspectiva del usuario.

Relaciones

Todas las clases en el sistema están relacionadas entre sí. Los objetos **no existen de forma aislada**, existen en relación con otros objetos.

Hay **tres tipos de relaciones** de objeto:

Agregación

Indica la relación entre un todo y sus partes.

Asociación

En este sentido, dos clases están relacionadas o conectadas de alguna manera, como una clase trabaja con otra para realizar una tarea o una clase actúa sobre otra clase.

Generalización

La clase secundaria se basa en la clase principal. Indica que dos clases son similares pero tienen algunas diferencias.

Herencia

La herencia es una gran característica que permite crear subclases a partir de una clase existente al heredar los atributos y/u operaciones de las clases existentes. Es una característica que representa la relación "es una" entre diferentes clases.

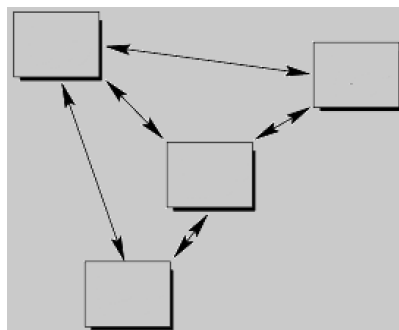
La herencia es un mecanismo en el que una clase **adquiere la propiedad de otra clase**. Por ejemplo, un empleado hereda de la clase persona. Con la herencia, podemos reutilizar los campos y métodos de la clase existente. Por lo tanto, la herencia facilita la reutilización y es un concepto importante de los POO.

Una de las principales ventajas de las técnicas de programación orientada a objetos sobre las técnicas de programación de procedimientos es que permiten a los programadores crear módulos que no necesitan ser cambiados cuando se agrega un nuevo tipo de objeto. Un programador puede simplemente crear un nuevo objeto que herede muchas de sus características de los objetos existentes. Esto hace que los programas orientados a objetos sean más fáciles de modificar.

Polimorfismo y Enlace Dinámico

El polimorfismo es la capacidad de adoptar muchas formas diferentes. Se aplica tanto a los objetos como a las operaciones. Un objeto polimórfico es aquel cuyo tipo verdadero se esconde dentro de una súper clase o clase madre.

En la operación polimórfica, la operación puede llevarse a cabo de manera diferente por diferentes clases de objetos. Nos permite manipular objetos de diferentes clases al conocer solo sus propiedades comunes.



Enfoque estructurado frente a enfoque orientado a objetos

La siguiente tabla explica cómo el enfoque orientado a objetos difiere del enfoque estructurado tradicional:

Enfoque estructurado	Enfoque orientado a objetos
Funciona con enfoque de arriba hacia abajo	Funciona con un enfoque ascendente
El programa se divide en varios submódulos o funciones	El programa se organiza teniendo varias clases y objetos
Se utiliza la llamada a la función	Se utiliza el paso de mensajes
La reutilización del software no es posible	La reutilización es posible
La programación de diseño estructurado generalmente se deja hasta las fases finales	Programación de diseño orientada a objetos realizada simultáneamente con otras fases
El diseño estructurado es más adecuado para la deslocalización	Es adecuado para el desarrollo interno
Muestra una transición clara del diseño a la implementación	No es tan clara la transición del diseño a la implementación
Es adecuado para sistemas en tiempo real, sistemas embebidos y proyectos donde los objetos no son el nivel de abstracción más útil	Es adecuado para la mayoría de las aplicaciones comerciales, proyectos de desarrollo de juegos, que se espera personalizar o ampliar
El diagrama DFD y ER modela los datos	El diagrama de clases, el diagrama de secuencia, el diagrama de estado y los casos de uso contribuyen
En esto, los proyectos se pueden gestionar fácilmente debido a fases claramente identificables	En este enfoque, los proyectos pueden ser difíciles de gestionar debido a transiciones inciertas entre fases

Arrastra las palabras de la columna Derecha en la columna Izquierda

Diferencian un objeto determinado de los restantes que pertenecen a parte de la misma organización y posee valores que dependen de la propiedad de que se hable.

arrastra...

son las operaciones que pueden hacerse sobre el objeto, que normalmente estarán implementados en forma de programas (código) que el objeto es capaz de realizar y que también pone a disposición de sus descendientes mediante la herencia.

arrastra...

Posibilitan que el objeto sea insertado en la organización y están compuestas esencialmente por punteros a otros objetos.

arrastra...

Propiedades

Métodos

Relaciones

Ingeniería web.

En la ingeniería de software se llama aplicación **Web** a aquellas herramientas

que los usuarios pueden usar ingresando a un servidor Web mediante Internet o de una intranet a través de un navegador. En otras palabras, es una aplicación software que está codificada en un lenguaje soportado por los navegadores Web en la que se permite la ejecución al navegador.

Las **aplicaciones Web** son populares dado lo práctico del navegador Web como **cliente liviano**, a la independencia del sistema operativo, así como a la sencillez para actualizar y mantener aplicaciones Web sin distribuir e instalar software a cientos de usuarios potenciales. Hay aplicaciones como los Webmails, wikis, Weblogs, tiendas en línea y la misma Wikipedia que son ejemplos conocidos de aplicaciones Web.



Es importante destacar que una página Web puede contener elementos que posibilitan una **comunicación activa** entre el usuario y la información. Esto permite que el usuario ingrese a los datos de modo interactivo, gracias a que la página realizará una respuesta a cada una de sus acciones, como por ejemplo rellenar y enviar formularios, entrar a participar en juegos diversos y acceder a gestores de base de datos de todo tipo.

Ingeniería web

La ingeniería web es **multidisciplinaria** y abarca contribuciones de diversas áreas: análisis y diseño de sistemas, ingeniería de software, ingeniería de hipertexto / hipertexto, ingeniería de requisitos, interacción humano-computadora, interfaz de usuario, ingeniería de información, indexación y recuperación de información., pruebas, modelado y simulación, gestión de proyectos y diseño gráfico y presentación.

Vemos claramente que la ingeniería web no es un clon ni un subconjunto de la ingeniería de software, aunque **ambos involucran programación** y desarrollo de software. Si bien la ingeniería Web utiliza principios de ingeniería de software, abarca nuevos enfoques, metodologías, herramientas, técnicas y pautas para cumplir con los requisitos únicos de las aplicaciones basadas en la Web.



Ingeniería web como disciplina

Los defensores de la ingeniería web apoyaron el establecimiento de la ingeniería web como disciplina en una **etapa temprana** de la web. Los principales argumentos para la ingeniería web como una nueva disciplina son:

- El proceso de desarrollo de los sistemas de información basados en web (WIS) es diferente y único.
- La ingeniería web es multidisciplinaria; ninguna disciplina única (como la ingeniería de software) puede proporcionar una base teórica completa, un cuerpo de conocimientos y prácticas para guiar el desarrollo de WIS.
- Cuestiones de evolución y gestión del ciclo de vida en comparación con aplicaciones más "tradicionales".
- Los sistemas y aplicaciones de información basados en la web son

generalizados y no triviales. La perspectiva de la Web como plataforma continuará creciendo y vale la pena ser tratada específicamente.

Sin embargo, ha sido controvertido, especialmente para las personas en otras disciplinas tradicionales como la ingeniería de software, reconocer la ingeniería web como un nuevo campo. El problema radica en lo **diferente e independiente** que resulta la ingeniería web, en comparación con otras disciplinas.

Áreas de interés

Los principales aspectos de la ingeniería de la Web **incluyen**, entre otras, las siguientes cuestiones:

- Diseño de procesos de negocio para aplicaciones web.
- Herramientas CASE para aplicaciones web.
- Generación de código para aplicaciones web.
- Desarrollo web colaborativo.
- Modelado conceptual de aplicaciones web.
- Diseño de Modelos de datos para sistemas de información web.
- Ingeniería web empírica.
- Entornos de desarrollo de aplicaciones web integrados.
- Herramientas de autor para contenido multimedia.
- Pruebas de rendimiento de aplicaciones basadas en web.
- Personalización y adaptación de aplicaciones web.
- Herramientas y métodos de prototipado.
- Control de calidad y pruebas de sistemas.
- Ingeniería de requisitos para aplicaciones web.
- Aplicaciones para la Web Semántica.
- Factorías de software para la web.
- Métodos, herramientas y automatización de pruebas para aplicaciones web.
- Aplicaciones web móviles y ubicuas.
- Usabilidad de aplicaciones web.
- Accesibilidad para la web.
- Metodologías de diseño web.
- Formación en ingeniería de la web.
- Diseño de interfaces de usuario.
- Métricas para la web, estimación de costes y medición.
- Gestión de proyectos web y gestión de riesgos.

- Desarrollo y despliegue de servicios web.

Categorías de los sitios web



Los sitios web se pueden distribuir en las siguientes **categorías**:

Sitio estático

Se enfoca en la organización de la estructura y el contenido, en la forma en que se va a presentar la información y que sea fácil de manejar por cualquier usuario. Sin descuidar la eficiencia y la confiabilidad.

Sitio estático con formularios de entrada

Este sitio tiene las mismas características que el anterior, y además permite a los usuarios la interacción mediante el uso de cuestionarios, comentario y sugerencias.

Sitio con acceso de datos dinámicos

Además de las características antes mencionadas, cuenta con bases de datos sobre las que el usuario puede realizar consultas y búsquedas.

Sitio creado dinámicamente

Tenemos los mismos requerimientos pero deben satisfacer las necesidades de cada usuario. El resultado son sitios dinámicos compatibles con el entorno de navegación de cada usuario.

Aplicación de software basada en la Web

Estos sitios pueden tener todas las características ya mencionadas, pero además dando la sensación de estar trabajando con una implementación cliente/servidor de escritorio.

Hay aplicaciones como los Webmails, wikis, Weblogs, tiendas en línea y la misma Wikipedia que son ejemplos conocidos de aplicaciones Web.

Verdadero.

Falso.

Ingeniería del software asistida por computador.

Existe **software** que nos ayudan a desarrollar la ingeniería de software, como por ejemplos los sistemas de análisis del software computarizados.

CASE significa **Computer Aided Software Engineering**. Significa, desarrollo y mantenimiento de proyectos de software con la ayuda de varias herramientas de software automatizadas.

Las herramientas CASE son un conjunto de programas de aplicación de software que se utilizan para automatizar las actividades SDLC (Software

Development Life Cycle). Los administradores de proyectos de software, analistas e ingenieros utilizan herramientas CASE para desarrollar un sistema de software.

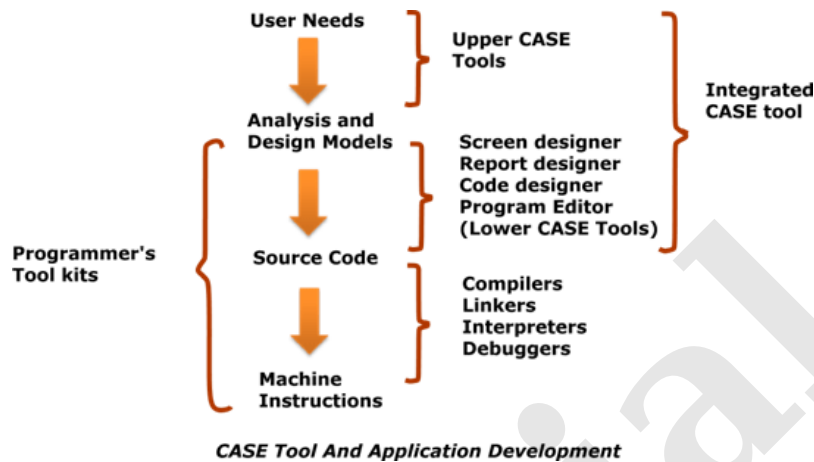
Hay varias herramientas CASE disponibles para simplificar varias etapas del ciclo de vida del desarrollo de software, como herramientas de análisis, herramientas de diseño, herramientas de gestión de proyectos, herramientas de gestión de bases de datos, herramientas de documentación, por nombrar algunas.

El uso de herramientas CASE acelera el desarrollo del proyecto para producir el resultado deseado y ayuda a descubrir fallos antes de avanzar con la siguiente etapa en el desarrollo de software.

Componentes de las herramientas CASE

Las herramientas CASE se pueden dividir en las siguientes partes según su uso en cada etapa SDLC:

- **Repositorio central** : las herramientas CASE requieren un repositorio central, que puede servir como fuente de información común, integrada y coherente. El repositorio central es un lugar central de almacenamiento donde se almacenan las especificaciones del producto, los documentos de requisitos, los informes y diagramas relacionados y otra información útil sobre la gestión. El repositorio central también sirve como diccionario de datos.
- **Herramientas Upper Case**: se utilizan en las etapas de planificación, análisis y diseño de SDLC.
- **Herramientas Lower Case**: se utilizan en la implementación, prueba y mantenimiento.
- **Herramientas integrated Case**: las herramientas integradas de CASE son útiles en todas las etapas de SDLC, desde la recopilación de requisitos hasta las pruebas y la documentación.



Las herramientas CASE se pueden agrupar si tienen una funcionalidad similar, actividades de proceso y la capacidad de integrarse con otras herramientas.

El **alcance** de las herramientas CASE abarca todo el SDLC.

Tipos de herramientas CASE

Revisamos brevemente varias herramientas CASE

Herramientas de diagrama

Estas herramientas se utilizan para representar los componentes del sistema, los datos y el flujo de control entre **varios componentes de software** y la estructura del sistema en forma gráfica. Por ejemplo, la herramienta Flow Chart Maker para crear diagramas de flujo de última generación.

Herramientas de modelado de procesos

El modelado de procesos es un método para crear un modelo de proceso de software, que se utiliza para **desarrollar el software**. Las herramientas de modelado de procesos ayudan a los gerentes a elegir un modelo de proceso o modificarlo según los requisitos del producto de software. Por ejemplo, EPF Composer

Herramientas de gestión de proyectos

Estas herramientas se utilizan para la planificación de proyectos, la estimación de costos y esfuerzos, la programación de proyectos y la planificación de recursos. Los gerentes deben cumplir estrictamente la ejecución del proyecto con cada paso mencionado en la gestión de proyectos de software. Las herramientas de gestión de proyectos ayudan a **almacenar y compartir información** del proyecto en tiempo real en toda la organización. Por ejemplo, Creative Pro Office, Trac Project, Basecamp.

Herramientas de documentación

La documentación en un proyecto de software comienza antes del proceso del software, abarca todas las fases del **SDLC** y después de la finalización del proyecto.

Las herramientas de documentación generan documentos para usuarios técnicos y usuarios finales. Los **usuarios técnicos** son en su mayoría profesionales internos del equipo de desarrollo que se refieren al manual del sistema, manual de referencia, manual de capacitación, manuales de instalación, etc. Los documentos del usuario final describen el funcionamiento y procedimientos del sistema, como el manual del usuario. Por ejemplo, Doxygen, DrExplain, Adobe RoboHelp para documentación.

Herramientas de análisis

Estas herramientas ayudan a reunir requisitos, comprueban automáticamente cualquier inconsistencia, inexactitud en los diagramas, redundancias de datos u omisiones erróneas. Por ejemplo, Accept 360, Accompa, CaseComplete para análisis de requisitos, Visible Analyst para análisis total.

Herramientas de diseño

Estas herramientas ayudan a los diseñadores de software a diseñar la estructura de bloques del software, que puede **desglosarse** en módulos más pequeños utilizando técnicas de refinamiento. Estas herramientas proporcionan detalles de cada módulo e interconexiones entre módulos. Por ejemplo, diseño de software animado.

Herramientas de gestión de configuración

Se lanza una instancia de software en una versión. Las herramientas de

gestión de configuración se ocupan de lo siguiente:

- Gestión de versiones y revisiones.
- Gestión de configuración de línea de base.
- Gestión de control de cambios.

Las herramientas CASE ayudan en este aspecto mediante el seguimiento automático, la gestión de versiones y la gestión de versiones. Por ejemplo, Fossil, Git, Accu REV.

Herramientas de control del cambio

Estas herramientas se consideran parte de las herramientas de administración de configuración. Se ocupan de los cambios realizados en el software después de que se fija su línea de base o cuando se lanza el software por primera vez. Las herramientas CASE automatizan el seguimiento de cambios, la gestión de archivos, la gestión de códigos y más. También ayuda a hacer cumplir la política de cambio de la organización.

Herramientas de programación

Estas herramientas consisten en entornos de programación como **IDE** (Entorno de desarrollo integrado), biblioteca de módulos incorporados y herramientas de simulación. Estas herramientas proporcionan una ayuda integral en la creación de productos de software e incluyen características para simulación y pruebas. Por ejemplo, Cscope para buscar código en C, Eclipse.

Herramientas de prototipos

El prototipo de software es una versión simulada del producto de software previsto. **Prototype** proporciona una apariencia inicial del producto y simula pocos aspectos del producto real.

Las herramientas de creación de prototipos CASE vienen esencialmente con bibliotecas gráficas. Pueden crear interfaces de usuario y diseño independientes del hardware. Estas herramientas nos ayudan a construir prototipos rápidos basados en la información existente. Además, proporcionan simulación de prototipo de software. Por ejemplo, el compositor prototipo de Serena, Mockup Builder.

Herramientas de desarrollo web



Estas herramientas ayudan a diseñar páginas web con todos los elementos aliados como formularios, texto, guiones, gráficos, etc. Las herramientas web también proporcionan una vista previa en vivo de lo que se está desarrollando y **cómo se verá una vez finalizado**. Por ejemplo, Fontello, Adobe Edge Inspect, Foundation 3, Brackets.

Herramientas de aseguramiento de calidad

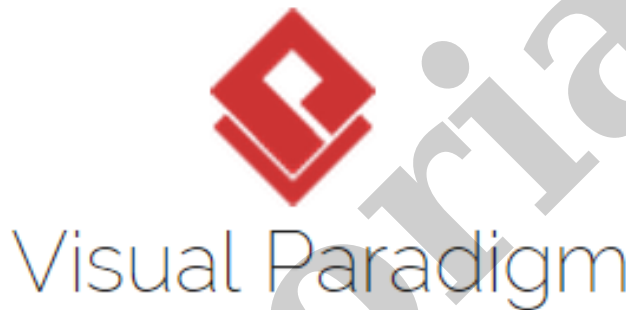
El aseguramiento de la calidad en una organización de software está monitoreando el proceso de ingeniería y los métodos adoptados para desarrollar el producto de software con el fin de garantizar la conformidad de la calidad según los estándares de la organización. Las herramientas de control de calidad consisten en herramientas de configuración y control de cambios y herramientas de prueba de software. Por ejemplo, SoapTest, AppsWatch, JMeter.

Herramientas de mantenimiento

El mantenimiento del software incluye modificaciones en el producto de software después de que se entrega. El registro automático y las técnicas de informe de errores, la generación automática de **tickets de error** y el análisis de causa raíz son pocas herramientas CASE, que ayudan a la organización del software en la fase de mantenimiento de SDLC. Por ejemplo, Bugzilla para el seguimiento de defectos, HP Quality Center.

Ejemplo de herramienta CASE

Visual paradigm Visual Paradigm es una herramienta CASE que ofrece todo lo necesario para modelar mediante **UML**, además aporta a los ingenieros de software, analistas y desarrolladores una asistencia durante todo el proceso de desarrollo de un software.



Esta herramienta es privada, Tiene **varias versiones**: Enterprise, Professional, Community, Standard, Modeler y Personal. Aunque tiene una versión de prueba de 30 días la cual vamos a utilizar a lo largo de este tema.

A continuación mostramos algunas de las **características de Visual Paradigm**:

- Facilita la reutilización, ya que disponemos de una herramienta centralizada donde se encuentran los modelos utilizados para otros proyectos.
- Coherencia entre diagramas.
- Corrección sintáctica.
- Permite generar diversos informes a partir de la información introducida en la herramienta.

Las herramientas de gestión de configuración se ocupan de:

Gestión de versiones y revisiones.

Gestión de configuración de línea de base.

Gestión de control de cambios.

Análisis de sistemas de información.

El **objetivo** de este proceso es la obtención de una especificación detallada del sistema de información que satisfaga las necesidades de información de los usuarios y sirva de base para el posterior **diseño del sistema**.

Principios del análisis.

Lo primero que debemos hacer para construir un sistema de información es averiguar qué es exactamente lo que tiene que hacer el sistema. La **etapa de análisis en el ciclo de vida del software** corresponde al proceso mediante el cual se intenta descubrir qué es lo que realmente se necesita y se llega a una comprensión adecuada de los requerimientos del sistema.

Ingeniería de requisitos.

El proceso de recogida de información, análisis y documentación sobre los requisitos software des del cliente, se conoce como ingeniería de requisitos.

El objetivo de este tipo de Ingeniería es el de desarrollar y mantener un documento de especificación de requisitos del sistema de forma sofisticada y descriptiva.

Es un proceso que consta de cuatro pasos:

- Estudio de viabilidad.
- Recogida de requisitos.
- Requisitos del Software.
- Validación de los requisitos de Software.

Estudio de viabilidad

Cuando el cliente se acerca a la organización para obtener el producto deseado desarrollado, expone una idea aproximada de las funciones que el software debe cumplir y qué características se esperan del software.

Refiriéndose a esta información, los analistas elaboran un estudio detallado sobre la viabilidad del sistema deseado y de sus funcionalidades, para

proceder a desarrollarlo.

Este estudio de viabilidad se centra en el objetivo de la organización. El estudio analiza la materialización práctica del producto software respecto a su implementación, la contribución de proyecto a la organización, los límites de costes, y según los objetivos y valores de la organización. Explora aspectos técnicos del proyecto y del producto, como la utilidad, el mantenimiento, la productividad y la capacidad de integración.

El resultado o output de esta fase debe ser un informe del estudio de viabilidad, conteniendo comentarios adecuados y recomendaciones para la gestión sobre si se debe tirar adelante o no el proyecto.

Recogida de requisitos

Si el informe de viabilidad es positivo en relación a tomar el proyecto, la siguiente fase empieza con la recolección de requisitos por parte del consumidor. Analistas e Ingenieros se comunican con el cliente y los consumidores para conocer sus ideas sobre qué debe aportar el software y qué características quieren que incluya este.

Requisitos del Software

El SRS (Especificación de requisitos de software) es un documento creado por los analistas de sistema después de recoger los requisitos.

El SRS define cómo va a interactuar el software que quiere crearse con el hardware, las interfaces externas, la velocidad operativa, el tiempo de respuesta del sistema, la portabilidad del software en las diversas plataformas, el mantenimiento, la velocidad de reponerse después de estropearse, su seguridad, calidad, limitaciones, etc.

Los requisitos recibidos por parte del cliente se escriben en lenguaje natural. Es responsabilidad del analista de sistemas documentar sobre los requisitos en lenguaje tecnológico para que puedan ser útiles y comprendidos por el equipo de desarrollo de software.



El **SRS** debe venir con las siguientes características:

- Los requisitos del usuario se deben expresar en lenguaje natural.
- Los requisitos técnicos se deben expresar en lenguaje estructurado, el cual se usará dentro de la organización.
- La descripción del diseño se debe escribir en pseudocódigo.
- El formato de Forms y GUI impresiones de pantalla.
- Anotaciones condicionales y matemáticas para DFD.

Validación de los requisitos de Software

Después del desarrollo de los requisitos, los que se mencionen en este documento serán validados. El usuario puede que pida soluciones ilegales y poco prácticas, y los expertos pueden que interpreten los requisitos de forma incorrecta. Estos resultados se incrementan en coste si no se cortan de raíz. Los requisitos se pueden evaluar en contraste con las siguientes condiciones:

- Si pueden ser implementados de manera práctica.
- Si son válidos a nivel de funcionalidad y dominio del software.
- Si hay alguna ambigüedad.
- Si se han completado.
- Si se pueden demostrar.

El proceso de recogida de información, análisis y documentación sobre los requisitos software del cliente, se conoce como:

Ingeniería de organización.

Ingeniería de requisitos.

Informes del análisis.

La **Gestión de proyectos** lleva una serie de riesgos que han de conocerse y tener en cuenta. Veamos dichos problemas y riesgos:

- No existencia de unas normas de gestión del proyecto. Desarrollar unas Normas de Gestión del Proyecto adecuado que recoja la definición del proyecto, la planificación global, las decisiones tomadas en la reunión Kick-Off (principales mecanismos de gestión y ejecución del proyecto), responsabilidades y línea base aprobada para guiar la ejecución y control del proyecto.
- Planificaciones no ajustadas a la realidad debido a: estimaciones erróneas, movilidad de los recursos, insuficiencia de recursos (cantidad y personal), fechas prefijada y cambios de requisitos. [severidad: alta] Utilizar los procedimientos, herramientas, técnicas y base de datos históricos adecuados para estimar el tamaño, el esfuerzo, el coste y los recursos necesarios para la planificación y seguimiento de proyectos.
- No se tiene una visión del estado real de los proyectos y por ende ni visibilidad de la evolución del proyecto. Establecer hitos principales y reuniones de seguimiento periódicas para realizar informes del progreso del proyecto.
- Retraso en la entrega de las aplicaciones e, incluso, reducción en las funcionalidades de las mismas. Realizar inventario de análisis de riesgos y problemas y mantenerlos actualizados durante todo el desarrollo del proyecto.
- Falta de procedimientos de comunicación entre los grupos de las distintas direcciones en el ámbito de un proyecto. Desarrollar un Plan

de Comunicación en el que se establezcan las necesidades de comunicación durante el desarrollo del proyecto.

- Falta de una matriz de escalado. Identificar donde o a quien escalar antes de un bloqueo en el proyecto.
- Falta de compromiso del cliente para el proyecto. Validar el plan de trabajo con el cliente, estableciendo compromisos del cliente para el proyecto y definiendo su participación dentro del mismo.
- Falta de cumplimiento de acuerdos establecidos durante reuniones con el cliente. Generación de agendas y actas para cada reunión con el cliente, y validación de las actas por parte del cliente.
- Existencia de personal del equipo de trabajo, sin que tenga establecida su participación en el proyecto. Definición de roles y responsabilidades de todos los miembros de equipo de acuerdo al plan de trabajo definido
- Documentación del progreso del proyecto obsoleto o sin cumplir con los estándares establecidos para su realización. Actualización periódica de la información en la carpeta de proyecto y revisión de los estándares de nomenclatura de todos los archivos.
- Pérdida de contacto con clientes potenciales, para futuros proyectos. Mantenimiento del contacto con el cliente tras el proyecto, especialmente si existen oportunidades de continuidad (actualización de la BBDD de contactos).
- Modificaciones del alcance en el transcurso del proyecto. Informar al cliente de los próximos pasos y de las tareas pendientes para gestionar expectativas y anticiparse. Mantener una comunicación constante de los responsables del proyecto – Reuniones de seguimiento. Revisar periódicamente las actas de reunión de kick-off y propuesta inicial con todo el equipo de proyecto.

Análisis de funciones o procesos.

Para escribir **programas** que den los **mejores resultados**, debemos de tener en cuenta una serie de **detalles**.

1. Corrección. Un programa es correcto si hace lo que tiene que hacer tal y como se especifico en las fases previas al desarrollo. Para especificar si un programa hace lo que debe, es muy importante determinar claramente qué tiene que hacer el programa antes de desarrollarlo y, una vez finalizado, compararlo con lo que realmente hace.



2. Claridad. Es de vital importancia que el programa sea lo más claro y legible que se pueda posible, para agilizar así su desarrollo y posterior mantenimiento. Al crear un programa se debe intentar que su estructura sea simple y coherente, así como cuidar el estilo en la edición; de tal forma que se ve facilitado el trabajo del programador, tanto en la fase de creación como en las fases siguientes de corrección de errores, ampliaciones, modificaciones, etc.

3. Eficiencia. Se trata de que el programa, además de realizar aquello para lo que fue creado (es decir, que sea correcto), lo haga gestionando de la mejor forma posible los recursos que utiliza.

4. Portabilidad. La portabilidad es una característica muy deseable para un programa, ya que permite, por ejemplo, a un programa que se ha desarrollado para sistemas GNU/Linux ejecutarse también en la familia de sistemas operativos Windows.

Análisis de datos.

Un **lenguaje** de programación se define como un idioma artificial diseñado para expresar **procesos** que pueden llevarse a cabo por máquinas como las computadoras. Pueden utilizarse para crear programas que controlen el comportamiento físico y lógico de una determinada máquina, para determinar algoritmos con precisión, o como modo de comunicación humana.

Está conformado por una serie de símbolos y reglas sintácticas y semánticas

que determinan su estructura y el significado de sus elementos y expresiones. Al proceso por el cual se escribe, se testea, se depura, se compila y se sostiene el código fuente de un programa informático se le define como programación.

También la palabra **programación** se determina como el proceso de creación de un programa de computadora, a través de la aplicación de procedimientos lógicos, a través de los siguientes pasos:

- El desarrollo lógico del programa para resolver un problema en particular.
- Escritura de la lógica del programa empleando un lenguaje de programación específico (codificación del programa).
- Ensamblaje o compilación del programa hasta transformarlo en lenguaje de máquina.
- Prueba y depuración del programa.
- Desarrollo de la documentación.

Análisis orientado a objetos.

El análisis orientado a objetos es el proceso que modela el dominio del problema mediante la identificación y la especificación de un conjunto de objetos semánticos que interaccionan y se comportan de acuerdo a los requisitos del sistema.

El objetivo del **Análisis Orientado a Objetos (AOO)** es desarrollar un modelo que describa el software de computadora necesario para satisfacer los requisitos definidos por el cliente. El modelo de análisis contiene el funcionamiento y el comportamiento de los elementos del modelo de objetos.

Nadie tiene muy claro por qué el Análisis Orientado a Objetos ha tardado tanto tiempo en ser aplicado a pesar de que maneja conceptos que aprendimos en la guardería como objeto, clase, miembro etc. No existe un acuerdo sobre los conceptos que sirven de base para el AOO, aunque se repiten a menudo un número de ideas clave.

El **propósito** es definir todas las clases, atributos, operaciones y relaciones de comportamiento asociado entre ellos que sean relevantes al problema que se va a resolver. Para realizar dicho análisis se deben ejecutar las siguientes

tareas:

- Identificar los escenarios o casos de uso.
- Identificar las clases, Definir sus atributos y métodos.
- Especificar la jerarquía entre las clases.
- Representar las relaciones entre los diferentes objetos del sistema.
- Modelar el comportamiento de cada objeto.
- Repetir iterativamente las tareas anteriores hasta completar el modelo.

Hoy en día el *paradigma de orientación a objetos* es el más utilizado para programar y prácticamente lo podemos encontrar en la mayoría de los lenguajes de programación más utilizados: Java, PHP, C++, etc.

Pero, ¿qué es realmente la orientación a objetos? La orientación a objetos es una forma de programar que rompe con la forma de programar clásica.

En un entorno clásico de programación el problema o sistema que se está analizando se divide en funciones: este sistema hace esto, esto y esto.

Para llevar a cabo el análisis con la programación orientada a objetos hay que tener en cuenta que los programas se codifican pensando en objetos que interactúan.

Los objetos tendrán unas determinadas **propiedades** y serán capaces de hacer ciertas cosas mediante los **métodos** que se les hayan definido.

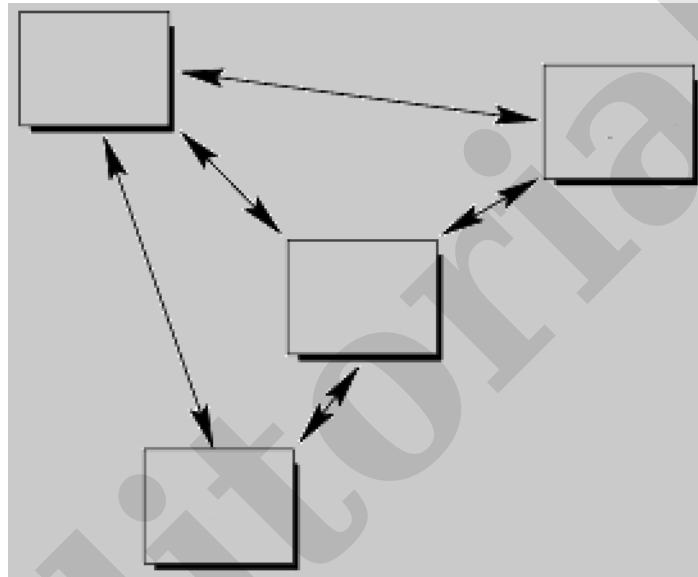
Si se hace un buen diseño de los objetos y de las clases, se podrán usar en nuevos programas y proyectos fácilmente, favoreciendo la **reusabilidad**.

La orientación a objetos es una forma de programar que rompe con la forma de programar clásica, esta programación codifica pensando en objetos que interactúan entre sí. Cada objeto tendrá unas determinadas propiedades y serán capaces de realizar ciertas cosas a través de los métodos definidos.

La ventaja de este tipo de programación es que si realizamos un buen diseño, este lo podremos reutilizar en futuros proyectos con mayor facilidad sin tener que volver a generarlos.

Cuando se diseña o analiza un sistema software relativamente complejo, se suele dividir en partes más sencillas y diferenciables, llamadas **módulos**. A esta técnica de análisis se la conoce como **Modularidad**.

El objetivo de la modularidad no es otro que hacer más sencillas las labores de desarrollo y testeo del **sistema**. Podrán diferenciarse y repartirse mejor las tareas de desarrollo, lo que en la práctica se traduce en poder realizar en paralelo el desarrollo de los módulos. Esto supone un ahorro de tiempo y la posibilidad de testear de manera independiente el funcionamiento de cada módulo por separado y finalmente el comportamiento del sistema al completo.



Para distinguir el concepto de módulo en el paradigma orientado a objetos vamos a ver un ejemplo real, y para ello vamos a suponer una aplicación empresarial *Java EE*.

Este tipo de aplicaciones están basadas en el paradigma orientado a objetos y están formadas por un conjunto de módulos donde cada módulo es un conjunto de uno o más componentes que se ejecutan en el mismo contenedor.

Un componente es un elemento de **software**. Podría ser un componente web como una página JSP o un servlet, un componente EJB, etc. Los componentes se ejecutan dentro de su correspondiente contenedor.

El contenedor es un entorno de ejecución que gestiona los componentes, por eso, los componentes deben de cumplir un contrato que establece el contenedor. Ese contrato no es más que un **conjunto de métodos** que se

deben implementar en el componente y que permite al contenedor interactuar con él.

El contenedor es el encargado de gestionar el ciclo de vida de los componentes, realizar la reserva de recursos, etc.

La Programación Orientada a Objetos tiene como principal finalidad:

Economizar código

Duplicar código

Eliminar código

Encapsular código

Señala si este enunciado es verdadero o falso: El paradigma de orientación a objetos es el más utilizado para Programar y prácticamente lo podemos encontrar en la mayoría de los lenguajes de programación más utilizados: Java, PHP, C++, etc.

Verdadero

Falso

Diagramas UML para el análisis de sistemas.

El **modelo dinámico** es usado para determinar y modelar el funcionamiento

del sistema a lo largo de un determinado espacio de tiempo.

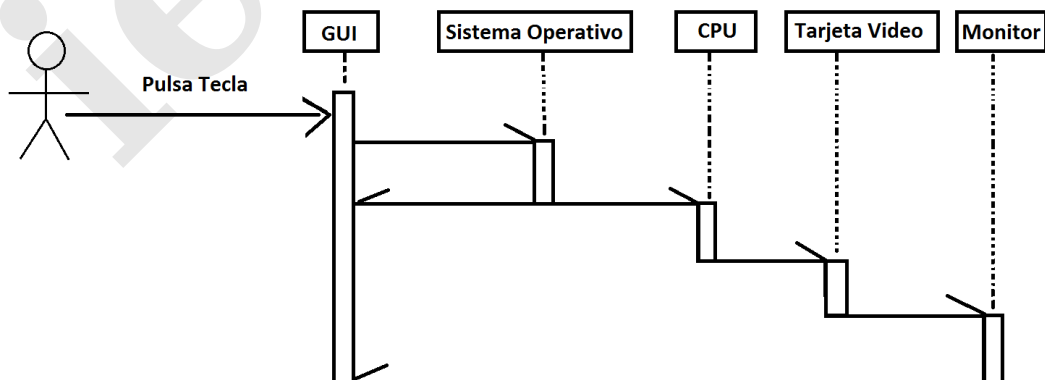
Está constituido por los aspectos de un sistema que tiene relación con el tiempo y con los cambios en los objetos. Además incluye una serie de tareas que actúan como soporte para los diagramas de secuencia, diagramas de comunicación y diagramas de estados.

Diagrama de secuencia

Una secuencia es una sucesión ordenada de acciones que guardan relación entre sí, en el que cada acción está determinada por el resultado de la anterior. Los diagramas de secuencia sirven para definir y exponer la dinámica de funcionamiento de un sistema y además se utilizan para representar y modelar la interacción entre los usuarios, las pantallas y las instancias de los objetos en el sistema, o lo que es lo mismo, en los diagramas de secuencia se muestran las acciones establecidas en un sistema y la correlación entre las mismas.

La dinámica de un sistema se presenta mediante un conjunto de diagramas de secuencia, vinculados cada uno de ellos a una subfunción del sistema.

El **diagrama** de secuencia incorpora un plano secuencial del camino que recorren los mensajes entre los objetos a lo largo del tiempo. En él se representan las interacciones entre los grupos de objetos presentando de forma secuencial los envíos de mensajes entre objetos.



Habitualmente, los diagramas de secuencias se encuentran bajo los casos de uso o los componentes en el modelo para representar un escenario, en el cual muestra cómo interactúa un usuario con el sistema y qué sucede en el interior para que todo se realice sin problemas.

Un diagrama de secuencias esta compuesto por:

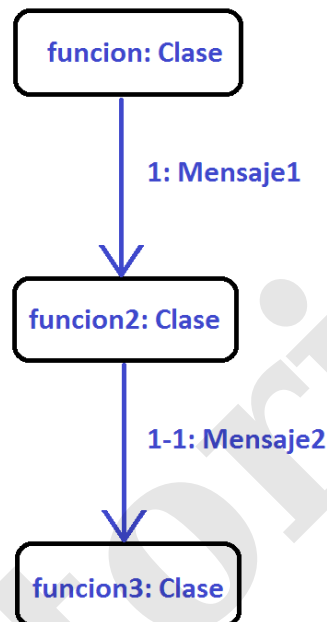
- Mensajes entre objetos: Los objetos pueden pasarse mensajes entre sí para realizar o completar acciones.
- Línea de vida activa: Esta línea representa el periodo de acción del objeto.
- Objetos: Las instancias que realizarán acciones a representar.

Diagrama de comunicación

El diagrama de comunicación es también conocido o denominado con el nombre de diagrama de colaboración. Es un diagrama de interacción el cual muestra una información parecida al diagrama de secuencia, la única diferencia es que se basa principalmente en la relación de los objetos.

El diagrama de comunicación modela las interacciones entre objetos o mensajes. Además representan un conjunto de **información** tomada desde el diagrama de clases, secuencia y diagrama de casos de uso describiendo la estructura estática y el comportamiento dinámico del sistema

Diagrama De Comunicación



El **diseño** de los **diagramas de comunicación** no tiene un formato específico, es decir, se pueden orientar o determinar los objetos en aquella posición en la que destaquen más sus relaciones. Antes de generar los mensajes se recomienda generar todos los objetos que intervienen en la interacción y posicionarlos de forma lógica. Generalmente, un diagrama de comunicación representa una instancia o un trazado de ejecución del programa.

Marcos de interacción

Al igual que en un sistema tenemos secuencias de acciones, en un sistema existen acciones o procesos en los cuales éstas actúan entre sí o dependen de otras, y también necesitan ser representadas debidamente. Para ello, disponemos de los marcos de interacción, que a su vez constituyen una parte del diagrama de secuencia. Estos diagramas van asociados a una etiqueta, la cual dispone de un operador que especifica la forma en que se ejecuta el diagrama.

El marco de interacción se representa mediante una región bordeada con un operador en el extremo superior izquierdo, el cual señala el funcionamiento que se desea representar, y seguidamente una condición que tiene que cumplirse para que la interacción que contiene el marco se ejecute.

Podemos distinguir 5 tipos de operadores:

- alt: Una ejecución alternativa que se ejecutará siempre y cuando la condición sea cierta.
- opt: Determina que la condición tiene que cumplirse para que se pueda ejecutar la secuencia del marco de interacción.
- par: El marco de interacción dispone de varias secuencias que se ejecutarán en paralelo.
- loop: Basándose en la condición el marco de interacción se ejecutará varias veces.
- region: Determina una región crítica que no puede ejecutarse en multihilo.

3. Modelado de objetos

Como ya hemos visto en las unidades anteriores, la finalidad del modelado de objetos UML es la de mejorar la calidad de nuestro software, permitiéndonos reducir considerablemente los tiempos de desarrollo, aumentando la productividad e incrementando las posibilidades de reutilización del mismo.

Cualquier procedimiento que nos ayude a producir un sistema partiendo de algún nuevo uso de un elemento anteriormente desarrollado, supone un ahorro de esfuerzos considerable, reduciendo así mismo la complejidad del sistema.

A continuación desarrollaremos los conceptos sobre el modelado de objetos.

Una secuencia es una sucesión ordenada de acciones que guardan relación entre sí, en el que cada acción está determinada por el resultado de la anterior.

Verdadero.

Falso.

ieeditorial

Diseño de software.

Un diseño puede ser caracterizado mediante una regla con tres partes, que expresa una relación entre: (a) un determinado contexto, (b) un problema y (c) una solución ha dicho problema.

Se pretende encontrar una solución al problema que satisfaga mejor el conjunto de requisitos e intereses de los afectados que reflejan como debe resolverse el problema en cada contexto específico, incluso cuando dichos intereses sean contradictorios.

Caracterización de un patrón de diseño efectivo

- Resuelve un problema: los patrones capturan soluciones, no solo principios abstractos y soluciones.
- Se trata de un concepto comprobado: los patrones capturan soluciones que poseen una "trayectoria", no se trata de teorías o pura especulación.
- Presentan soluciones no obvias: muchas técnicas de resolución de problemas (tales como paradigmas o métodos de diseño de software) intentan derivar soluciones de principios fundamentales. Los mejores patrones generan indirectamente una solución a un problema; lo cual es el enfoque adecuado para atacar los problemas de diseño más difíciles.

Describen una relación: los patrones no solo describen módulos, sino que describen mecanismos y estructuras del sistema más profundas.

Los patrones minimizan la **intervención humana**: todo software aporta confort o calidad de vida; los mejores patrones explícitamente reclaman estética y utilidad.

Los buenos patrones de diseño capturan un conocimiento práctico sobre como diseñar un sistema que permite a otros reutilizar dicho conocimiento un millón de veces sin tener que hacerlo de la misma manera dos veces seguidas.

Los patrones de diseño, si se utilizan con efectividad, producirán siempre un mejor diseño de software.

Tipos de patrones

Patrones creacionales

Se enfocan hacia la creación, composición y representación de objetos. Proporcionan mecanismos que facilitan la instanciación de objetos en un sistema y refuerzan las restricciones en el tipo y número de objetos que se pueden crear dentro de un sistema.

Patrones estructurales

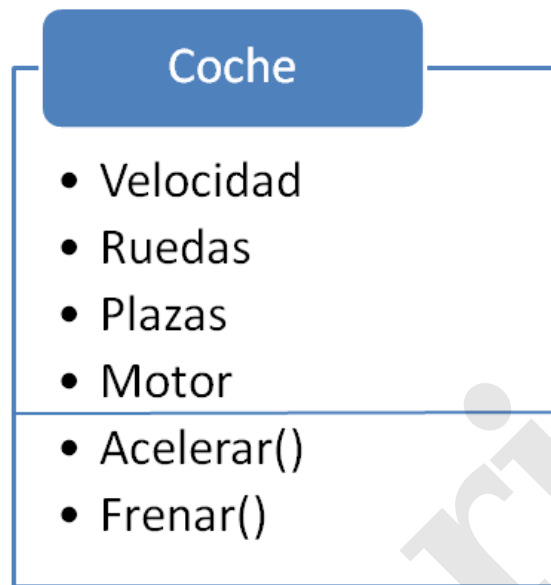
Se enfocan hacia los problemas relacionados con la organización e integración de clases y objetos dentro de estructuras grandes. Por ejemplo, patrones estructurales relacionados con las clases pueden propiciar la definición de mecanismos de herencia que lleven al desarrollo de interfaces más eficaces. Patrones con una orientación a objetos más explícita dan la pista para combinar los objetos entre sí o para integrarlos en una estructura más amplia.

Patrones comportamentales

Sirven para resolver problemas que tienen que ver con la asignación de responsabilidades entre objetos y la forma en que se lleva a cabo la comunicación entre estos.

Enfoques en el diseño.

El comportamiento de un **objeto** viene definido por la serie de **procedimientos** o **funciones** que definimos este puede desarrollar. Estos métodos, que también definimos nosotros, son operaciones que se pueden realizar con el objeto. Si en un objeto coche tenemos los métodos Acelerar y Frenar, que aumentan o disminuyen el valor de la variable Velocidad, estos procedimientos constituyen el comportamiento del objeto coche.



La identidad, como en cualquier otra variable, no es más que el identificador. Cuando definimos “cómo” es un objeto, después podemos crear ese tipo de objetos. Si ya hemos definido nuestro objeto coche, podemos declarar diferentes instancias de este objeto, como coche01, coche02, etc. Cada vez que queramos acceder a cualquiera de los métodos o atributos, lo haremos llamando al objeto por el identificador correspondiente.

El elemento que conforma la definición de un objeto recibe el nombre de Clase. Una clase no es otra cosa que una plantilla para la creación de objetos, según un modelo que definamos. Las clases nos permiten abstracción sobre los datos y las operaciones asociadas al objeto. A continuación explicaremos más extensamente el concepto de abstracción.

Niveles de diseño.

En los distintos niveles del diseño podemos encontrarnos con:

- UML Diagrama de clases de diseño Diagrama de componentes.
- Diagrama de despliegue.
- Diagrama de secuencia.

- Diagrama de colaboración.
- Diagrama de estados Diseño de datos.

Principios del diseño.

Se hace referencia a determinar cómo funcionará el **software** de forma común sin entrar en **especificaciones**. se basa en incorporar consideraciones de la implementación tecnológica, como son hardware, la red, etc. Se determinan los casos de uso para cubrir las funciones que hará el sistema, y se convertirá las entidades determinadas en el análisis de requisitos en clases de diseño, teniendo un modelo cercano a la programación orientada a objetos.

Obtener los requisitos y requerimientos de un producto de software es la primera etapa para generarlo. Mientras que los clientes piensan que ellos saben lo que el software tiene que realizar, se requiere de habilidad y experiencia en la ingeniería de software para reconocer requerimientos que no están completos, ambiguos o contradictorios. El resultado del análisis de especificaciones con el cliente se plasma en el documento ERS, Especificación de Requerimientos del Sistema, el cual tiene una estructura la cual puede venir definida por varios estándares, tales como CMMI. Asimismo, se determina un diagrama de Entidad/Relación, en el que se plasman las entidades principales que participarán en el desarrollo del software.

La **captura**, análisis y especificación de requerimientos, es una parte crucial; de esta etapa depende en gran medida el logro de los objetivos propuestos. Se han generado modelos y diversos procedimientos de trabajo para estos fines. Aunque aún no está de forma normalizada, ya se habla de la Ingeniería de requerimientos.

La IEEE Std. 830-1998 normaliza la creación de las especificaciones de requerimientos de software (Software Requirements Specification).

Diseño de la interfaz de usuario.

El diseño de **interfaz** de usuario o ingeniería de la interfaz es el resultado de definir la forma, función, utilidad, ergonomía, imagen de marca y otros aspectos que afectan a la apariencia externa de las interfaces de usuario en sistemas de todo tipo (computadoras de uso general, sistemas de control,

dispositivos de comunicación móviles, software de sistemas, software de aplicaciones, sitios web, etc). El diseño de la interfaz de usuario es una disciplina asociada al diseño industrial y se enfoca en maximizar la usabilidad y la experiencia de usuario. El **objetivo** final del diseño de la interfaz de usuario es hacer que la interacción entre el usuario y el sistema del que es interfaz sea tan simple y eficiente como sea posible, en términos de cumplimiento de los objetivos del usuario. Sigue por ello una filosofía de diseño centrado en el usuario.

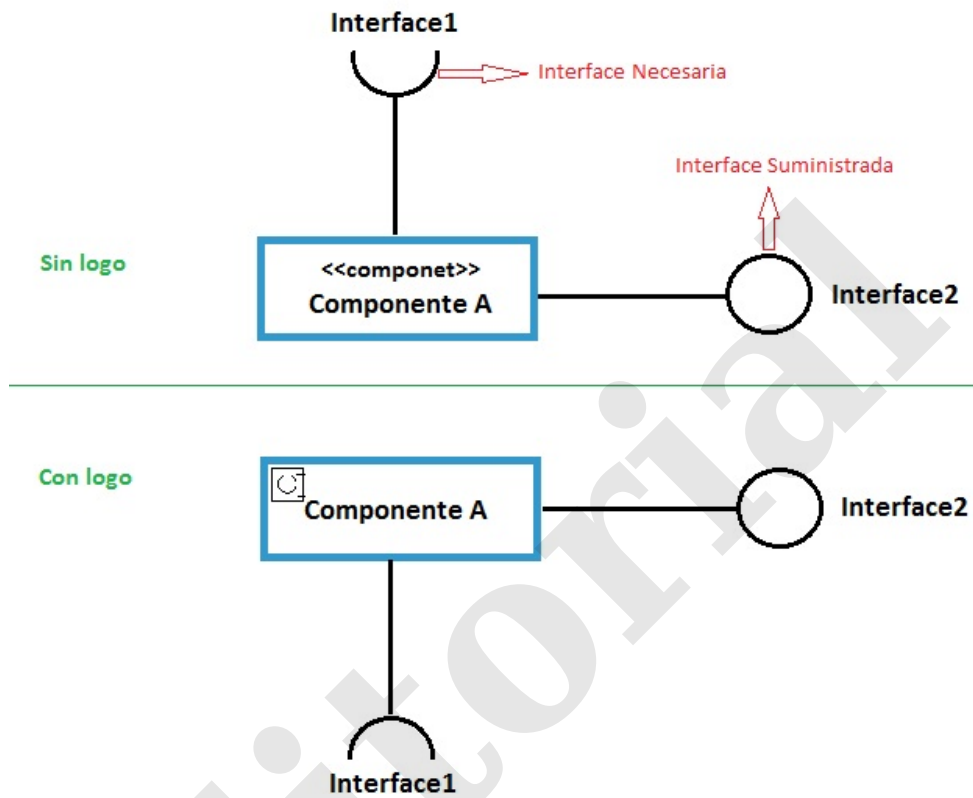
Un buen diseño de la interfaz de usuario facilita la compleción de tareas a realizar sin que el usuario vea atraída su atención hacia la forma. El diseño gráfico y la tipografía se combinan para ofrecer usabilidad, influyendo en cómo el usuario realiza ciertas interacciones y mejorando la apariencia estética del diseño; la estética del diseño puede mejorar o dificultar la capacidad de los usuarios para utilizar las funciones de la interfaz. El proceso de diseño debe balancear la funcionalidad técnica y los elementos visuales (es decir, el modelo mental) para crear un sistema que no solo sea operativo, sino también usable y adaptable a la evolución de las necesidades del usuario.

Normalmente el diseño de interfaces de usuario es una actividad multidisciplinar que involucra a varias ramas tales como el diseño gráfico, el diseño industrial, el diseño web, el diseño de software y la ergonomía; y puede aparecer como actividad en un amplio rango de proyectos, desde el desarrollo de sistemas informáticos hasta el desarrollo de aviones comerciales. En este sentido las **disciplinas** del diseño industrial y diseño gráfico se encargan de que la actividad a desarrollar se comunique y aprenda lo más rápidamente, a través de recursos como los gráficos, los pictogramas, los estereotipos y la simbología, todo sin afectar un funcionamiento técnico eficiente.

Diagrama

Las interfaces suministradas se representan con un círculo, mientras que las **interfaces** necesarias se representan mediante un semicírculo. La notación gráfica de los componentes es dentro de un rectángulo con el estereotipo <>, el estereotipo puede ser sustituido por el logo del componente.

A continuación podemos visualizar una representación gráfica de un componente y sus interfaces. Vemos la representación del componente tanto con el logo como con el **estereotipo**.



El objetivo final del diseño de la interfaz de usuario es hacer que la interacción entre el usuario y el sistema del que es interfaz sea tan simple y eficiente como sea posible, en términos de cumplimiento de los objetivos del usuario. Sigue por ello una filosofía de diseño centrado en el usuario.

Verdadero.

Falso.

Diseño arquitectónico.

El Diseño arquitectónico, es la versión más abstracta del sistema. Identifica el software como un sistema con distintos componentes que interactúan entre ellos. En este momento es cuando los diseñadores conciben la idea de posibles soluciones de dominio.

El **diagrama** de despliegue describe la arquitectura física del sistema. Está compuesto de nodos. Un nodo es una unidad material capaz de recibir y de ejecutar software. La mayoría de nodos son ordenadores. Los vínculos físicos entre nodos también pueden describirse en el diagrama de despliegue, corresponden a las ramas de la red.

Los nodos contienen software en su forma física, conocida como artefact. Los archivos ejecutables, las bibliotecas compartidas y los scripts son ejemplos de formas físicas de software.

Los componentes que constituyen la **arquitectura del software** del sistema se representan en el diagrama de despliegue mediante un artefacto que, con frecuencia, es un ejecutable o una biblioteca compartida.

Diseño detallado.

El **Diseño** detallado diseña acuerdos con la parte de implementación de lo que se ve como **sistema** y sus **subsistemas** con los dos tipos de diseño mencionados con anterioridad. Es más detallado en cuanto a los módulos y a su implementación. Define estructuras lógicas de cada módulo y de sus interfaces para comunicarse con los otros módulos.

Técnicas de diseño.

Algunas reglas para el diseño de los diagramas de uso son:

- Mantener todos los casos de uso de un diagrama al mismo nivel de abstracción.
- Mantener el diagrama ordenado.

- Evitar tener demasiados casos de uso en el mismo diagrama.
- Evitar el uso complejo de relaciones de extensión especialización e inclusión.

Un diagrama de casos de uso está compuesto por los elementos que se muestran a continuación:

- Casos de uso.
- Relaciones de uso.
- Actor.
- Escenarios.

En definitiva. Un diagrama de casos de uso representa un esbozo general de un proceso y sus diferentes pasos, para facilitar su **comprensión** y, por tanto, su desarrollo en el proyecto, constituyendo un lenguaje gráfico sencillo y modulable.

Técnicas de diseño de datos.

Los **atributos**, como indicamos anteriormente, son los datos que hacen referencia al estado de un objeto, o lo que es lo mismo, a las variables que lo pueden componer. Los objetos pueden contener cualquier número de atributos, según las necesidades que tengamos al crearlo.

Los métodos son las diferentes **funciones** que pueden aplicarse desde el objeto, en el caso del ejemplo sería Acelerar y Frenar. Los métodos pueden o no modificar los atributos de la clase, según lo definamos. Diferentes procedimientos pueden formar parte de nuestro objeto, devolviendo valores, leyendo o modificando atributos, etc.

Por protección de nuestros **objetos**, no es conveniente que sus datos miembro o estado, sean accesibles de cualquier forma y en cualquier momento. Para evitar esto, se procede al encapsulamiento, que protege los datos asociados de un objeto contra su modificación por quien no tenga derecho a acceder a ellos, previniendo posibles problemas posteriores por usos indebidos o no autorizados. La encapsulación va de la mano de la abstracción, ya que parte de la finalidad del encapsulamiento también es el conocimiento innecesario del usuario del cómo y porqué funciona un objeto, limitándose a usarlo.

No es necesario que el **usuario** de un **objeto** sepa cómo está estructurado por dentro. Los de **encapsulamiento** nos proporcionan tres distintos niveles de acceso a los datos de un objeto o clase, que son los siguientes:

Público

Es el nivel de protección más bajo y permite el acceso de cualquier función a los datos miembro de la clase.

Protegido

El acceso a estos datos está restringido a esta clase y a sus subclases (Ver punto HERENCIA).

Privado

Es el nivel más alto de protección. Sólo los métodos de esta clase en particular podrán tener acceso a los datos.

Técnicas de diseño estructurado.

¿Qué es diseño estructurado?

El diseño es el proceso de aplicar distintas técnicas y principios con el objetivo de definir y orientar un dispositivo, proceso, o sistema, con los suficientes detalles como para permitir su realización física.

El diseño estructurado se enfoca en el desarrollo de especificaciones del software.

El objetivo del Diseño estructurado, es programas formados por módulos independientes unos de otros desde el punto de vista funcional.

Estas metodologías proponen modelos del sistema que representen los procesos, los flujos y las estructuras de datos de una forma descendente “top-

down". Se basan en el modelo básico entrada/proceso/salida, es decir los datos entran al sistema y éste los transforma para dar lugar a las salida.

Elementos del diseño estructurado

Técnicas de documentación: Incluyen herramientas gráficas y de texto. (Flujos de datos, Diagramas Hipo, Diagrama de estructura). La herramienta fundamental del Diseño estructurado es el diagrama estructurado que es de naturaleza gráfica y evitan cualquier referencia relacionada con el hardware o detalles físicos. Su finalidad no es mostrar la lógica de los programas (que es la tarea de los diagramas de flujo). Los Diagramas Estructurados describen la interacción entre módulos independientes junto con los datos que un módulo pasa a otro cuando interacciona con él.

Criterios de evaluación del Diseño: Tiene como objetivo evaluar los defectos del diseño conforme a la documentación del diagrama Hipo y el diagrama estructural.

Heurísticas del diseño: Se refieren al tamaño del módulo y al control del módulo.

Estrategias del Diseño: Las estrategias requieren un buen nivel de experiencia, sentido común y decisión. Existe una basada en el flujo de datos y otra basada en la estructura de datos.

Etapas del Diseño estructurado

Descomposición: Para ello se requiere un adecuado análisis de dicho problema, siendo necesario definir primeramente el problema, para lo cual deberá de contener una detallada pero concisa descripción del mismo.

Jerarquía de módulos: Ésta es una consecuencia directa de la descomposición del problema mediante refinamientos sucesivos, el resultado será un conjunto de módulos estratificados en capas a modo de pirámide donde en la cima habrá un único módulo que representará al programa global y en los niveles inferiores aparecerán los módulos resultantes de las sucesivas divisiones.

Independencia: independencia del diseño como tal.

Las técnicas para este modelo pueden ser:

- Declaración de los objetivos.
- Lista de eventos.
- Tabla de estímulo-respuesta.
- Diagrama de contexto.
- Diccionario de datos inicial.



Ejemplo de un diseño estructurado

Técnicas de diseño orientado a objetos.

El diseño Orientado a Objetos (DOO) difiere considerablemente del diseño estructurado ya que en DOO no se realiza un problema en términos de tareas (subrutinas) ni en términos de datos, sino que se analiza el problema como un sistema de objetos que interactúan entre sí.

Un problema desarrollado con técnicas orientadas a objetos requiere, en primer lugar saber cuáles son los objetos del programa. Como tales objetos son instancias de clases, la primera etapa en el desarrollo orientado a objetos requiere de la identificación de dichas clases (atributos y comportamiento), así como las relaciones entre éstas y su posterior implementación en un lenguaje de programación.

Aunque no siempre están bien delimitadas las etapas de análisis y diseño en la OO, se pueden sintetizar de alguna forma las ideas claves de las distintas tecnologías existentes dentro del desarrollo orientado a objetos al que denominaremos diseño.

En el diseño orientado a objetos, los objetos necesitan interactuar y comunicarse, para realizar dicha comunicación, los objetos utilizan su propia interfaz pública, dicha interfaz se compone principalmente de métodos y propiedades.

El diseño orientado a objetos transforma el modelo del análisis en un modelo de diseño que sirve como anteproyecto para la construcción de software.

Características principales del Diseño Orientado a Objetos:

- Los objetos son abstracciones del mundo real o entidades del sistema que se administran entre ellas mismas.
- Los objetos son independientes y encapsulan el estado y la representación de información.
- La funcionalidad del sistema se expresa en términos de servicios de los objetos.
- Las áreas de datos compartidas son eliminadas. Los objetos se comunican mediante paso de parámetros.
- Los objetos pueden estar distribuidos y pueden ejecutarse en forma secuencial o en paralelo

Ventajas del Diseño Orientado a Objetos:

- Fácil de mantener, los objetos representan entidades auto-contenidas.
- Los objetos son componentes reutilizables.
- Para algunos sistemas, puede haber un mapeo obvio entre las entidades del mundo real y los objetos del sistema.

Componentes del Diseño Orientado a Objetos

- La identificación de objetos, sus atributos y servicios
- La organización de objetos dentro de una jerarquía
- La construcción de descripciones dinámicas de objetos que muestran cómo se usan los servicios
- La especificación de interfaces de objetos

Atributos de la calidad del software.

El diseño es importante porque permite que un equipo de software evalúe la calidad de éste antes de que se implemente, momento en el que es fácil y barato corregir errores, omisiones o inconsistencias.

Durante el diseño, la calidad se evalúa por medio de la realización de una serie de revisiones técnicas. Una revisión técnica es una reunión celebrada por miembros del equipo de software. Por lo general, participan dos, tres o cuatro personas, en función del alcance de la información del diseño que se revisará.

Técnicas para el diseño:

- Debe tener una arquitectura que se haya creado con el empleo de estilos o patrones arquitectónicos reconocibles, esté compuesta de componentes con buenas características de diseño y se implementen en forma evolutiva de modo que faciliten la implementación y las pruebas.
- Debe ser modular, es decir, el software debe estar dividido de manera lógica en elementos o subsistemas.
- Debe contener distintas representaciones de datos, arquitectura, interfaces y componentes.
- Debe conducir a estructuras de datos apropiadas para las clases que se van a implementar y que surjan de patrones reconocibles de datos.
- Debe llevar a componentes que tengan características funcionales independientes.
- Debe conducir a interfaces que reduzcan la complejidad de las conexiones entre los componentes y el ambiente externo.
- Debe obtenerse con el empleo de un método repetible motivado por la información obtenida durante el análisis de los requerimientos del software.
- Debe representarse con una notación que comunique con eficacia su significado.

Ventajas del Diseño Orientado a Objetos:

Fácil de mantener, los objetos representan entidades auto-contenidas.

Para algunos sistemas, puede haber un mapeo obvio entre las entidades del mundo real y los objetos del sistema.

Los objetos son componentes únicos.

Los objetos son componentes reutilizables.

Diagramas UML para el diseño de software.

El Lenguaje Unificado de Modelado o UML ("Unified Modeling Language") es un lenguaje estandarizado de modelado. Está especialmente desarrollado para ayudar a todos los intervinientes en el desarrollo y modelado de un sistema o un producto software a describir, diseñar, especificar, visualizar, construir y documentar todos los artefactos que lo componen, sirviéndose de varios tipos de diagramas.

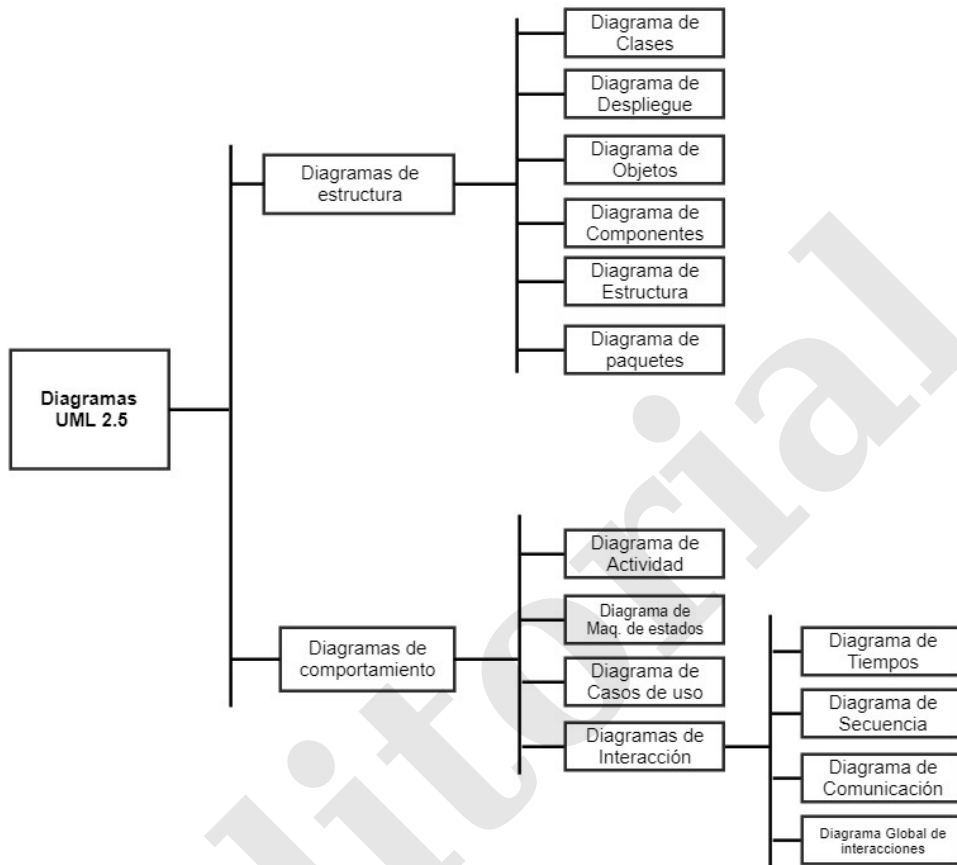
Estos diagramas contenidos en UML son la forma más común y más utilizada de modelado de software. Modelar consiste en crear un diseño previo de una aplicación antes de proceder a su desarrollo e implementación, aunque en ocasiones concretas puede hacerse posteriormente. De la misma forma que un arquitecto dibuja y diseña planos sobre el edificio que va a construir, un analista de software (u otros perfiles que cargan con este rol) crea distintos diagramas UML que sirven de base para la posterior

construcción/mantenimiento del sistema. El modelado es la principal forma de visualizar el diseño de una aplicación con la finalidad de compararla con los requisitos antes de que el equipo de desarrollo comience a codificar.

¿Por qué UML?

UML ofrece ese modelado utilizando diagramas y se denomina lenguaje por ser una forma común de expresarse por todos los analistas, desarrolladores y usuarios. Está desarrollado para ayudar a todos estos perfiles a especificar, visualizar, construir y documentar todos los componentes de un proyecto. Cada diagrama UML en particular aporta su visión particular al modelado, el lenguaje en su conjunto tiene algunas características:

- Es muy sencillo. Pese a que si es usado de forma completa puede llegar a complicarse, lo normal es que se simplifique.
- Es capaz de modelar todo tipo de sistemas.
- Es un lenguaje universal, haciendo que todos los miembros del equipo se relacionen a través de sus diagramas sean del ámbito que sean.
- Es fácilmente extensible. Tiene mecanismos sencillos para especializar los conceptos fundamentales.
- Es visual y, por lo tanto, intuitivo.
- Es independiente del desarrollo, del lenguaje y de la plataforma.
- Bien ejecutado aporta un conjunto considerable de buenas prácticas.
- No está completo. Utilizando los distintos diagramas no podemos estar seguros de comprender con totalidad el sistema que va a desarrollarse. Los diagramas, para facilitar su comprensión y suelen omitir información, pueden tener partes que se entienden de distintas maneras o, incluso, pueden tener conceptos que no pueden ser representados por ningún diagrama.



Existen dos clasificaciones de diagramas dentro de los UML: Los diagramas estructurales y los diagramas de comportamiento.

Prueba del software.

Existen muchos tipos de **testing**, o **pruebas de software**, que podemos usar para confirmar que nuestro software continúa funcionando correctamente tras introducir cambios nuevos sobre nuestro código fuente.

No todas las pruebas son iguales. Es por ello que en este artículo vamos a ver cómo difieren las principales pruebas de software.

Las personas pueden tener definiciones que difieren un poco entre sí, pero la **idea general** es la misma.

También hay que tener en cuenta que a veces los equipos se organizan para ejecutar conjuntos de pruebas. A estos grupos de pruebas se les conoce como "**test suites**" e incluyen pruebas de los distintos tipos.

Por ejemplo, es posible ejecutar un "test suite" que englobe integration tests y regression tests.

También ten en cuenta que en algunos casos los equipos deciden "armar su propio vocabulario" y asignan nombres a sus grupos de tests.

Pruebas unitarias.

Pruebas Unitarias (Unit test) o de Componente

Las **pruebas unitarias** son a bajo nivel (cercanas al código fuente de nuestra aplicación).

Este tipo de testing consiste en probar de forma individual las funciones y/o métodos (de las clases, componentes y/o módulos que son usados por nuestro software).

Debido a lo específicas que son, generalmente son las pruebas automatizadas de menor coste, y pueden ejecutarse rápidamente por un servidor de continuous integration (integración continua).

Más detalles acerca de las pruebas unitarias:

- Idealmente, cuando planeamos y escribimos pruebas unitarias, debemos aislar la funcionalidad hasta un punto en que no se pueda desglosar más, y entonces escribir pruebas a partir de ello. Justamente, el nombre de este tipo de testing hace referencia a una "unidad de código", que es independiente del resto.
- Estas pruebas verifican que el nombre de la función o método sea adecuado, que los nombres y tipos de los parámetros sean correctos, y así mismo el tipo y valor de lo que se devuelve como resultado.
- Dado que las pruebas unitarias no deben tener ningún tipo de dependencia, se suele reemplazar los llamados a APIs y servicios externos por funcionalidad que los imite (para que no exista interacción que vaya más allá de la unidad que está siendo probada).
- Si no es factible aislar el uso de bases de datos de nuestras pruebas unitarias, será importante tener en cuenta el rendimiento y buscar optimizar nuestras consultas.

En muchos casos inclusive se suele reemplazar las consultas a bases de datos, de modo que la prueba se enfoque en operar a partir de los valores de entrada, sin depender de ninguna fuente externa.

Esto es importante, porque si nuestras pruebas unitarias son de larga duración, resultará incómodo ejecutarlas y ralentizará significativamente los tiempos de desarrollo.

Cuando se habla de **Test Driven Development** (desarrollo guiado por pruebas), se hace referencia a unit tests. Es decir, se usan pruebas de este tipo como especificaciones de lo que nuestro código debe hacer.

Pruebas del sistema.

Pruebas de Sistema

Las pruebas del sistema tienen como objetivo ejercitar profundamente el sistema comprobando la integración del sistema de información globalmente, verificando el funcionamiento correcto de las interfaces entre los distintos subsistemas que lo componen y con el resto de sistemas de información con los que se comunica.

Son pruebas de integración del sistema de información completo, y permiten probar el sistema en su conjunto y con otros sistemas con los que se relaciona

para verificar que las especificaciones funcionales y técnicas se cumplen. Dan una visión muy similar a su comportamiento en el entorno de producción.

Una vez que se han probado los componentes individuales y se han integrado, se prueba el sistema de forma global. En esta etapa pueden distinguirse los siguientes tipos de pruebas, cada uno con un **objetivo** claramente diferenciado:

Pruebas funcionales

Dirigidas a asegurar que el sistema de información realiza correctamente todas las funciones que se han detallado en las especificaciones dadas por el usuario del sistema.

Pruebas de comunicaciones

Determinan que las interfaces entre los componentes del sistema funcionan adecuadamente, tanto a través de dispositivos remotos, como locales. Asimismo, se han de probar las interfaces hombre/máquina.

Pruebas de rendimiento

Consisten en determinar que los tiempos de respuesta están dentro de los intervalos establecidos en las especificaciones del sistema.

Pruebas de volumen

Consisten en examinar el funcionamiento del sistema cuando está trabajando con grandes volúmenes de datos, simulando las cargas de trabajo esperadas.

Pruebas de sobrecarga

Consisten en comprobar el funcionamiento del sistema en el umbral límite de los recursos, sometiénolo a cargas masivas. El objetivo es establecer los

puntos extremos en los cuales el sistema empieza a operar por debajo de los requisitos establecidos.

Pruebas de disponibilidad de datos

Consisten en demostrar que el sistema puede recuperarse ante fallos, tanto de equipo físico como lógico, sin comprometer la integridad de los datos.

Pruebas de facilidad de uso

Consisten en comprobar la adaptabilidad del sistema a las necesidades de los usuarios, tanto para asegurar que se acomoda a su modo habitual de trabajo, como para determinar las facilidades que aporta al introducir datos en el sistema y obtener los resultados.

Pruebas de operación

Consisten en comprobar la correcta implementación de los procedimientos de operación, incluyendo la planificación y control de trabajos, arranque y re arranque del sistema, etc.

Pruebas de entorno

Consisten en verificar las interacciones del sistema con otros sistemas dentro del mismo entorno.

Pruebas de seguridad

Consisten en verificar los mecanismos de control de acceso al sistema para evitar alteraciones indebidas en los datos.

Relaciona los elementos de la columna Derecha con la columna Izquierda

Pruebas de sobrecarga

1

Pruebas funcionales

2

Pruebas de volumen

3

Consisten en comprobar el funcionamiento del sistema en el umbral límite de los recursos, sometiénolo a cargas masivas.

Dirigidas a asegurar que el sistema de información realiza correctamente todas las funciones que se han detallado en las especificaciones dadas por el usuario del sistema.

Consisten en examinar el funcionamiento del sistema cuando está trabajando con grandes volúmenes de datos, simulando las cargas de trabajo esperadas.

Las pruebas del sistema tienen como objetivo ejercitar profundamente el sistema comprobando la integración del sistema de información globalmente, verificando el funcionamiento correcto de las interfaces entre los distintos subsistemas que lo componen y con el resto de sistemas de información con los que se comunica.

Pruebas de implantación.

El objetivo de las pruebas de implantación es comprobar el funcionamiento correcto del sistema integrado de hardware y software en el entorno de operación, y permitir al usuario que, desde el punto de vista de operación, realice la aceptación del sistema una vez instalado en su entorno real y en base al cumplimiento de los requisitos no funcionales especificados.

Una vez que hayan sido realizadas las pruebas del sistema en el entorno de desarrollo, se llevan a cabo las verificaciones necesarias para asegurar que el sistema funcionará correctamente en el entorno de operación. Debe comprobarse que responde satisfactoriamente a los requisitos de rendimiento, seguridad, operación y coexistencia con el resto de los sistemas de la instalación para conseguir la aceptación del usuario de operación.

Las pruebas de seguridad van dirigidas a verificar que los mecanismos de protección incorporados al sistema cumplen su objetivo; las de rendimiento a asegurar que el sistema responde satisfactoriamente en los márgenes establecidos en cuanto tiempos de respuesta, de ejecución y de utilización de recursos, así como los volúmenes de espacio en disco y capacidad; por último con las pruebas de operación se comprueba que la planificación y control de trabajos del sistema se realiza de acuerdo a los procedimientos establecidos, considerando la gestión y control de las comunicaciones y asegurando la disponibilidad de los distintos recursos.

Asimismo, también son llevadas a cabo las pruebas de gestión de copias de **seguridad y recuperación**, con el objetivo de verificar que el sistema no ve comprometido su funcionamiento al existir un control y seguimiento de los procedimientos de salvaguarda y de recuperación de la información, en caso de caídas en los servicios o en algunos de sus componentes. Para comprobar estos últimos, se provoca el fallo del sistema, verificando si la recuperación se lleva a cabo de forma apropiada. En el caso de realizarse de forma automática, se evalúa la inicialización, los mecanismos de recuperación del estado del sistema, los datos y todos aquellos recursos que se vean implicados.

Las verificaciones de las pruebas de implantación y las pruebas del sistema tienen muchos puntos en común al compartir algunas de las fuentes para su diseño como pueden ser los casos para probar el rendimiento (pruebas de sobrecarga o de stress).

El **responsable** de implantación junto al equipo de desarrollo determina las verificaciones necesarias para realizar las pruebas así como los criterios de aceptación del sistema. Estas pruebas las realiza el equipo de operación, integrado por los técnicos de sistemas y de operación que han recibido previamente la formación necesaria para llevarlas a cabo.

Pruebas de aceptación.

El objetivo de las pruebas de aceptación es validar que un sistema cumple con el **funcionamiento** esperado y permitir al usuario de dicho sistema que determine su aceptación, desde el punto de vista de su funcionalidad y rendimiento.

Las **pruebas** de aceptación son definidas por el usuario del sistema y preparadas por el equipo de desarrollo, aunque la ejecución y aprobación final corresponden al usuario.

Estas pruebas van dirigidas a comprobar que el sistema cumple los requisitos de funcionamiento esperado, recogidos en el catálogo de requisitos y en los criterios de aceptación del sistema de información, y conseguir así la aceptación final del sistema por parte del usuario.

El responsable de usuarios debe revisar los criterios de aceptación que se especificaron previamente en el plan de pruebas del sistema y, posteriormente, dirigir las pruebas de aceptación final.

La validación del sistema se consigue mediante la realización de pruebas de caja negra que demuestran la conformidad con los requisitos y que se recogen en el plan de pruebas, el cual define las verificaciones a realizar y los casos de prueba asociados. Dicho **plan** está diseñado para asegurar que se satisfacen todos los requisitos funcionales especificados por el usuario teniendo en cuenta también los requisitos no funcionales relacionados con el rendimiento, seguridad de acceso al sistema, a los datos y procesos, así como a los distintos recursos del sistema.

La formalidad de estas pruebas dependerá en mayor o menor medida de cada organización, y vendrá dada por la criticidad del sistema, el número de usuarios implicados en las mismas y el tiempo del que se disponga para llevarlas cabo, entre otros.

Pruebas de regresión.

El objetivo de las pruebas de regresión es eliminar el efecto onda, es decir, comprobar que los cambios sobre un componente de un sistema de

información, no introducen un comportamiento no deseado o errores adicionales en otros componentes no modificados.

Las pruebas de regresión se deben llevar a cabo cada vez que se hace un cambio en el sistema, tanto para corregir un error como para realizar una mejora. No es suficiente probar solo los componentes modificados o añadidos, o las funciones que en ellos se realizan, sino que también es necesario controlar que las modificaciones no produzcan efectos negativos sobre el mismo u otros componentes.

Normalmente, este tipo de pruebas implica la repetición de las pruebas que ya se han realizado previamente, con el fin de asegurar que no se introducen errores que puedan comprometer el funcionamiento de otros componentes que no han sido modificados y confirmar que el sistema funciona correctamente una vez realizados los cambios.

Las pruebas de regresión pueden incluir:

- La repetición de los casos de pruebas que se han realizado anteriormente y están directamente relacionados con la parte del sistema modificada.
- La revisión de los procedimientos manuales preparados antes del cambio, para asegurar que permanecen correctamente.
- La obtención impresa del diccionario de datos de forma que se compruebe que los elementos de datos que han sufrido algún cambio son correctos.

El responsable de realizar las pruebas de regresión será el equipo de desarrollo junto al técnico de mantenimiento, quien a su vez, será responsable de especificar el plan de pruebas de regresión y de evaluar los resultados de dichas pruebas.

Recuerda

[[[Elemento Multimedia]]]

ieeditorial

Autoevaluación

El diseño de los casos, debe considerar la elaboración de casos...

- a) Negativos.
- b) Positivos y negativos.
- c) Positivos.

El plan de pruebas que determina que funcionalidades del producto y/o software serán probadas durante el transcurso de la prueba es...

- a) Estrategia de pruebas.
- b) El tipo de prueba.
- c) El alcance de la prueba.

Indica si el siguiente enunciado es verdadero o falso:

“Pruebas de Integración: este tipo de pruebas son ejecutadas por el equipo de desarrollo y no consisten en la comprobación de que elementos del software que interactúan entre sí”.

- a) Verdadero.
- b) Falso.

Indica si la siguiente afirmación es verdadera o falsa: “El alcance de la prueba determina que funcionalidades del producto y/o software no serán probadas durante el transcurso de la prueba”.

- a) Verdadero.
- b) Falso.

Indica si la siguiente afirmación es verdadera o falsa: “Pruebas Unitarias o de Componente: este tipo de pruebas son ejecutadas normalmente por el equipo de desarrollo, básicamente consisten en la ejecución de actividades que le permitan verificar al desarrollador que los componentes unitarios están codificados bajo condiciones de robustez”.

a) Verdadero.

b) Falso.

ieeditorial

Soluciones :

1:

b) Positivos y negativos.

2:

c) El alcance de la prueba.

3:

b) Falso.

4:

b) Falso.

5:

a) Verdadero.

ieeditorial

ieeditorial

GESTIÓN DE PROYECTOS DE INGENIERÍA DEL SOFTWARE.

- Introducción
- Objetivos
- Mapa Conceptual
- Introducción a la gestión de proyectos informáticos.
 - Conceptos: Objetivos. Temporalidad. Presupuesto. Ciclo de vida de un producto informático. Especificaciones del Proyecto. El Director del Proyecto. Tipos de Proyectos. Organización de un proyecto.
 - Etapas de un proyecto: Iniciación, Planificación, Ejecución, Control, Cierre.
- Definición del problema y estrategias de solución.
 - Objetivos a alcanzar.
 - Especificaciones del producto.
 - Los requerimientos de los interesados.
 - Búsqueda de una estrategia de solución y su desarrollo.
- Planificación del tiempo.

- Representación gráfica de la ordenación de tareas.
- De los gráficos de barras al análisis de red.
- PERT (Performance Evaluation and Review Technique).
- Camino crítico.
- Tiempos.
- Hitos, documentos y revisiones.
 - Ordenar las etapas.
 - Relación de tareas.
 - Diagrama de Gantt.
 - Los hitos y sus fechas límite.
 - La documentación técnica como herramienta de seguimiento de la planificación.
 - Conflictos entre actividades.
- Factores de productividad.
 - Métricas de productividad del software.
 - Herramientas que mejoran la productividad.
 - Disponibilidad de los recursos.
 - La experiencia y el entrenamiento del equipo de desarrollo.
 - COCOMO (Constructive Cost Model).
- Recuerda
- Autoevaluación

Introducción

Hoy en día la **gestión de proyectos** se está generalizando cada vez más en las actividades de las organizaciones; sin embargo, muchas compañías, ya sean grandes o pequeñas, tienen dificultades frente a cómo deben gestionar sus proyectos. La ausencia de buenas métricas para gestionar todo el ciclo del proyecto impide avances, optimización de tiempo, recursos, veracidad y conformidad que el cliente adquiere frente a lo que se le está ofreciendo, entre otros aspectos decisivos para el beneficio de la empresa.

Como respuesta a estos múltiples impactos, gran parte de las empresas y organizaciones confían sus sistemas de información a los elementos que integran el desarrollo del software. Básicamente, la finalidad que persigue la gestión de proyectos es hacer cuestionamientos de estimaciones frente a lo que sucederá, por ejemplo, con nuevas soluciones, analizar lo que pasó con un proyecto previo y, posteriormente, tratar de dar respuestas cuantitativas a preguntas claves como: ¿Cuál será el plazo de entrega para cada proyecto?, ¿Cuánto costará cada proyecto? y ¿Qué recurso humano se requiere?

Objetivos

- Gestionar el inicio y la evolución de un proyecto.
- Controlar y responder ante problemas que surjan durante un proyecto.
- Facilitar la finalización y aprobación del proyecto.

Mapa Conceptual

[[[Elemento Multimedia]]]

ieeditorial

Introducción a la gestión de proyectos informáticos.

La necesidad de la informática en cualquier ámbito de la empresa es, actualmente, innegable. La mayor parte de procesos se pueden automatizar para que sean más productivos y eficaces. A su vez, los adelantos de esta ciencia han posibilitado que estos recursos sean económicamente accesibles y flexibles, y capaces de amoldarse a cualquier requerimiento del cliente.

Esto último ha dado lugar a la proliferación de proyectos informáticos. Las empresas ofrecen a sus clientes productos únicos que responden a las necesidades específicas de cada situación y que se desarrollan gracias a equipos de trabajo, en lugar de procesos industriales mecanizados.

Por lo tanto, un proyecto desarrolla un producto único con un **grupo concreto** de gente (un equipo de trabajo que se forma con esta finalidad). Estas no son sus únicas características; además, un proyecto se lleva a cabo dentro de un plazo y con unos recursos limitados.

La **gestión** de proyectos informáticos, por lo tanto, se debe ocupar de todo ello: conseguir un producto que cumpla unas especificaciones concretas, dirigir a un grupo de gente, gestionar unos recursos limitados y planificar la agenda que lo haga posible, además de contar con los riesgos que implica el desarrollo de cualquier nuevo producto. Tampoco hay que olvidar que se lleva a cabo una estructura de trabajo diferente de la jerarquía tradicional. Estas actividades, comunes a cualquier tipo de proyecto, estarán marcadas por las peculiaridades propias de la naturaleza del producto informático.

Conceptos: Objetivos. Temporalidad. Presupuesto. Ciclo de vida de un producto informático. Especificaciones del Proyecto. El Director del Proyecto. Tipos de Proyectos. Organización de un proyecto.

En primer lugar habría que definir qué entendemos por proyecto.

Un **proyecto** es un esfuerzo temporal, único y progresivo, emprendido para crear un producto o un servicio también único.

1. Temporal. Significa que cada proyecto tiene un comienzo definido y un final definido. El final se alcanza cuando se han cumplido con todos los objetivos del proyecto (completed) o cuando el proyecto sea cancelado (terminated). Temporal no necesariamente significa de corta duración, solo que la duración de un proyecto es limitada. También, que el equipo del proyecto se crea solo con el fin de llevar a cabo el proyecto, luego se disolverá.

2. Producto, servicio o resultado singular. Los proyectos crean productos entregables singulares. Productos entregables son resultados, productos y/o servicios. La singularidad es una característica importante de los productos entregables de un proyecto.

3. Elaboración gradual. Significa desarrollar por pasos e ir aumentando la precisión incrementalmente.

Portafolios, Programas y Proyectos.

La relación entre estos tres conceptos, puede ser una relación de inclusión, ya que el portafolio se refiere al conjunto de programas, subprogramas, proyectos, operaciones... que están unidos a la consecución de objetivos estratégicos empresariales. Este conjunto de programas y proyectos de un portafolio, no tienen por qué tener una dependencia recíproca, simplemente se dirigen hacia los objetivos estratégicos de la organización.



Los programas agrupan proyectos que están, de alguna manera, relacionados entre sí, y que pueden ser ejecutados de manera secuencial o paralela. Los proyectos se dan cuando existen actividades nuevas, incluyendo mejoras nuevas. Tienen un inicio y fin, objetivos específicos, entregables y son únicos.

El término “dirección de proyectos” se convierte a veces en “dirección por proyectos” usándose para describir un enfoque directivo de la organización tanto respecto a la gestión de los proyectos como de algunas operaciones, que pueden ser redefinidas como proyectos. Ha habido una tendencia en los últimos años a gestionar la empresa utilizando masiva y sistemáticamente la dirección de proyectos. Esto no quiere decir que todas las operaciones de la organización puedan o deban organizarse en proyectos, aunque los procesos corporativos de investigación, desarrollo, innovación, mejora y entrega singular suelen gestionarse por proyectos. La adopción de “dirección por proyectos” también está relacionada con la adopción de una cultura de la organización próxima a la cultura de dirección de proyectos. También hay que mencionar que todas las organizaciones suelen formular, desplegar e implementar su estrategia utilizando masivamente la “dirección por proyectos”. Este es un asunto muy importante para la profesión, puesto que, efectivamente, la “dirección por proyectos” se suele aplicar a todas las iniciativas de desarrollo de mercado, penetración de mercado, desarrollo de producto y diversificación.

Cualquier tipo de producto o servicio puede ser supervisado en su implementación por el Gerente de Proyectos.

Ciclo de vida del proyecto y organización

A la hora de acercarnos a la identidad de un proyecto, necesariamente debemos detenernos en algunas de sus características, que desarrollaremos posteriormente: carácter complejo, dinámico y procesual que tiende a crear algo nuevo. La gestión de proyectos en una empresa es algo complejo ya que hemos de asumir el reto de ir gestando progresivamente una realidad nueva, inexistente hasta el momento, siguiendo un plan diseñado en etapas hasta alcanzar un resultado satisfactorio.

Complejidad y dinamismo se dan la mano a lo largo de un proceso, no carente de dificultades, que reclamará reajustes, revisiones, control de riesgos, hasta alcanzar los objetivos propuestos. Pero sería ilusorio pretender alcanzar el final del proyecto sin poner las **bases**, crear un buen clima de trabajo y establecer estructuras de comunicación. Cada una de las fases que desarrollaremos posteriormente es importante, pues unas ponen las bases de las otras.

La experiencia acumulada de los **directores** y **gestores de proyectos** son elocuentes en la medida que ayudan a clarificar los incidentes críticos que suelen amenazar el desarrollo de un proyecto. Si el proyecto no tiene un buen comienzo, si la fase de inicio no se cuida, se corre el peligro de generar una situación en la que el equipo de proyecto tendrá una perspectiva diferente acerca de los objetivos del mismo y su desarrollo estará marcado por la discrepancia.

La fase de inicio debe procurar asegurar que todos los miembros del equipo partan de un mismo lugar de forma que, cuando aparezca el conflicto, las dudas, las concepciones contrapuestas se puedan releer a la luz de los objetivos prioritarios. Ésta es una de las formas, tal y como desarrollaremos posteriormente, que posibilitan la identificación con el proyecto.

Si un proyecto no parte de una buena planificación es difícil asegurar el ritmo de trabajo, conocer la progresión, los departamentos y personas implicadas en cada momento, las repercusiones que pueden tener la alteración o cambio de lo proyectado. El valor de la planificación radica en la anticipación, en partir con ventaja. Si se planifica el futuro inmediato y se priorizan las tareas, la evolución del proyecto es más fluida, clara y manejable.

Pero, aun así, el **proyecto** no sigue el rumbo por sí mismo, necesita una guía, una dirección, un constante esfuerzo por aunar energías que lo impulsen hacia la meta. Si decae la motivación, se pierden de vista los objetivos y se deteriora la comunicación, es muy posible que el proyecto conduzca a un rotundo y estrepitoso fracaso.

Sin embargo, un **control sistemático** y regular de los **procesos**, los equipos de trabajo, las normas de calidad y los márgenes de tiempo y costes, permite evitar el hundimiento del proyecto.

Finalmente el producto o servicio por el cual se desplegó el proyecto toma forma. No es en este momento cuando el trabajo termina. Es cierto que el fin era la creación o desarrollo de un objeto tangible, pero todo resultado debe llegar al usuario o cliente que los solicita en las condiciones de calidad pactadas. Al delegar responsabilidades en este punto del proceso o frenar la entrega del producto deteniéndonos en continuos detalles y retoques, no hacemos más que prolongar el esfuerzo o dar por terminado algo inconcluso.

Para facilitar la gestión, los directores de proyectos o la organización pueden dividir los proyectos en fases. El conjunto de estas fases es conocido como ciclo de vida del proyecto. El ciclo de vida de un proyecto suele alterar en cada área de aplicación. Muchas organizaciones determinan ciclos de vida para cada tipo de proyecto.

La definición del ciclo de vida del proyecto puede ayudar al director del proyecto a determinar si deberá tratar cada fase del proyecto como un proyecto separado u independiente. Cuando el resultado del proyecto no esté claramente identificado, lo mejor es tratar cada fase como un proyecto por separado. Como puede ser, recomendar un proyecto que no incluya el estudio del producto y su implementación, que se separe en varios proyectos independientes. Las diferentes fases del ciclo de vida de un proyecto no son lo mismo que los **Grupos de Procesos PM**, que se determinan en todas y cada una de las fases del proyecto.

Efectivamente, el ciclo de vida del proyecto comprende todos los trabajos a realizar, directivos y productivos, y establece cuando o en qué fase se harán, mientras que los grupos de procedimientos solo a los trabajos directivos: **47 procesos** que se repiten en cada procedimiento del proyecto.

El ciclo de vida del proyecto generalmente determina:

- Que productos entregables hay que realizar en cada fase.
- Cómo se revisa, verifica y validan los productos entregables.
- Quién está involucrado en cada fase.
- Cómo se aprueba el cierre de una fase y el comienzo de la siguiente.

Los programas agrupan proyectos que están, de alguna manera, relacionados entre sí, y que pueden ser ejecutados de manera secuencial o paralela.

Verdadero.

Falso

Etapas de un proyecto: Iniciación, Planificación, Ejecución, Control, Cierre.

El proyecto se divide en cinco etapas o grupos de procesos:

- Iniciación.
- Planificación.
- Ejecución.
- Seguimiento y control.
- Cierre.



Iniciación

En la etapa de iniciación, la dirección de la compañía identifica de diferentes maneras un problema o necesidad, lo interpreta o conceptualiza en forma de proyecto, encarga y analiza su viabilidad técnica y económica y los riesgos y, en su caso, lo aprueba.

Habitualmente, en la agenda de la dirección y en el presupuesto de la compañía, un proyecto compite con otros para ser aprobado. Por lo tanto, esta primera fase suele incluir actividades de priorización y selección de proyectos. El producto de esta fase se documenta en formatos propios del proceso presupuestario general de la compañía o del presupuesto del área de organización y sistemas de información. En otras ocasiones, un proyecto es parte de un plan, programa o proyecto mayor, que gestiona una oficina de proyectos.

Planificación

La planificación detallada del trabajo es la etapa o grupo de procesos en la que se establece la hoja de ruta que tendrá que seguir el proyecto para alcanzar sus objetivos y producir los resultados o entregables esperados.

En primer lugar, revisar y, sobre todo, obtener un acuerdo o contrato explícito acerca de los temas del proyecto. El resultado principal es un documento detallado de alcance (scope definition, o documento de alcance), es decir, la definición, como no nos cansaremos de decir, de lo que se hará y lo que no se hará.

Ejecución

La planificación es tan importante que la fase de ejecución habitualmente contiene un ejercicio permanente de preparación de planes más detallados, revisión de los planes elaborados y comprobación de su estado de avance, replanificación de trabajos, etc.

La gestión y documentación rigurosa de los cambios es otro aspecto central de esta fase. Además de estos trabajos de seguimiento y reporte, la ejecución es un ejercicio de gestión y de manejo de personas e incidentes, que, en proyectos de cierto tamaño, justifican de sobras la dedicación de recursos experimentados sólo para controlar y manejar su ejecución. La ejecución incluye también los procesos de aseguramiento de la calidad, gestión de recursos humanos y técnicos, gestión de la comunicación y administración de las compras y contratos.

Seguimiento y control

Como hemos dicho, los procesos de seguimiento y control puede decirse que son permanentes y paralelos a todo el proyecto, especialmente "pesados" en la fase de ejecución. Todos los aspectos contenidos en los diferentes planes deben ser perseguidos, evaluados y, en su caso, reajustados.

Los procesos más críticos en esta fase son los de control de cambios (cualquier petición o incidencia que afecta a la planificación inicial) y los de gestión de riesgos.

Cierre

La etapa de cierre incluye todas las actividades necesarias para finalizar la

gestión del proyecto y completar las obligaciones contenidas en el contrato.

Normalmente incluye la aceptación de los productos por parte del cliente, hacer las revisiones acordadas posteriores al cierre, cerrar los contratos con el cliente y proveedores, documentar las lecciones aprendidas, etc.

El proyecto se divide en cinco etapas o grupos de procesos:

Iniciación.

Planificación.

Ejecución.

Seguimiento y control.

Cierre.

ieeditorial

Definición del problema y estrategias de solución.

El **propósito** de este proceso es elaborar y proponer una solución efectiva para conseguir el resultado, producto y/o servicio del proyecto, que incluya una descripción suficientemente detallada de los principales trabajos, productivos y directivos, que es necesario realizar para implementarla. La elaboración de un Enunciado del Alcance del proyecto suficientemente detallado es crítico para el éxito del proyecto. Incluye tanto una descripción del resultado, producto y/o servicio del proyecto, como de los trabajos directivos y productivos necesarios para conseguirlos efectivamente.

Define y justifica una solución concreta y viable describiendo su **alcance** con suficiente detalle y especificidad, dado que ya se tiene información acerca de los requisitos negociados con los actores interesados en el proyecto. Se reconsideran una y otra vez las necesidades, deseos y expectativas de los interesados, traducidas a dichos requisitos. Se verifican riesgos, suposiciones y restricciones, con el objeto de comprobar que se han considerado todas y, caso de ser necesario, se agregan riesgos, suposiciones y restricciones adicionales.

El equipo de **dirección del proyecto**, colaborando con los actores interesados, se encarga de preparar y realizar todos estos trabajos.

Objetivos a alcanzar.

Las personas que se encarguen de dirigir el **proyecto** precisan algo más que el simple esbozo de una descripción de los objetivos de los que serán responsables. La **ambigüedad en los objetivos** puede conducir a malos entendidos, desilusiones y caras repeticiones del trabajo.

“Consideremos este ejemplo de un objetivo no muy bien definido: “Desarrollar una página web capaz de proporcionar información rápida, exacta y coste efectiva sobre nuestros productos y satisfacer a nuestros clientes”. Esta podría ser la forma en que un patrocinador describa el objetivo del proyecto en la carta. ¿Pero qué quiere decir exactamente? ¿Qué es “rápido”? ¿Cómo se define la exactitud? ¿Un error en mil transacciones es aceptable, o las

expectativas del patrocinador son de un error en diez mil operaciones? ¿Hasta dónde debe llegar la efectividad de costes del sitio web? Cada una de estas preguntas debería contestarse consultando con el patrocinador y las principales partes interesadas”.

Cuanto mejor se expresen los fines, mayor **posibilidad de éxito**.

Los objetivos deben ser decididos por la persona que dirija el proyecto, por el líder del equipo que vaya a trabajar en él y por los miembros de dicho equipo, pues así no se minará la moral ni se cuestionará la capacidad del equipo.

Y es que los equipos trabajan mejor cuando los objetivos son claros y están perfectamente delimitados. De todas formas, cuando los fines están especificados, pero los medios no, los miembros del equipo pueden basarse en todo su complemento de conocimientos, habilidades y experiencias para pensar y ejecutar una forma de operar que esté bien emparejada con el propósito y las circunstancias del equipo.

Las personas que se encarguen de dirigir el proyecto precisan algo más que el simple esbozo de una descripción de los objetivos de los que serán responsables. La ambigüedad en los objetivos puede conducir a malos entendidos, desilusiones y caras repeticiones del trabajo.

Verdadero.

Falso.

Especificaciones del producto.

Los **controles de calidad** son otra forma de mantener el rumbo de un proyecto. Tienen una función importante en cada proyecto, igual que en cualquier proceso normal de trabajo. El director del proyecto examina una unidad de trabajo en un momento apropiado para comprobar que cumple con

las especificaciones. Por ejemplo, si el proyecto es crear un nuevo sitio de comercio electrónico, el control puede ser poner a prueba componentes del sistema de software a medida que se desarrollan para comprobar que funcionan según el plan.

El director del proyecto no debe esperar hasta el final para hacer las comprobaciones porque para entonces, arreglar cualquier aspecto superando lo planificado costaría mucho tiempo y dinero.

Los controles periódicos indican las condiciones que se escapan de las especificaciones, por lo que el equipo encargado del proyecto puede buscar la causa y reconducir los procesos que causaron los problemas. Esa clase de supervisión y acción correctiva garantiza que los resultados de las tareas subsiguientes cumplan con los niveles de calidad establecidos.

En esta sección se describen las pruebas que se realizarán para aumentar la probabilidad de conseguir un resultado que cumpla las expectativas. Puede dividirse en **dos partes**:

Pruebas de calidad del producto

Incluye las pruebas especiales que haya que realizar sobre productos concretos. Por ejemplo, el documento de diseño de la solución puede necesitar una revisión formal por parte del comité correspondiente para asegurarse de que está conforme con los objetivos estratégicos de la organización. El formulario de las pruebas especiales debe ser descrito también con mayor detalle. Así, se puede esquematizar el proceso oficial de revisión de la calidad, para que los altos cargos gestores conozcan su alcance y valor, antes de comprometer el dinero necesario para financiarlo.

Pruebas de calidad del proyecto

Se trata de las pruebas que se realizarán en las etapas de grandes proyectos o de proyectos que implican muchos productos.

Los requerimientos de los interesados.

Se trata del proceso encargado de determinar y documentar las necesidades y expectativas de los actores interesados en el proyecto con el fin de definir sus requisitos válidos y poder determinar los objetivos del proyecto. Dicha descripción debe ser lo suficientemente especificada como para poder asegurar una exacta comprensión de las necesidades y expectativas del patrocinador, del cliente y demás actores que se han interesado en el proyecto. El éxito del proyecto varía en gran manera de hacer este proceso con mucho cuidado y con mucha precisión.

Se parte del **Acta de Constitución del proyecto y del Registro de Interesados**. Muchas organizaciones clasifican los requisitos en requisitos de producto (requisitos técnicos, de seguridad, de rendimiento, etc.) y requisitos de proyecto (de negocio, de entrega, de realización de trabajos, etc.).

Búsqueda de una estrategia de solución y su desarrollo.

Convertir un grupo de personas en un buen **equipo** no es algo que se pueda lograr de un día para otro. Las personas llegan al grupo con sus agendas e intereses personales. Muchas ven en sus compañeros de equipo a competidores que les disputan ascensos, reconocimientos y recompensas. También hay quien alimenta inquinas personales contra una o más de las personas con las que se le ha reunido, y nunca falta tampoco la persona que no tiene las habilidades sociales para trabajar en **grupo**.

Las agendas personales, las competencias internas, las inquinas y las habilidades sociales escasas son elementos que hasta cierto punto están presentes en todos los equipos. Si no se controlan o neutralizan pueden socavar la efectividad del proyecto.

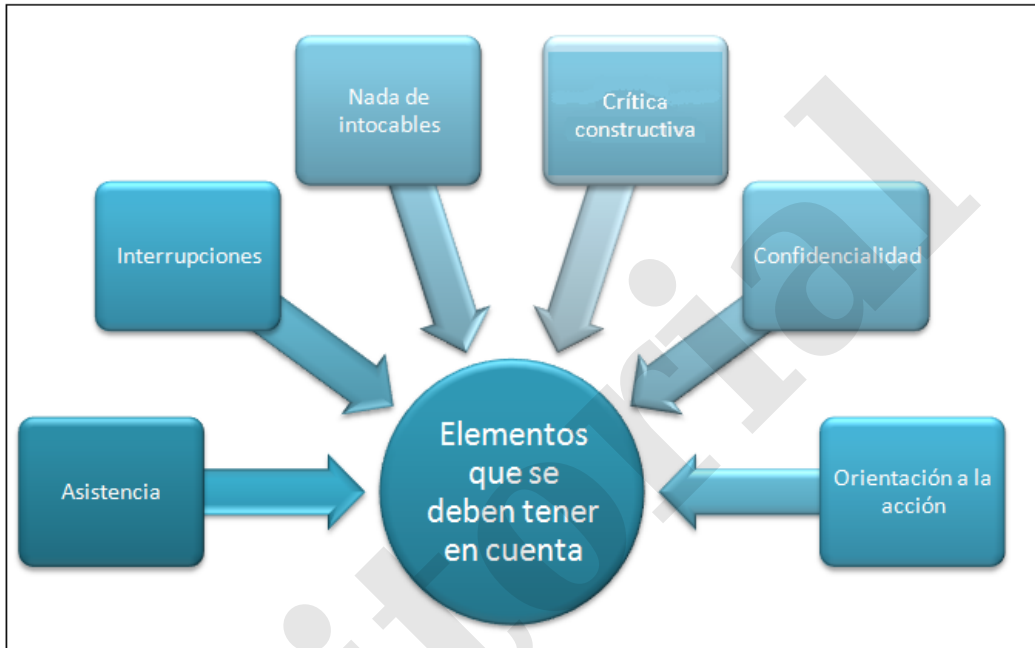
La diversidad de **especialidades** y estilos de trabajo, que con tanto trabajo usted ha conseguido traer al equipo, también pueden dificultar la colaboración. Los especialistas técnicos, por ejemplo, generalmente hablan una jerga conocida sólo para ellos y eso puede separarlos de sus compañeros de equipo. Y las personas, habitualmente no confían en aquello que

desconocen.

Una de las mejores maneras de solventar estos problemas es establecer claras **normas de comportamiento** que se apliquen a todos por igual. Las reglas más importantes están relacionadas con:

- **Asistencia:** los miembros y líderes deben comprender que el equipo no puede tomar decisiones y hacer su trabajo, si sus miembros no asisten a las reuniones o las sesiones de trabajo conjunto. Si usted es el líder, los demás seguirán su ejemplo. Si siempre llega tarde o simplemente no aparece, los miembros del equipo seguirán su ejemplo.
- **Interrupciones:** los teléfonos móviles se deben apagar durante reuniones y sesiones de trabajo. Atender una llamada durante una reunión del equipo indica que ésta es más importante que lo que está haciendo el grupo. También hay que dejar claro que no se debe interrumpir a nadie durante las reuniones. Todo el mundo tiene derecho a hablar.
- **Nada de intocables:** hay que acordar que ningún tema quedará excluido. Por ejemplo, si un equipo encargado de un proceso de reestructuración sabe que un cambio molestará a un determinado directivo, sus miembros no deben tener miedo ni ser reacios a discutirlo. Evitar la cuestión debido a las posibles objeciones del directivo indicará que el equipo y sus esfuerzos no tienen sentido.
- **Crítica constructiva:** la solución de problemas tiende a producir soluciones que compiten entre sí. Los responsables de estas soluciones deben comprender que tienen el poder de representar sus opiniones, pero no el derecho de socavar la de otros mediante engaños o reteniendo datos relevantes. Los miembros de un equipo también deben aprender a expresar su desacuerdo de forma constructiva.
- **Confidencialidad:** algunas de las cuestiones a tratar pueden ser confidenciales. Los miembros del equipo no las discutirán con toda libertad si no tienen la seguridad de que lo que se dice dentro del equipo no saldrá de ahí.
- **Orientación a la acción:** los equipos no se forman para reunirse y discutir; los que sólo hacen eso no logran nada. El verdadero propósito es actuar y producir resultados. Eso es algo que debe quedar claro desde el principio. Un equipo orientado a la acción significa que todo el

mundo hace trabajo de verdad y todos los miembros tienen una tarea asignada y la cumplen.



¿Cuáles deberían ser las normas de comportamiento de su grupo? Pues depende del propósito del mismo y de la personalidad de sus integrantes. En todo caso, las normas deben ser claras y concisas. Deben incluir los aspectos básicos: respeto para todos los miembros del grupo, el compromiso de escuchar activamente y la comprensión de cómo expresar preocupaciones y manejar conflictos.

Para garantizar el **libre flujo de ideas**, algunos grupos van más allá, por ejemplo, manifestando de forma explícita que todos los miembros tienen el derecho de no estar de acuerdo con los demás. También pueden querer adoptar unas guías específicas para:

- Apoyar la asunción calculada de riesgos.
- Establecer procedimientos de reconocimiento y manejo del fracaso.
- Fomentar la expresión individual.
- Estimular una actitud distendida.

Sea como sea, todos los miembros del grupo deben haber participado en la

elaboración de las normas que adopte el grupo y todos deben suscribirlas. Esto evitará muchos problemas en el futuro.

Además, debemos ser conscientes de que las normas, en ocasiones, surgen de una manera inesperada, aunque las condiciones se hayan discutido y especificado al comienzo del proyecto, e incluso se hayan reflejado en el **Documento Inicial del Proyecto**.

Por ejemplo, si algunos egos se resienten durante las primeras reuniones del equipo, en las subsiguientes interacciones se podrá observar una sutil hostilidad entre los individuos afectados.

Esa hostilidad, probablemente se manifestará mediante la búsqueda de intenciones ocultas, críticas, comentarios sarcásticos u otros comportamientos contraproducentes. Las personas que se encarguen de liderar el grupo deberán atajar este tipo de conductas e incidir en la idea de respeto mutuo y de máxima utilidad de las ideas. Hay ciertas líneas que nunca se deben sobrepasar.

Las agendas personales, las competencias internas, las inquinas y las habilidades sociales escasas son elementos que hasta cierto punto están presentes en todos los equipos. Si no se controlan o neutralizan pueden socavar la efectividad del proyecto.

Verdadero.

Falso.

Planificación del tiempo.

OBJETIVOS

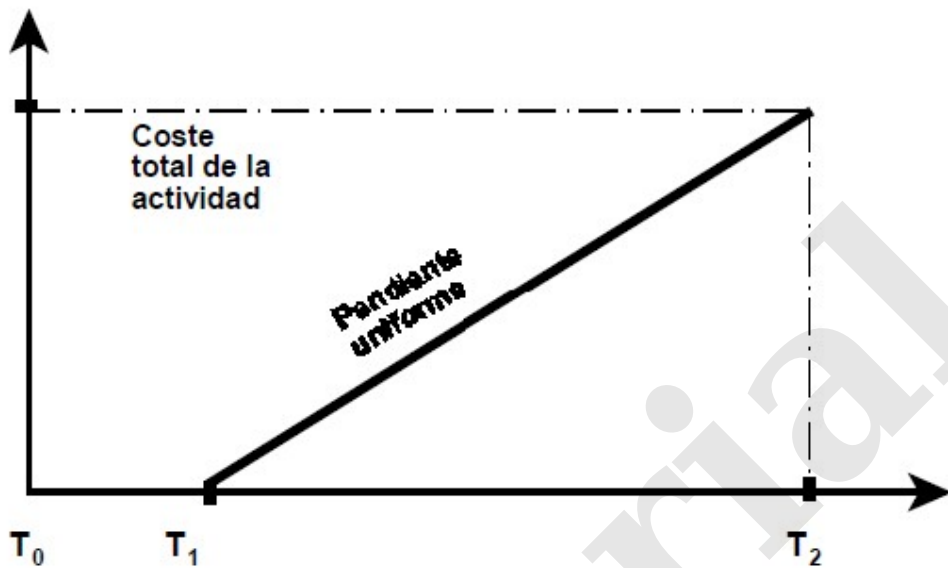
- Comprender las implicaciones de la medición del avance y su relevancia en la gestión de tiempos.
- Conocer los usos de la “Curva S” del proyecto, como una herramienta de análisis de tiempos clave.
- Desarrollar la relación entre la presupuestación de costes y de tiempos.
- Analizar los proyectos según su transcurso en el tiempo, incidiendo en la estructuración de sus costes y su programación.
- Explicar las causas de alteración del avance en el tiempo de un proyecto, así como analizar las posibles medidas para contrarrestarlas.
- Conocer y profundizar en los parámetros para medir la actividad de un proyecto.

Representación gráfica de la ordenación de tareas.

Como sabemos un proyecto puede descomponerse en un conjunto de actividades que se desarrollan en el tiempo. Éstas tienen asociado un valor de coste de cuya suma se desprende el presupuesto.

Los costes de las **actividades** tienen un determinado componente temporal, dado que ellas se desarrollan durante una serie de intervalos de tiempo, o lo que es lo mismo, una determinada relación entre el avance de la actividad y coste incurrido.

Si los costes asociados a una actividad fuesen lineales o uniformes en el tiempo, tendríamos el siguiente gráfico de coste con relación a su avance el cual tendría una pendiente uniforme:



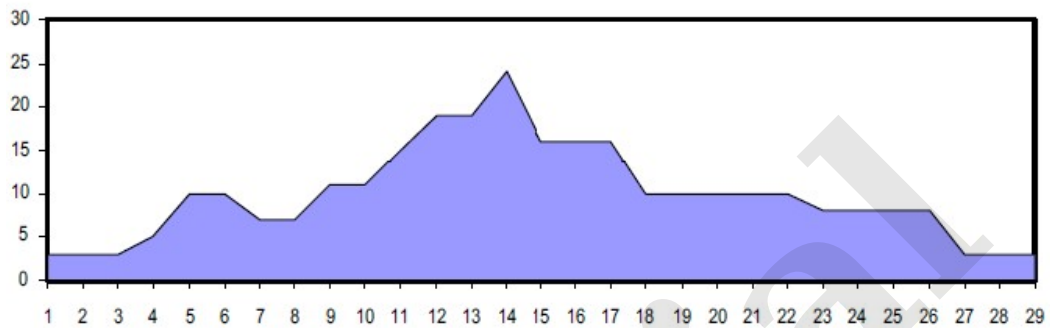
Pendiente uniforme de costes acumulados para una actividad

Para un proyecto de varias actividades, que podemos representar en un diagrama de Gantt, los costes estarán prorrateados en el intervalo de tiempo correspondiente.

Actividad	Duración	Programación																														
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
1	6	3	3	3	3	3	3																									
2	7				2	2	2	2	2	2	2	2																				
3	18				5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5								
4	6									4	4	4	4	4	4																	
5	7											6	6	6	6	6	6	6														
6	3											4	4	4																		
7	12													5	5	5	5	5	5	5	5	5	5	5	5	5	5	5				
8	7																							3	3	3	3	3	3	3	3	
SUMA		3	3	3	5	10	10	7	7	11	11	15	19	19	24	21	21	21	10	10	10	10	10	10	8	8	8	8	3	3	3	

A partir de la **programación** podemos construir las curvas de coste del proyecto: costes por período, es decir la suma total de los costes de actividad para cada período durante el proyecto, y la curva de costes totales o

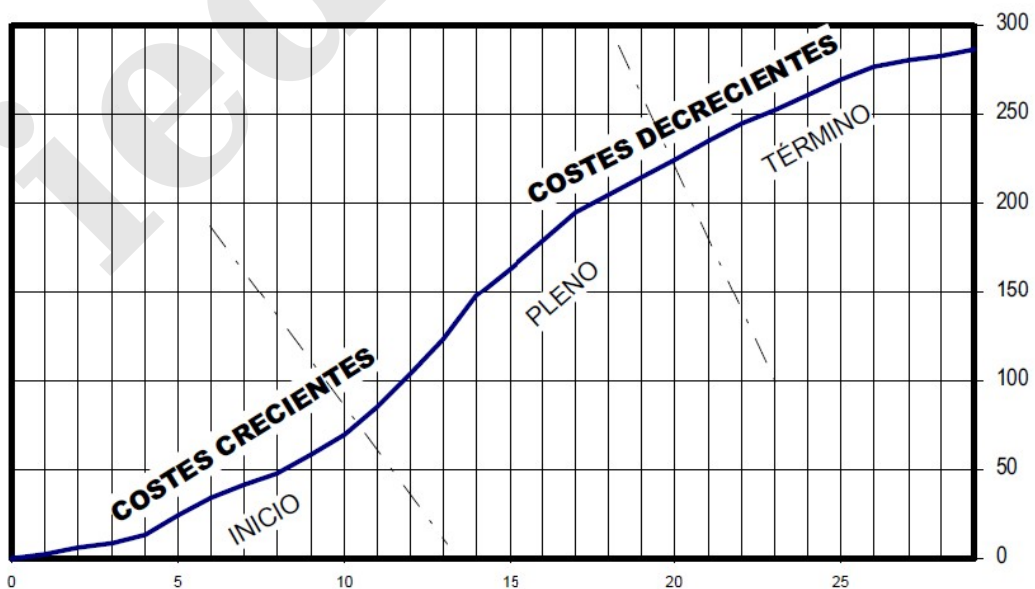
acumulados.



Costes por periodo

Como se puede observar en el ejemplo, los costes por período del proyecto se inician con una tendencia al aumento hasta llegar a un máximo y posteriormente decrecer hacia el término.

El gráfico que resulta dependerá de las características propias del proyecto, en cuanto a su estructura de costes y la programación; sin embargo, generalmente tiende a estas formas, con mayor o menor asimetría y suavidad.



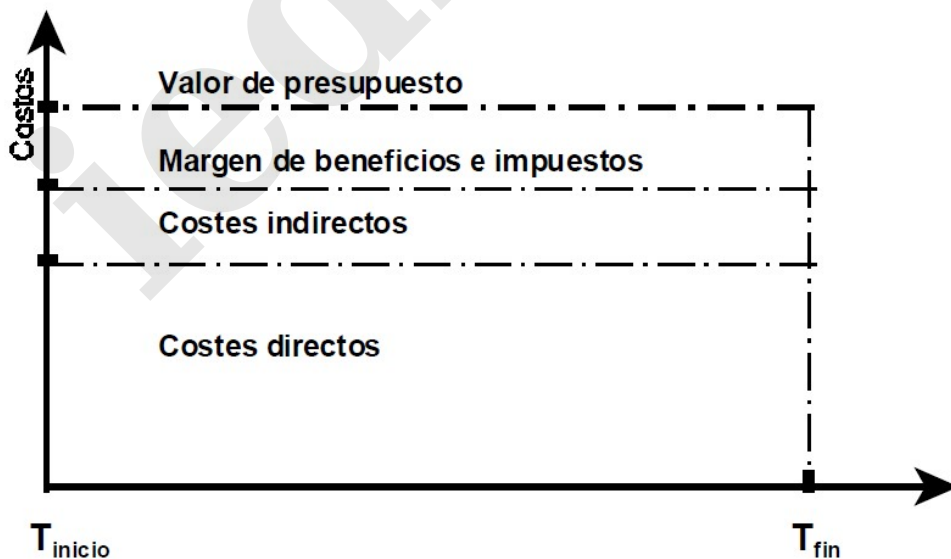
Costes acumulados en la Curva “S” y períodos característicos

La curva S representa entonces lo que podemos denominar un “valor de avance” del proyecto, de modo que al menos en términos de coste, el avance real del proyecto lo podemos realizar comparando los valores presupuestados versus los costes incurridos y ejecutados.

Antes de continuar con el análisis de la curva S, conviene revisar algunas consideraciones de esta relación: avance del proyecto versus costes acumulados y presupuestos.

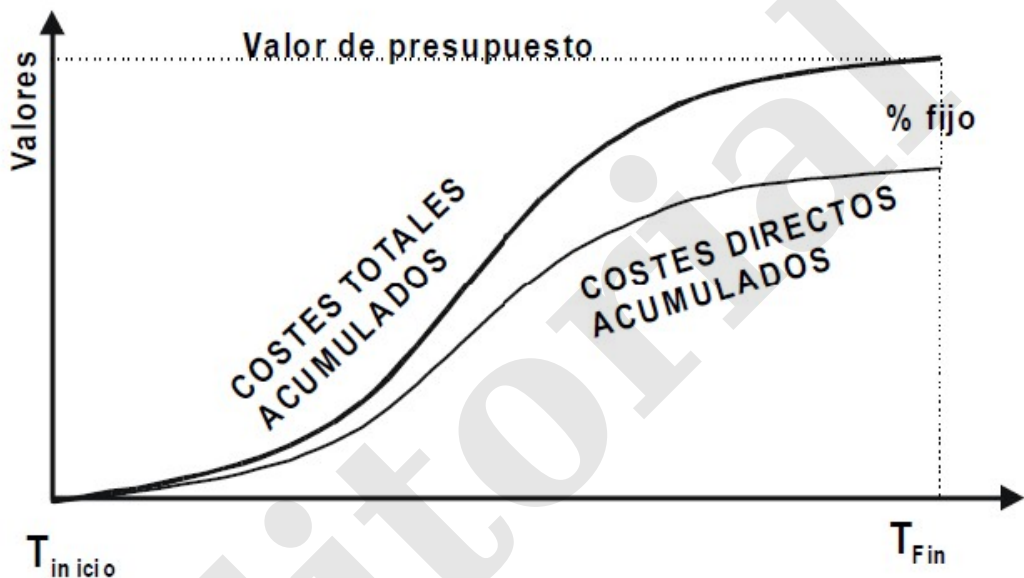
Hay varias razones, algunas de carácter práctico, por las cuales es conveniente desarrollar la relación del avance con los costes directos y no con los costes totales:

- Los costes directos son los relacionados directamente con la actividad, por lo cual la información para los posteriores análisis comparativos es más sencilla de recabar.
- Los costes indirectos pueden seguir generándose a pesar de que no existan avances reales en el proyecto,
- En los costes totales se encuentran algunos valores iniciales de inversión que pueden llegar a distorsionar las curvas y su posterior análisis.



Costes y presupuestos

Generalmente para los efectos de presupuesto, los costes indirectos y otros valores agregados, se prorratean como si de un porcentaje sobre los costes directos.



Proporcionalidad de costes directos y totales

Para generar las curvas S de valores acumulados, debemos de optar por utilizar una escala real o una escala unitaria. La real nos da la oportunidad de comparar los presupuestos con los costes en cifras determinadas. La escala unitaria da la posibilidad de comparar curvas totales proyectadas con curvas de costes directos reales, ambas determinadas en porcentaje.

Esta última escala tiene la **ventaja** de que permite cierta comparación de formas y comportamientos con otros proyectos.

Por ello se optará, en general, por la escala unitaria y agregaremos algunas anotaciones que reflejen valores concretos en aquellos períodos que nos interese analizar.

Las **curvas** de avance en base a costes directos, y su contrapartida, las certificaciones, se pueden construir mediante uno de los dos conceptos

siguientes: por valores devengados, es decir por aquellos realizados y comprometidos pero no íntegramente percibidos y cancelados; o por valores efectivos, es decir por ingresos efectuados. El primer método, de devengado, es el recomendable para los análisis de avance; en tanto el segundo para los aspectos financieros del proyecto.

La curva S representa entonces lo que podemos denominar un “valor de avance” del proyecto, de modo que al menos en términos de coste, el avance real del proyecto lo podemos realizar comparando los valores presupuestados versus los costes incurridos y ejecutados.

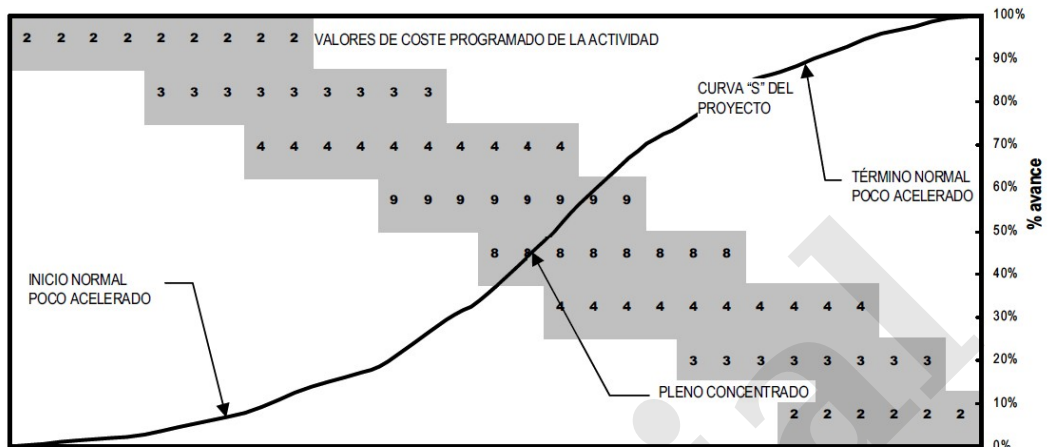
Verdadero.

Falso.

De los gráficos de barras al análisis de red.

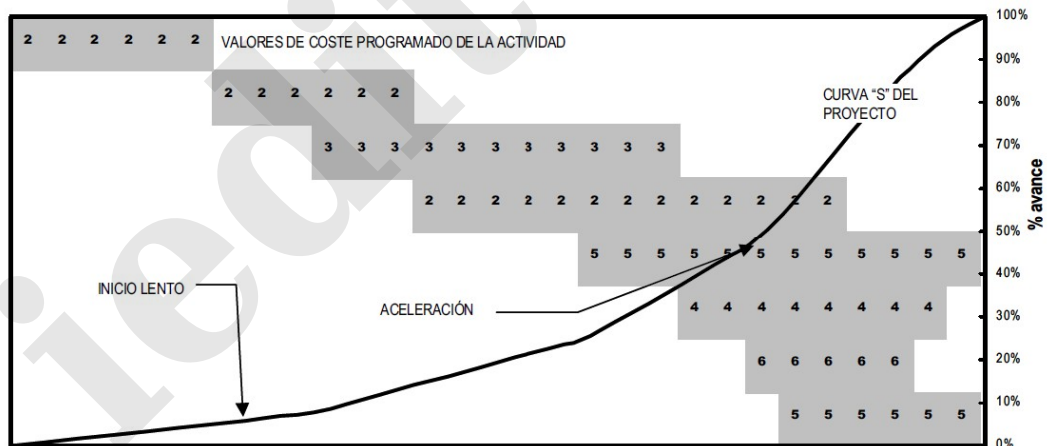
Las **curvas S** tendrán diferentes formas según sea la estructura de costes de las actividades y de la programación que se tome.

Un proyecto normal, con costes equilibrados entre el inicio y el término, concentrará actividades durante el período pleno y tendrá su curva S una forma como en la figura siguiente.



Curva "S" de proyecto normal

En el caso de proyectos retardados, las actividades se basan hacia el tercio final de la duración total, lo que da resultado la curva S que continua.



Curva "S" de proyecto tardío

Los proyectos tardíos son propios de aquellas situaciones en la que los costes de las actividades iniciales del proyecto son bajos, lo que permite extenderlos en el tiempo y minimizar los riesgos de alteraciones importantes de costes hacia los períodos finales. Son propias de proyectos de fuertes riesgos y que a

la vez no requieren acelerar su conclusión para la incorporación temprana del producto al mercado. También se puede aplicar a un proyecto para asegurar algunas condiciones de financiamiento.

Los **proyectos tempranos** son propios de aquellos casos en que las actividades principales se concentran al inicio, quedando para las etapas finales aquellas menores de ajustes diversos.

Un ejemplo de estos casos es de aquellos proyectos que se desarrollan en ingeniería concurrente, en los cuales al comienzo se incorpora el núcleo principal del proceso proyectual, restando hacia el término la realización de comprobaciones y revisiones menores.

Proyectos de estas características implican un mayor **riesgo**, pero permiten incorporar al mercado productos de forma temprana, ya sea en forma de producto piloto o versiones de prueba y estudiar su comportamiento para realizar los ajustes necesarios para las versiones finales. Al mismo tiempo, al conocer en períodos tempranos su configuración (en el caso de productos), permitirá iniciar las campañas de marketing e introducción anticipada del concepto en el mercado, para ganar a productos de la competencia o para conocer si lo que se está proyectando cumplirá con las expectativas del consumidor o el cliente final.

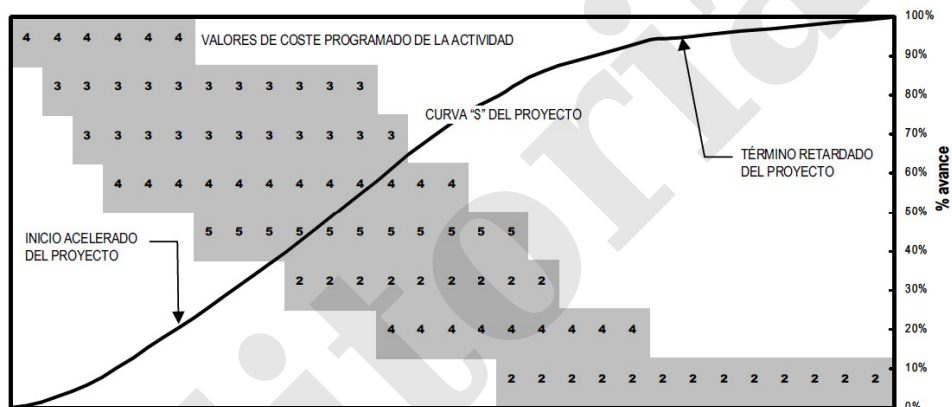
PERT (Performance Evaluation and Review Technique).

En su traducción al español, **PERT** quiere decir “técnica de revisión y evaluación de resultados”. En los gráficos **PERT**, cada tarea está representada por un nódulo que se encuentra a su vez, conectado con otros nódulos o tareas necesarias para llevar a cabo el proyecto.

En función a esto, un gráfico PERT establecerá multitud y diferentes redes paralelas e interconectadas de tareas, fomentando con ello las revisiones periódicas de ciertos proyectos complejos. A este respecto, cabe mencionar que su diferencia con los gráficos Gantt se da en el momento en que los primeros recogen todas las relaciones importantes entre tareas, además de los hitos del proyecto.

Todas estas representaciones se realizan a través de **grafos**, esto es “las representaciones gráficas en las que los diferentes puntos de un conjunto se relacionan entre sí mediante enlaces que, generalmente, están orientados en una dirección”. Dichas uniones, se realizarán a través de elementos tales como los vértices o arcos, siendo los primeros cada uno de los puntos representados, y los arcos, cada enlace que une un vértice (origen) con otro (destino).

Un ejemplo de dicha representación sería la siguiente:



La primera etapa de aplicación del método PERT es el llamado PERT-Tiempos, que permite determinar el tiempo total de ejecución del proyecto, desde los tiempos de las actividades que lo forman.

Posteriormente, añadiendo la información de recursos, y en función del ritmo al que se quiere desarrollar el proyecto, es posible determinar el coste directo del mismo, mediante la extensión denominada **PERT-Costes**.

El método PERT está basado en la teoría de grafos para desarrollar la programación del proyecto, con el fin de optimizar (minimizar) el tiempo de ejecución.

Un grafo es un diagrama compuesto de nodos (lugares, estados, sucesos, actividades, etc.) y de arcos (relaciones, caminos, períodos de tiempo, etc.)

Para ello estudia las relaciones (dependencias) y condicionantes (calendarios, disponibilidad de recursos, restricciones externas, etc.) del conjunto de

actividades que integran el proyecto.

Los principales resultados que se obtienen tras la aplicación del método PERT son:

- El conjunto de tareas denominado camino crítico (o ruta crítica).
- La fecha de inicio más tardío admisible (tiempo más lejano), para cada actividad.
- La holgura o margen disponible para cada actividad o suceso.

Los sistemas PERT emplean un grafo, denominado red del proyecto, para visualizar gráficamente las interrelaciones entre sus elementos. Sobre éste se realizan las operaciones que permiten calcular la ruta crítica, los tiempos más lejanos y las holguras.

Para aplicar el método **PERT** a un **proyecto**, es necesario seguir las etapas básicas del proceso de planificación, ya presentadas en el apartado 2.3., y que se **resumen** en:

1. Generar la lista de actividades (Por ejemplo, a partir de la EDT).
2. Establecer las relaciones entre las actividades.
3. Construir el grafo de actividades o red del proyecto.
4. Estimar los tiempos de ejecución de cada actividad.
5. Utilizar el método PERT (propiamente dicho).
 - 5.1. Calcular el tiempo más próximo en que podría iniciarse cada actividad.
 - 5.2. Calcular el tiempo mínimo de ejecución del proyecto, o lo que es lo mismo, el tiempo más próximo en que podría concluir la última actividad del proyecto.
 - 5.3. Calcular el tiempo más lejano en que podría concluir cada actividad.
 - 5.4. Calcular la holgura para cada suceso (inicio o fin de actividad).
 - 5.5. Calcular la holgura para cada actividad.

5.6. Identificar el camino crítico.

5.7. Analizar el resultado con el fin de minimizar, si fuera posible, la duración determinada.

6. Asignar recursos a cada actividad.

7. Calcular costes directos para cada actividad.

8. Definir los hitos principales e incluirlos en el plan del proyecto.

Una vez finalizado el proceso de planificación, puede comenzar el de programación, cuyas etapas fundamentales son:

1. Definir el calendario del proyecto (días hábiles, duración de la jornada, etc.).

2. Programar los tiempos (asignar fecha de inicio y fin a cada actividad).

3. Programar los recursos.

4. Calcular los costes de ejecución de cada actividad.

5. Verificación y ajuste de la programación. Para ello se utiliza la extensión del método denominada PERT-Costes.

5.1. Ajuste de los tiempos de tarea, y los tiempos del proyecto.

5.2. Ajuste de la asignación de recursos (resolviendo los conflictos de disponibilidad, sobreasignación y equilibrado).

5.3. Ajuste de los costes de tarea y de proyecto.

Está basado en la teoría de grafos para desarrollar la programación del proyecto, con el fin de optimizar (minimizar) el tiempo de ejecución:

El método PERT.

El diagrama compuesto de NODOS.

Camino crítico.

El **análisis del camino crítico y la gestión del valor ganado**, son exclusivos del área de la dirección de proyectos. Sin embargo, comprender y aplicar los conocimientos, habilidades, herramientas y técnicas generalmente reconocidas como buenas prácticas no es suficiente por sí solo para una dirección de proyectos efectiva. Una dirección de proyectos efectiva requiere que el equipo de dirección del proyecto comprenda y use los conocimientos y las habilidades correspondientes a, por lo menos, cinco áreas de experiencia:

- Fundamentos de la Dirección de Proyectos.
 - Ciclo de vida del proyecto.
 - Cinco Grupos de procesos de Dirección de Proyectos.
 - Diez Áreas de Conocimiento.
- Conocimientos, normas y regulaciones del área de aplicación.
 - Legislación europea.
 - Legislación estatal.
 - Legislación autonómica.
 - Legislación municipal.
 - Normativa técnica.
- **Comprensión del entorno del proyecto.**
 - Entorno Cultural y social.
 - Entorno internacional y político.
 - Entorno físico.

- Macro y micro entorno económico.
- **Conocimientos y habilidades de dirección general.**
 - Planificación.
 - Organización.
 - Selección de personal.
 - Gestión financiera y contabilidad.
 - Compras y adquisiciones.
 - Ventas y comercialización.
 - Contratos y derecho mercantil.
- **Habilidades interpersonales.**
 - Comunicación efectiva.
 - Influencia en la organización.
 - Liderazgo.
 - Motivación.
 - Negociación y gestión de conflictos.
 - Resolución de problemas.

No es preciso que cada miembro del equipo experto en cada una de las áreas; pero sí que es imprescindible que entre todo el equipo conozca todas estas habilidades y el Director del Proyecto ha de tener un conocimiento generalizado suficientemente amplio sobre todas estas habilidades y sobre la Gestión integral de un Proyecto.

El Director de proyecto ha de ser capaz de sacar lo mejor de cada persona y asigna las tareas adecuadas en función de los conocimientos y habilidades de cada uno.



El análisis del camino crítico y la gestión del valor ganado, son exclusivos del área de la dirección de proyectos. Sin embargo, comprender y aplicar los conocimientos, habilidades, herramientas y técnicas generalmente reconocidas como buenas prácticas no es suficiente por sí solo para una dirección de proyectos efectiva.

Verdadero.

Falso.

Tiempos.

Se acaba de ver el diferente **enfoque** determinista/probabilista entre CPM y PERT. Pero no acaba ahí la diferencia existente entre ambos métodos al respecto de la estimación de tiempos, ya que en ella aparece implícita la consideración de la cantidad de recursos que van a utilizarse para la ejecución de una tarea.

El **método PERT** asume que la duración de la tarea se calcula considerando que se dispone de todos los recursos que se necesitan para desarrollarla, al menos con el procedimiento, tecnología y herramientas que la organización. Por tanto, no procede correlacionar el tiempo de ejecución de la tarea con el empleo de un mayor o menor número de recursos (mayor o menor coste).

Sin embargo el CPM atiende una duración de tarea fija, pero para un específico nivel de uso de recursos, determinando una correlación lineal entre el tiempo y el coste de realización de la tarea.

La **justificación** de esta diferencia también puede deberse al origen y enfoque inicial de ambos métodos. Mientras el objetivo fundamental del PERT era optimizar los plazos en un proyecto con un elevado nivel de incertidumbre, el CPM pretendía optimizar simultáneamente plazo y coste del proyecto, por lo que la extensión del método a la programación de recursos resulta imprescindible.

Hitos, documentos y revisiones.

Los **hitos** son **acontecimientos o eventos** que deben ocurrir en un momento determinado del desarrollo del proyecto. En muchos casos reflejan el inicio (o cierre) de una fase o etapa, o la creación de una influencia procedente del exterior del proyecto.

Normalmente, si el hito no sucede en el momento previsto, es posible que se haya un determinado retraso en el proyecto. Un hito debe estar controlado al igual que cualquier otra tarea. Con una diferencia una tarea no tiene duración.

Ordenar las etapas.

En este sentido, es necesario ser coherentes con dicho **enfoque**, y por tanto, gestionar adecuadamente los procesos ya mencionados. Para ello, la gestión de un **proceso** se realizará en función a las siguientes **etapas**:

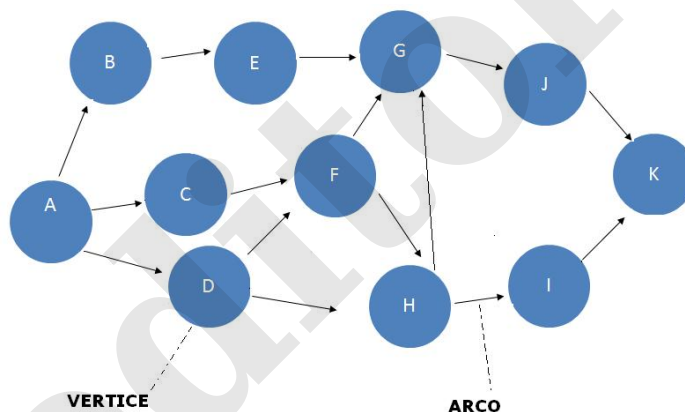
- Fijar los límites del proceso. Definir input y output, los proveedores y usuarios del producto.
- Asignar y comunicar la finalidad del proceso, su razón de ser y existir, y los objetivos de calidad/funcionalidad, tiempo/servicio y coste del proceso. Todos ellos deben ser coherentes con los requisitos del cliente, con las partes interesadas y con la estrategia de la empresa.
- Planificar el proceso. Es conveniente representarlo gráficamente mediante un flujograma. Además de dicho documento, se deberá crear la hoja de proceso, definir el equipo de proceso y el sistema de control.
- Identificar, caracterizar y comprender las interacciones realizadas con el resto de procesos.
- Asegurar la disponibilidad de recursos físicos, materiales e información necesarios para la operación y el control del proceso. Igualmente, se exige una adecuada formalización de la interacción con los procesos de apoyo y de gestión.
- A lo largo de la ejecución y siempre que el responsable no sea el ejecutor directo, el gestor del proceso se inmiscuirá en la resolución de las incidencias, en la erradicación de riesgos y asegurará el funcionamiento de los controles.
- Medición y seguimiento. Se trata de la recogida de datos mediante las

herramientas de medición del proceso y con una frecuencia adecuada.

La metodología del desarrollo de software es una forma sistemática de construir, administrar y gestionar un proyecto para llevarlo a cabo con éxito. Esta sistematización indica cómo un proyecto se divide en otros más pequeños para estandarizar la forma en que se lleva a cabo.

Así, una metodología de desarrollo de software es el proceso seguido sistemáticamente para diseñar, implementar y mantener un producto de software desde el momento en que surge la necesidad hasta que se logra el propósito para el cual fue creado.

En concreto, la gestión del proceso se desarrollaría de la misma forma que se muestra en este gráfico:



En función a este gráfico, aquellos procesos cuya gestión es óptima, presentan ciertas características que los diferencian de los demás:

- Tienen identificados sus proveedores y clientes.
- Tienen una finalidad claramente definida en términos de su contribución al desarrollo de ésta, así como de las políticas de empresa.
- Disponen de objetivos cuantitativos y cualitativos para satisfacer las expectativas de su cliente.
- Cuentan con un responsable del proceso, funcionamiento, resultados y mejora.
- Poseen límites concretos de comienzo y final.

- Tienen asignados los recursos necesarios y, en consecuencia, se da el uso adecuado de las tecnologías de la información.
- Utilizan un sistema de medidas de control de la eficacia, eficiencia y flexibilidad, con la finalidad de centrar la atención del personal y tomar decisiones de mejora.
- Poseen mínimos puntos de control, revisión y espera.
- Operan bajo control estadístico.
- Está normalizado y documentado.
- Muestra con claridad las interacciones con otros procesos.
- Contribuye al desarrollo de ventajas competitivas propias, sostenibles y duraderas.

Relación de tareas.

La **Estructura de Descomposición del Proyecto (EDP)**, o “**Work Breakdown Structure**” (WBS), es una técnica de descomposición funcional de las actividades y tareas del proyecto, plasmada en un listado de tareas organizadas en forma de diagrama jerárquico en árbol.

Su objetivo es reducir sistemáticamente la complejidad del proyecto, de forma que éste quede descompuesto en “fragmentos” constituidos por bloques de tareas (o paquetes de trabajo).

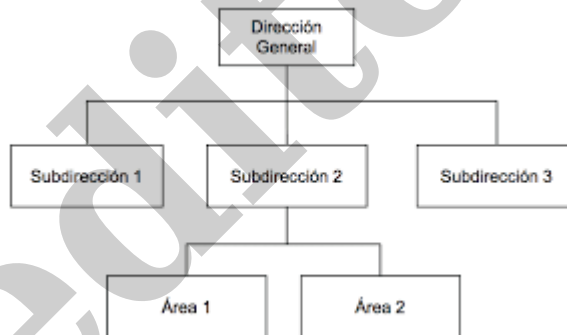
Los bloques de tareas pueden subdividirse en tantos niveles como sea necesario, para alcanzar el nivel de detalle (descomposición) deseado. El nivel de división más pequeño debe ser suficientemente fino para permitir operar a nivel de tarea, aunque sin llegar a generar un volumen de tareas inmanejable.

Es posible estructurar los diferentes niveles del árbol EDP en cinco tipos jerárquicos: meta, objetivos, actividades, subactividades y paquetes de trabajo.

Los pasos para establecer un **diagrama de descomposición del proyecto** son:

1. Establecer el nivel superior del árbol como nivel cero (meta).

2. Dividir el proyecto en sus objetivos-funciones principales de manera que el proyecto quede claramente definido.
3. Subdividir cada objetivo/función en las actividades-sistemas que es necesario desarrollar para alcanzarlo. Para su representación gráfica en el árbol, conviene asignar una frase concisa y descriptiva a cada actividad.
4. En el caso de que las actividades compuestas de dos o más características principales es necesario subdividir las en las subactividades (subsistemas) correspondientes. El conjunto de todos los sistemas y subsistemas debe describir completamente la función/actividad situada en el nivel superior.
5. Proseguir sucesivamente, dividiendo cada elemento en celdas de menor nivel jerárquico, hasta que las subactividades (subsistemas) posean la complejidad necesaria para poder ser traducidas en paquetes de trabajo.
6. Cada elemento (celda) de la EDP es inscrito mediante un código único. Estos identificadores se denominan código de cuentas.



La Estructura de Descomposición del Proyecto (EDP), o “Work Breakdown Structure” (WBS), es una técnica de descomposición funcional de las actividades y tareas del proyecto, plasmada en un listado de tareas organizadas en forma de diagrama jerárquico en árbol.

Verdadero.

Falso.

Diagrama de Gantt.

La **técnica clásica de programación de tiempos** es el denominado diagrama de Gantt (en reconocimiento a Henry L. Gantt, que lo utilizó durante la Primera Guerra Mundial). Se trata de una herramienta muy útil y de sencilla elaboración. Sin embargo presenta el inconveniente de su laboriosidad, ya que cualquier modificación supone reconstruir la práctica totalidad del diagrama.

Actualmente se utilizan aplicaciones informáticas basadas en los métodos de grafos orientados (PERT, CPM, etc.).

Los hitos y sus fechas límite.

Una de las diferencias fundamentales existente entre las distintas técnicas de planificación de proyectos estriba en la consideración de los costes:

- El **método PERT** clásico no los considera. Su extensión denominada PERT/Costes incluye como subproducto una contabilidad de los costes directos asociados al uso de recursos.

Este método presenta un problema importante. Dado que el PERT/Costes proporciona los costes directos asociados a cada tarea, y por agregación, los

del proyecto, requiere que de forma independiente se contemple los costes indirectos (estructura de empresa, gastos generales, costes financieros, etc.). Por otra parte, la empresa que realiza el proyecto debe de llevar su sistema de contabilidad conforme a la reglamentación fiscal. Todo ello provoca el uso de varias contabilidades paralelas (por proyecto, por departamentos, por criterio de ingresos y pagos) que disuaden del uso real del PERT/Costes.

El método CPM

Permite la introducción de una relación entre el coste de ejecución en el tiempo normal previsto y el coste necesario para desarrollar la actividad con ritmo intensivo, suponiendo una relación lineal entre ambos puntos extremos e incorporando algoritmos heurísticos para obtener la mayor reducción de tiempo posible dado un coste máximo admisible.

Método MCE (Minimum Cost Expediting)

Aplica algoritmos de programación lineal mixta entera (dado que aparecen tanto variables continuas como enteras, tales como el número de recursos humanos) para obtener el mínimo coste de desarrollo una vez especificada la máxima duración admisible del mismo. La resolución de este problema se basa en establecer una correlación entre tiempo y coste para cada actividad, lo que supone asumir una serie de hipótesis cuya validez se discutirá en próximos apartados.

Puede notarse que existe cierta confusión respecto a las anteriores denominaciones, ya que algunos autores consideran la utilización de algoritmo de programación lineal (**método MCE**) como parte inseparable del **CPM**, mientras otros lo identifican con el PERT/Costes (Thierauf 1972).

En general, existe una abundante bibliografía sobre los métodos PERT, CPM y sus extensiones, pero ya desde sus orígenes (similitud entre PERT y CPM) han existido los problemas para delimitar el ámbito y contenido de cada método.

Relaciona los elementos de la columna Derecha con la columna Izquierda

El método CPM

1

Método MCE

2

Permite la introducción de una relación entre el coste de ejecución en el tiempo normal previsto y el coste necesario para desarrollar la actividad con ritmo intensivo, suponiendo una relación lineal entre ambos puntos extremos e incorporando algoritmos heurísticos para obtener la mayor reducción de tiempo posible dado un coste máximo admisible.

Aplica algoritmos de programación lineal mixta entera (dado que aparecen tanto variables continuas como enteras, tales como el número de recursos humanos) para obtener el mínimo coste de desarrollo una vez especificada la máxima duración admisible del mismo.

Puede notarse que existe cierta confusión respecto a las anteriores denominaciones, ya que algunos autores consideran la utilización de algoritmo de programación lineal (método MCE) como parte inseparable del CPM, mientras otros lo identifican con el PERT/Costes (Thierauf 1972).

La documentación técnica como herramienta de seguimiento de la planificación.

La **Gestión de proyectos** acarrea una serie de riesgos que han de tenerse en cuenta. En este post enumeramos los problemas más comunes y riesgos que podemos encontrarnos:

- No existencia de unas normas de gestión del proyecto. Desarrollar unas Normas de Gestión del Proyecto adecuado que recoja la definición del

proyecto, la planificación global, las decisiones tomadas en la reunión Kick-Off (principales mecanismos de gestión y ejecución del proyecto), responsabilidades y línea base aprobada para guiar la ejecución y control del proyecto.

- Planificaciones no ajustadas a la realidad debido a: estimaciones erróneas, movilidad de los recursos, insuficiencia de recursos (cantidad y personal), fechas prefijada y cambios de requisitos. [severidad: alta] Utilizar los procedimientos, herramientas, técnicas y base de datos históricos adecuados para estimar el tamaño, el esfuerzo, el coste y los recursos necesarios para la planificación y seguimiento de proyectos.
- No se tiene una visión del estado real de los proyectos y por ende ni visibilidad de la evolución del proyecto. Establecer hitos principales y reuniones de seguimiento periódicas para realizar informes del progreso del proyecto.
- Retraso en la entrega de las aplicaciones e, incluso, reducción en las funcionalidades de las mismas. Realizar inventario de análisis de riesgos y problemas y mantenerlos actualizados durante todo el desarrollo del proyecto.
- Falta de procedimientos de comunicación entre los grupos de las distintas direcciones en el ámbito de un proyecto. Desarrollar un Plan de Comunicación en el que se establezcan las necesidades de comunicación durante el desarrollo del proyecto.
- Falta de una matriz de escalado. Identificar donde o a quien escalar antes de un bloqueo en el proyecto.
- Falta de compromiso del cliente para el proyecto. Validar el plan de trabajo con el cliente, estableciendo compromisos del cliente para el proyecto y definiendo su participación dentro del mismo.
- Falta de cumplimiento de acuerdos establecidos durante reuniones con el cliente. Generación de agendas y actas para cada reunión con el cliente, y validación de las actas por parte del cliente.
- Existencia de personal del equipo de trabajo, sin que tenga establecida su participación en el proyecto. Definición de roles y responsabilidades de todos los miembros de equipo de acuerdo al plan de trabajo definido
- Documentación del progreso del proyecto obsoleto o sin cumplir con los estándares establecidos para su realización. Actualización periódica de la información en la carpeta de proyecto y revisión de los estándares de nomenclatura de todos los archivos.
- Pérdida de contacto con clientes potenciales, para futuros proyectos. Mantenimiento del contacto con el cliente tras el proyecto, especialmente si existen oportunidades de continuidad (actualización de la BBDD de contactos).

- Modificaciones del alcance en el transcurso del proyecto. Informar al cliente de los próximos pasos y de las tareas pendientes para gestionar expectativas y anticiparse. Mantener una comunicación constante de los responsables del proyecto – Reuniones de seguimiento. Revisar periódicamente las actas de reunión de kick-off y propuesta inicial con todo el equipo de proyecto.

Conflictos entre actividades.

Planificar requiere que el **gerente de proyecto** decida qué gente, recursos y presupuestos se requieren para completar el mismo.

Usted debe definir qué actividades se requieren para producir los productos, utilizando técnicas tales como Estructura Analítica de Proyectos.

Usted debe estimar los tiempos y los esfuerzos requeridos para cada actividad, las dependencias entre actividades y luego decidir un programa realista para completarlas. Involucre al equipo de proyecto en la estimación de la duración de las actividades. Establezca hitos que indiquen fechas críticas durante el desarrollo del proyecto. Escríbalas en su planificación. Pida a los accionistas principales que la revisen y presten su consentimiento al plan.

Factores de productividad.

De forma continuada el equipo va mejorando su forma de trabajar, pero además esta se va simplificando conforme avanzan en el desarrollo del proyecto. Los componentes del grupo de trabajo sincronizan su trabajo y van conociendo el avance del trabajo de los demás así como diariamente resuelven los problemas que surjan para conseguir el objetivo de cada iteración.

En este sentido hemos de entender que la **comunicación** y la **adaptación** a las diferentes necesidades son clave. De esta manera no se podrán realizar tareas innecesarias y se han de evitar en todo caso las ineficiencias.

La estimación de esfuerzo, así como la optimización de las tareas par a completar un requisito establecido la realizarán las personas que van a desarrollar la iteración y por tanto van a cumplir los requisitos. Con las iteraciones cortas, la precisión de las estimaciones aumentan.

Con este **método de trabajo**, se asegura la calidad del producto, puesto que de manera sistemática y objetiva, se plasmará el nivel de satisfacción del cliente, y si los requisitos han sido cumplidos y están listos para ser utilizados respecto de la calidad interna del producto.

Métricas de productividad del software.

Una vez que se disponga de la **lista de oportunidades**, habrá que trabajar en la localización de la más importante. Debería empezar identificando el beneficio que puede generarse si se mejora el proceso, si se arregla la deficiencia detectada o si se crea un nuevo servicio.

Normalmente, las razones por las que una empresa decide tomar un nuevo camino suele ser una o una combinación de las siguientes:

- Para mejorar el beneficio.
- Para reducir costes.
- Para mejorar la productividad.
- Para reducir el riesgo.

Herramientas que mejoran la productividad.

Ahora que ya tenemos una idea de qué **oportunidades** son más convenientes para los intereses generales y del beneficio que se puede obtener si se llevan a cabo, habrá que decidir cuáles llevar a cabo. Quizá no se haya descubierto aún una oportunidad de tal importancia que sea imperativo llevarla a cabo inmediatamente. Más probable, no obstante, es la circunstancia de que se disponga de distintas opciones y que se tenga que decidir cuál de ellas es la más importante.

Primero, habrá que priorizar las oportunidades en orden de su beneficio potencial. Después, habrá que averiguar el grado de desarrollo en que se encuentra el proyecto. Para ello, se escribirá en un par de líneas cómo resolver el problema. Si no se tiene idea alguna de cómo hacerlo, el proyecto comenzará siendo un trabajo de investigación acerca de cómo resolver el problema.

El siguiente paso será cuantificar el **beneficio** que obtendrá por medio de los **números**. En caso de tener claro que lo que se quiere es ahorrar costes, la solución parece sencilla. Si pronosticamos que reducir el papeleo redundará en un ahorro de los costes mensuales en papel, consumibles de impresoras y fotocopadoras, podemos obtener una cifra total del ahorro que lograremos. Cuando se trata de predecir incrementos en ventas o en la productividad, los cálculos se vuelven un tanto más complejos. No debe cometerse el error de representar estos valores en términos de caja, sino de expresarlos en términos de beneficio tanto como sea posible, de forma que después se pueda ir a hablar sobre el proyecto con el responsable adecuado.

Después, se deben **priorizar los proyectos** en términos de los que sean más sencillos o más baratos de implementar. Dependiendo de cada situación, las preguntas relativas a la facilidad o la posibilidad de realización tendrán una mayor o menor importancia. Si se puede asignar personal de la organización al proyecto, se obtendrá un mejor baremo al respecto de la rapidez en que se podrá realizar el proyecto.

El rango completo de herramientas y buenas prácticas que pueden ser útiles para el descubrimiento de los proyectos dentro de una organización incluyen:

- Obtención de ideas.

- Gestión del portafolio.
- Organización.
- Planificación de los recursos.

Si se puede asignar personal de la organización al proyecto, se obtendrá un mejor baremo al respecto de la rapidez en que se podrá realizar el proyecto.

Verdadero.

Falso.

Disponibilidad de los recursos.

Para una **actividad del proyecto**, la estimación de los recursos de las actividades del cronograma implica especificar los recursos (personas, equipos y/o materiales) necesarios para realizarla, qué cantidad de cada recurso se usará y cuándo estarán disponibles estos recursos. El proceso de Estimación de Recursos de las Actividades se coordina estrechamente con el proceso de Estimación de Costes.

Muchas organizaciones cuentan en su seno, como mínimo, con un par de gestores de proyectos. La estructura organizativa desempeña un importante papel en la efectividad de la gestión. Si la estructura organizativa no se implica, es muy probable que los proyectos fracasen.

La experiencia y el entrenamiento del equipo de desarrollo.

En el proceso de **Desarrollar el Equipo del Proyecto** se potencian tanto las competencias de cada uno de los miembros del equipo como las interacciones con los demás miembros a fin de potenciar el trabajo en equipo y el rendimiento del proyecto. Los objetivos de este procedimiento incluyen:

- Mejorar la motivación, las habilidades y la capacidad de los miembros del equipo a fin de aumentar su competencia para completar las actividades del proyecto.
- Mejorar los sentimientos de confianza y cohesión entre los miembros del equipo con el fin de incrementar su productividad mediante un mejor trabajo en equipo.
- Crear una dinámica e interactiva cultura de cooperación, trabajo en equipo y capacidad para compartir conocimiento y experiencia.

Los esfuerzos para el desarrollo del equipo son más beneficiosos cuando se realizan en las fases tempranas del proyecto, pero deberían tener lugar durante todo su ciclo de vida.

A este proceso también se le denomina **“team building”** y algunos trucos de esta dinámica de construcción de un equipo a partir de un grupo de desconocidos:

- Ofrecer “retos” al equipo.
- Recompensar y reconocer al equipo de proyecto.
- Promover el conocimiento y la comunicación entre los miembros del equipo.
- Ser un facilitador de las relaciones interpersonales.
- Buscar el compromiso profesional de los miembros del equipo.
- Situar físicamente al equipo en un mismo lugar y durante un cierto tiempo, de forma que los miembros puedan comunicarse fácilmente, establecer relaciones personales y conseguir compromisos profesionales efectivos.



Gestionar el Equipo del Proyecto

El proceso de Dirigir el Equipo del Proyecto implica hacer un seguimiento del rendimiento de los miembros del equipo, recoger de ellos información de retroalimentación, resolver incidencias, polémicas y coordinar los cambios necesarios para mejorar el rendimiento del proyecto. El equipo de dirección del proyecto debe observar el comportamiento del equipo, gestionar cualquier conflicto que se presente y evaluar el rendimiento de los miembros del equipo.

COCOMO (Constructive Cost Model).

Es un **modelo matemático** de base **empírica** utilizado para estimación de costos¹ de software. Incluye tres submodelos, cada uno ofrece un nivel de detalle y aproximación, cada vez mayor, a medida que avanza el proceso de desarrollo del software: básico, intermedio y detallado.

Recuerda

[[[Elemento Multimedia]]]

ieeditorial

Autoevaluación

Indica si el siguiente enunciado es verdadero o falso: Un proyecto es un esfuerzo temporal, único y progresivo, emprendido para crear un producto o un servicio también único...

- a) No.
- b) Si.
- c) Dependiendo del proyecto será temporal o no.

Indica si el siguiente enunciado es verdadero o falso: Que un proyecto sea temporal se refiere a que no tiene ni un comienzo definido ni un final definido...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: El portafolio se refiere al conjunto de programas, subprogramas, proyectos, operaciones...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Los proyectos se dan cuando existen actividades nuevas, incluyendo mejoras nuevas...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: El término “dirección de proyectos” se convierte a veces en “dirección por proyectos”...

- a) Si, a veces.
- b) Si.

c) No, nunca.

ieeditorial

Soluciones :

1:

b) Si.

2:

a) Falso.

3:

b) Verdadero.

4:

b) Verdadero.

5:

b) Si.

EVALUACIÓN.

- Introducción
- Objetivos
- Mapa Conceptual
- Test de conocimientos.
- Trabajo voluntario.
- Trabajo obligatorio.
- Recuerda
- Autoevaluación

Introducción

En este apartado vamos a desarrollar los conocimientos ya adquiridos, con ejercicios resueltos.

ieeditorial

Objetivos

Aprender a realizar un estudio detallado, elaborando y especificando los requisitos funcionales de un sistema a desarrollar.

ieeditorial

Mapa Conceptual

[[[Elemento Multimedia]]]

ieeditorial

Test de conocimientos.

Una biblioteca tiene copias de libros. Estos últimos se caracterizan por:

Nombre.

País.

Año.

Todas son correctas.

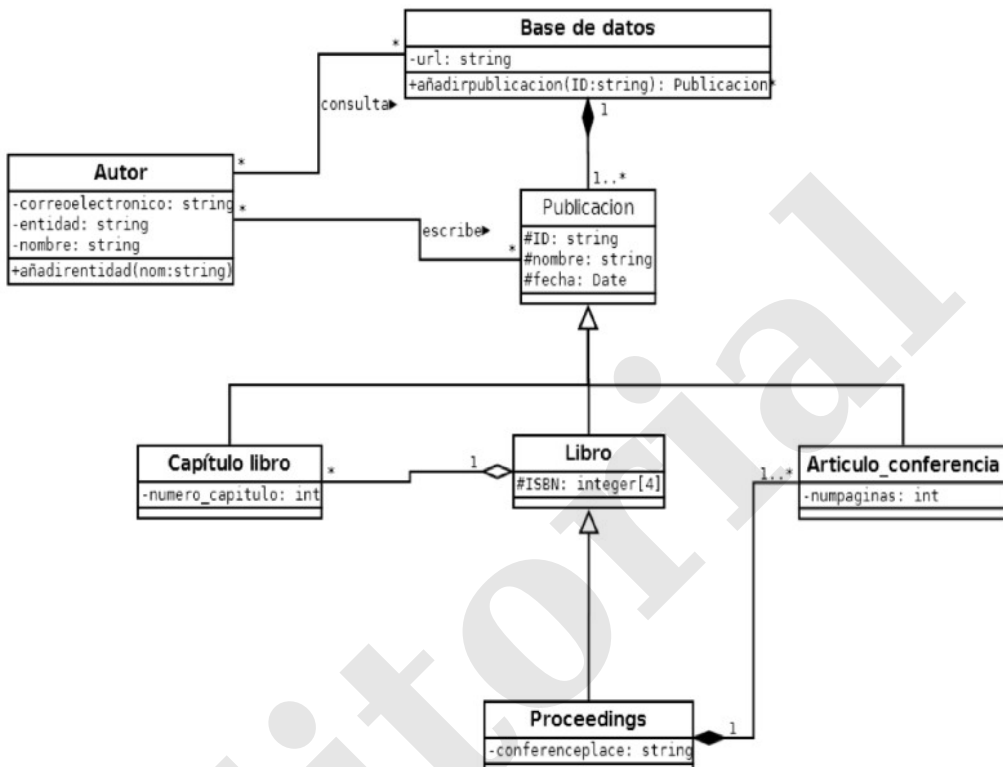
Trabajo voluntario.

Se desea realizar el modelo de un **sistema de gestión bibliográfica**. Para ello se consideran los siguientes conceptos:

- Hay Bases_de_datos que tienen un atributo que es su “url”, y un método “anadir_publicación” (que devuelve un puntero a “Publicación” y tiene un parámetro que es un string).
- Las bases de datos se componen de “Publicaciones”.
- Una publicación tiene dos atributos: “nombre” (de tipo string) y “fecha” (de tipo Date -suponerla ya implementada).
- Hay varios tipos de publicaciones: “Articulo_de_conferencia”, “Capitulo_de_libro” y “Libro”. A su vez hay un tipo de libro que son los “Proceedings”, que son libros compuestos por varios artículos de una misma conferencia. Los artículos de conferencia tienen un atributo: “num_paginas” (int). Los capítulos de libro tienen un atributo: “num_capitulo” (int). Los libros tienen un atributos: “ISBN” (array de 4 enteros). Los proceedings tienen un atributo: “lugar_conferencia” (string).
- Los libros pueden contener capítulos de libro.
- Los proceedings están compuestos por artículos de conferencia.
- También hay “Autores”, que tienen tres atributos: “nombre” (string), “correo_electronico” (string) y “entidad” (string), y un método: “añadir_entidad” (no devuelve nada, y tiene un parámetro de tipo string).
- Los autores pueden escribir artículos y consultar bases de datos.

Utilizando herencia siempre que se pueda, realizar un diseño UML de clases del sistema indicando:

Las clases que se consideren necesarias con sus atributos y métodos. El tipo de privacidad más idóneo. Las relaciones entre clases: asociación, dependencia, herencia, agregación, composición, etc. La multiplicidad en las relaciones entre clases. Al menos una de las clases debe ser abstracta.



Las clases que se consideren necesarias con sus atributos y métodos. El tipo de privacidad más idóneo. Las relaciones entre clases: asociación, dependencia, herencia, agregación, composición, etc. La multiplicidad en las relaciones entre clases. Al menos una de las clases debe ser:

Dinamizada.

Organizada.

Abstracta.

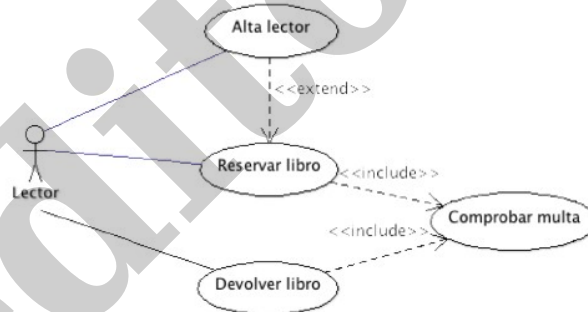
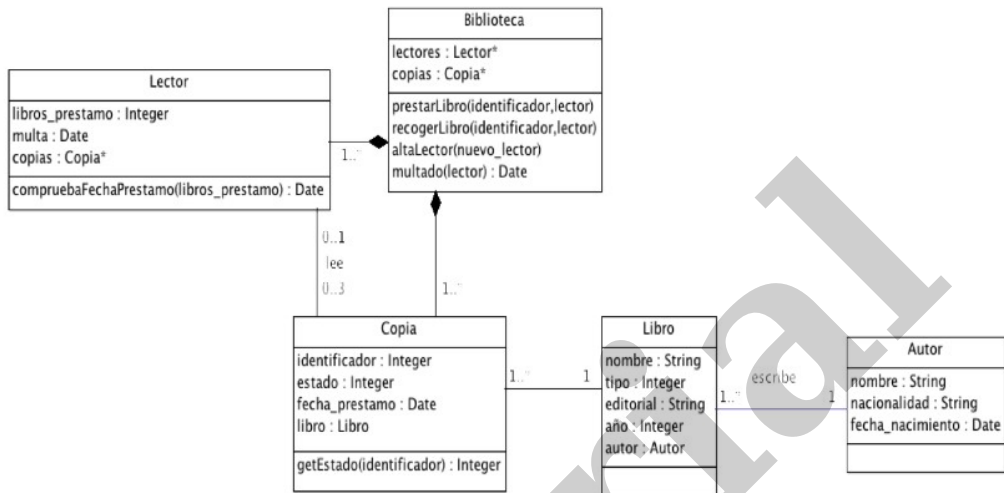
Trabajo obligatorio.

Una **biblioteca** tiene copias de libros. Estos últimos se caracterizan por su nombre, tipo (ingeniería, literatura, informática, historia...), editorial, año y autor.

Los autores se caracterizan por su nombre, nacionalidad y fecha de nacimiento. Cada copia tiene un identificador, y puede estar en la biblioteca, prestada, con retraso o en reparación. Los lectores pueden tener un máximo de 3 libros en préstamo. Cada libro se presta un máximo de 30 días, y por cada día de retraso, se impone una “multa” de dos días sin posibilidad de coger un nuevo libro.

Realiza un diagrama de clases y añade los métodos necesarios para realizar el **préstamo y devolución de libros**.

Realiza un diagrama de casos de usos.



Los autores se caracterizan por su nombre, nacionalidad y fecha de nacimiento. Cada copia tiene un identificador, y puede estar en la biblioteca, prestada, con retraso o en reparación. Los lectores pueden tener un máximo de 3 libros en préstamo. Cada libro se presta un máximo de 30 días, y por cada día de retraso, se impone una “multa” de dos días sin posibilidad de coger un nuevo libro.

Verdadero.

Falso.

Recuerda

[[[Elemento Multimedia]]]

ieeditorial

Autoevaluación

¿Qué diferencia existe entre un actor y una clase?

- a) El actor representa entidades modeladas dentro del sistema, la clase también.
- b) La clase representa entidades externas al sistema al igual que el actor.
- c) El actor representa entidades externas al sistema, sin embargo, la clase representa entidades modeladas dentro del mismo.
- d) Ninguna de las respuestas anteriores es correcta.

Cuál de las siguientes afirmaciones pertenece a un modelo:

- a) Un modelo es una representación gráfica de un conjunto de elementos.
- b) Un modelo es una abstracción de un sistema que, provisionalmente, puede ser incompleto.
- c) Un modelo debe ser preciso, no puede estar modelado desde distintos puntos de vista complementarios.
- d) Ninguna de las anteriores.

¿Cuándo un requisito debe ser Funcional?

- a) Cuando documenta una operación o actividad que el sistema no puede desempeñar.
- b) Cuando describa una función o componente del sistema de software.
- c) Cuando se necesite ingresar datos en el sistema.
- d) Todas las anteriores son correctas.

¿Cuál de las siguientes afirmaciones es cierta en relación con la composición en diagramas de clases?

- a) Es un tipo especial de asociación que representa una relación

todo-parte, transitiva y asimétrica.

b) La multiplicidad sólo puede ser 0..1 ó 1..1

c) Es un tipo especial de agregación compartida.

d) Composición significa encapsulamiento y acceso restringido.

¿Qué opción no se corresponde con los criterios para la selección de una arquitectura?

a) Extensibilidad.

b) Modificabilidad.

c) Simplicidad.

d) Ineficiencia.

Soluciones :

1:

c) El actor representa entidades externas al sistema, sin embargo, la clase representa entidades modeladas dentro del mismo.

2:

b) Un modelo es una abstracción de un sistema que, provisionalmente, puede ser incompleto.

3:

b) Cuando describa una función o componente del sistema de software.

4:

b) La multiplicidad sólo puede ser 0..1 ó 1..1

5:

b) Modificabilidad.

IFCT055PO GESTIÓN DE PROYECTOS DE
DESARROLLO DE SOFTWARE
Glosario

Glosario

A

Arquetipo: Provee a Maven de plantillas para la generación de proyectos. Un arquetipo conforma un patrón o modelo sobre el que podremos crear cosas del mismo tipo. Los arquetipos ayudan a crear plantillas de proyectos para los usuarios, que a su vez podrán generar versiones parametrizadas de esas plantillas según sus necesidades.

B

Base de datos: Una base de datos o banco de datos (abreviada como BD o b. d.) es un conjunto de datos de un mismo contexto y que se almacenan de forma sistemática para su uso posterior.

C

Clase: En el contexto de la programación orientada a objetos, una clase es una estructura que se usa como un modelo (o plantilla) para crear objetos (o instancias) de ese tipo. El modelo describe un conjunto de estados y comportamientos que comparten todos los objetos de la misma clase.

Código: En el ámbito de la informática, el código fuente es el texto desarrollado en un lenguaje de programación concreto y que para ser ejecutado en un ordenador debe ser compilado o interpretado.

Cuantificable: El proceso de convertir un objeto en un conjunto de valores discretos, medibles, como puede ser un número entero. En función del campo de estudio este término puede tomar diferentes definiciones.

H

Herramientas CASE (Computer Aided Software Engineering): Son aplicaciones informáticas que tiene como objetivo aumentar la productividad en el Desarrollo de software reduciendo su coste en tiempo y de dinero. Las herramientas CASE prestan su ayuda en todos los aspectos del ciclo de vida de desarrollo del software. Son de gran utilidad en tareas como el diseño de proyectos, cálculo de costes, implementación automática de parte del código a partir del diseño, compilación automática, documentación, detección de

errores, etc.

I

Ingeniería de software: Aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software, así como también el estudio de estos enfoques. Es decir, la aplicación de procesos de ingeniería al software. Es la aplicación de la ingeniería al software, porque integra matemáticas, ciencias de la computación y prácticas que originalmente se utilizaban en ingeniería.

M

Middleware: Es una capa de lógica de intercambio de información entre aplicaciones. Consiste en un programa que da soporte a aplicaciones en cuanto a la comunicación con otras aplicaciones, sistemas operativos, redes, o dispositivos. De esta forma se simplifica el trabajo de los desarrolladores de sistemas distribuidos a la hora de trabajar con conexiones y de sincronizar datos.

Metodología: Conjunto de procedimientos racionales que se utilizan para alcanzar una serie de objetivos definidos en una investigación científica, una exposición doctrinal o tareas que requieran habilidades, conocimientos o cuidados específicos. La metodología también se puede definir como el estudio o elección de un método pertinente para un determinado objetivo.

Metodologías ágiles: Permiten adaptar la forma de trabajo a las condiciones del proyecto, obteniendo flexibilidad e inmediatez en la respuesta, adaptando el proyecto y su desarrollo a las circunstancias específicas del entorno. Las empresas que apuestan por esta metodología gestionan sus proyectos de forma eficaz, reduciendo los costes e incrementando su productividad.

O

Objeto: En el contexto de la programación orientada a objetos (POO, OOP en inglés), un objeto se define como la unidad que realiza las tareas de un programa en tiempo de ejecución. También a un nivel más básico se define como la instancia de una clase.

P

Programación: Es el proceso de diseñar, codificar, depurar y mantener el código fuente de programas computacionales. El código fuente es escrito en un lenguaje de programación.

Pruebas de software: Verificación del comportamiento de un programa en un conjunto finito de casos de prueba. Estos casos de prueba son debidamente seleccionados de entre (por lo general) infinitas ejecuciones de dominio. Esta verificación se lleva a cabo comparando el resultado obtenido con el comportamiento esperado. También se pueden definir como una serie de actividades que se realizan con el propósito de encontrar los posibles fallos de implementación, calidad o usabilidad de un programa u ordenador, probando el comportamiento del sistema.

R

Rentable: Capacidad que tiene algo para generar suficiente utilidad o beneficio.

S

Sistema: Un sistema (del latín *systema*, proveniente del griego σύστημα) es un objeto compuesto cuyos componentes se relacionan con al menos algún otro componente. Puede ser material o conceptual.

Software: Conjunto de programas, instrucciones y reglas informáticas para ejecutar ciertas tareas en un ordenador.

Scriptlet: En el ámbito de JSP (Java Server Pages), es un trozo de código escrito en java que se embebe en el interior del código HTML de la página JSP. El scriptlet sería todo lo que se encuentre entre las etiquetas `<% %>`.

STUB: En el ámbito de la programación distribuida, es un código que durante una llamada RPC (Remote Procedure Call) se encarga de la conversión de los parámetros pasados entre el cliente y el servidor.

U

UML (Unified Modeling Language): Es un lenguaje gráfico para visualizar, especificar y documentar cada una de las partes que comprende el desarrollo de software. UML proporciona una forma de modelar elementos conceptuales como los procesos de negocio y funciones de sistema, además de cosas

concretas como escribir clases en un lenguaje determinado, esquemas de base de datos y componentes de software reusables.

T

Tecnología: Es el conjunto de conocimientos técnicos, ordenados científicamente, que permiten diseñar y crear bienes y servicios que facilitan la adaptación al medio ambiente y satisfacer tanto las necesidades esenciales como los deseos de las personas.

ieeditorial

ieeditorial

IFCT055PO GESTIÓN DE PROYECTOS DE
DESARROLLO DE SOFTWARE
Bibliografía

ieeditorial

Bibliografía

Bibliografía básica

- Contexto de la gestión del proyecto Drudis, A. (2018).
- Gestión de proyectos. Cómo planificarlos, organizarlos y dirigirlos. Barcelona: Gestión 2000.
- Proceso de la gestión del proyecto Badawy, M.K. (2017).
- Temas de gestión de la innovación para científicos e ingenieros. Madrid: Clásicos Cotec. Pressman, R.S. (1993).
- Ingeniería del software. Un enfoque práctico. Madrid: McGraw-Hill. Definición y planificación del proyecto Amescua, A. y otros (1995).
- Ingeniería del software de gestión. Análisis y diseño de aplicaciones. Paraninfo. Bentley, C. (1983).
- Computer Project Management. John Wiley & Sons. Boehm, B.W. (1989).
- Software Risk Management. IEEE Computer Society Press. Kendall, K.E.; Kendal, J.E. (1997).
- Análisis y diseño de sistemas. Madrid: Prentice Hall. Kruse, R.L. (1984).
- Data Structures and Program Design. Prentice Hall. Macro, A. (1990).
- Software Engineering: Concepts and Management. Prentice Hall. Pressman, R.S. (1993).
- Ingeniería del software. Un enfoque práctico. Madrid: McGraw-Hill. Shatz, S.; Wang, J.P. (1989).
- Distributed Software Engineering: Tutorial. IEEE Computer Press. Previsión de los gastos y el tiempo Boehm, B.W. (1981).
- Software Engineering Economics. Englewood Cliffs: Prentice Hall. DeMarco, T. (1982).
- Controlling Software Projects. Yourdon Press. Fenton, N.E.; Pfleeger, S.L. (1997).
- Software Metrics. A Rigorous and Practical Approach. McConnel, S. (1997).
- Desarrollo y gestión de proyectos informáticos. McGraw-Hill. Dirección y coordinación del equipo de proyecto Druker, P. (1975).
- Management. Heinemann. Posner, B.Z.; Randolph, W.A. (1993).
- Gerencia de proyectos. Cómo dirigir exitosamente equipos de trabajo. McGraw-Hill. Weinberg, G.M. (1971).
- The Psychology of Computer Programming. Nueva York: Van Nostrand. Seguimiento y control de la ejecución del proyecto; conclusión del proyecto Beizer, B. (1990).
- Software Testing Techniques. Van Nostrand Reinhold. Dunn, R. (1984).

- Software Defect Removal. McGraw-Hill.

Bibliografía complementaria

- Contexto de la gestión del proyecto Bacharach, S.B.; Lawler, J.E. (1981).
- Power and Politics in Organizations. Jossey-Bass.
- Definición y planificación del proyecto Gupta, R.; Horowitz, E. (ed.), (1991). Object-Oriented Databases with Applications to CASE, Networks, and VLSI CAD. Prentice Hall.
- Lee, E. (1990, mayo).
- "User-Interface Development Tools". IEEE Software (pág. 23-31).
- Mellor, S.J.; Ward, P.T. (1985). Structured Development for Real-Time System. Prentice Hall.
- Meyer, B. (1998).
- Construcción de software orientado a objetos. Prentice Hall. Muller, P.A. (1997).
- Modelado de objetos con UML. Eyrolles: Gestión 2000. Smith, S.L.; Mosier, J.N. (1986). "Guidelines for Designing User Interface Software".
- The Mitre Corporation (report EDS-TR-86-278). National Technical Information Service. Sommerville, I. (1988).
- Ingeniería de software. Addison-Wesley Iberoamericana. Software Quality Engineering. Testing Tools Reference Guide, version 7.0. Yourdon, E. (1993).
- Análisis estructurado moderno. Prentice Hall. Dirección y coordinación del equipo de proyecto Baker, F.T. (1972). "Chief Programmer Team Management of Production Programming".
- IBM System Journal (vol. 11, núm. 1, pág. 56-73). Seguimiento y control de la ejecución del proyecto.
- Conclusión del proyecto Ntafos, S.C. (1988). "A Comparison of Somme Structural Testing Strategies".
- IEEE Trans. Software Engineering (vol. 14, núm. 6, pág. 868-874). Ragland, B. (1995). "Measure, Metric or Indicator: What's the Difference?". Crosstalk (vol. 8, núm. 3, pág. 29-30).

ieeditorial

IFCT055PO GESTIÓN DE PROYECTOS DE
DESARROLLO DE SOFTWARE
Examen

Examen

Indica si el siguiente enunciado es verdadero o falso: El portafolio se refiere al conjunto de programas, subprogramas, proyectos, operaciones...

- a) No, esa es la definición de programa.
- b) No, esa es la definición de proyecto.
- c) Si.

Indica si el siguiente enunciado es verdadero o falso: Los test de conocimientos los encontramos en los apartados de cada tema Autoevaluación...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: El trabajo obligatorio comienza al igual que el voluntario con la idea de otro programa, pero esta vez habrá que crear un diagrama UML menos completo ...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Como trabajo voluntario, se pueden realizar una serie de pasos para aplicar los conocimientos aprendidos en este curso...

- a) Falso.
- b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Un proyecto es un esfuerzo temporal, único y progresivo, emprendido para crear un producto o un servicio también único...

- a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Que un proyecto sea temporal se refiere a que tiene un comienzo definido y un final definido...

a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Los proyectos se dan cuando existen actividades nuevas, incluyendo mejoras nuevas...

a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: El término "dirección de proyectos" se convierte a veces en "dirección por proyectos"...

a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: Traducir un diseño a código no tiene porque ser la parte más complicada...

a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: La especificación es la tarea de escribir de forma no detallada el software a ser desarrollado, en una forma matemáticamente rigurosa...

a) Falso.

b) Verdadero.

Indica si el siguiente enunciado es verdadero o falso: La tarea de mantenimiento se encarga de mantener y mejorar el software para

solventar errores descubiertos y tratar con nuevos requisitos...

- a) Falso.**
- b) Verdadero.**

Indica si el siguiente enunciado es verdadero o falso: El ciclo del vida del software es una estructura aplicada al desarrollo de un producto de software...

- a) Falso.**
- b) Verdadero.**

Indica si el siguiente enunciado es verdadero o falso: El ciclo de vida del software fue inicialmente propuesto por Royce en 1975...

- a) Falso.**
- b) Verdadero.**

Las etapas de un proyecto informatico son: Iniciación, Planificación, Ejecucion, Control, Cierre.

- a) Verdadero.**
- b) Falso.**

Indica si el siguiente enunciado es verdadero o falso: Las pruebas de implantación validan que un sistema cumple con el funcionamiento esperado y permitir al usuario de dicho sistema que determine su aceptación.

- a) Falso.**
- b) Verdadero.**

Soluciones :

1:

c) Si.

2:

b) Verdadero.

3:

a) Falso.

4:

b) Verdadero.

5:

b) Verdadero.

6:

b) Verdadero.

7:

b) Verdadero.

8:

b) Verdadero.

9:

b) Verdadero.

10:

a) Falso.

11:

b) Verdadero.

12:

b) Verdadero.

13:

a) Falso.

14:

a) Verdadero.

15:

a) Falso.

ieeditorial

ieeditorial